🌿 KodeQuiz Book

Buku 1 — Excel & SQL

18 bab · 201 soal dengan pembahasan

Cara memakai buku ini

1. Baca materi tiap bab (±3–5 menit).
2. Kerjakan *Uji Diri* tanpa melihat kunci — di kepala atau di kertas.
3. Cocokkan dengan *Kunci & Pembahasan* di akhir bab. Baca juga kenapa pilihan lain salah.
4. Soal yang salah? Tandai, lalu ulangi besok di aplikasi lewat Ulangan Pintar.

Dibuat 1 November 2026

Daftar Isi

1. Excel

1. Dasar Rumus & Referensi ($)
2. SUMIFS & COUNTIFS — Menjumlah dan Menghitung dengan Syarat
3. XLOOKUP (dan VLOOKUP) — Mengambil Data dari Tabel Lain
4. IF Bertingkat, IFS, AND, OR
5. Pivot Table — Meringkas Ribuan Baris dalam Detik
6. Conditional Formatting — Membuat Data Bicara
7. Teks & Membersihkan Data
8. Tanggal & Waktu — Analisis per Periode
9. INDEX-MATCH & Dynamic Array (FILTER, UNIQUE, SORT)
10. Error, IFERROR & Data yang Rapi

2. SQL

1. SELECT, WHERE, ORDER BY — Mengambil Data
2. Agregat, GROUP BY & HAVING — Meringkas Data
3. JOIN — Menggabungkan Tabel
4. Subquery — Query di Dalam Query
5. CASE WHEN — Logika IF di dalam SQL
6. NULL & Fungsi Bawaan (Teks, Angka, Tanggal)
7. CTE (WITH) & UNION — Query Bertahap yang Rapi
8. Window Function — Peringkat, Total Berjalan & Pertumbuhan
9. Lampiran: Tantangan Lab SQL

BAGIAN 1

Excel

SUMIFS, XLOOKUP, pivot, teks & tanggal, INDEX-MATCH, FILTER

10 bab · 117 soal

EXCEL · BAB 1

Dasar Rumus & Referensi ($)

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Semua latihan Excel di sini memakai **data penjualan yang sama**. Perhatikan huruf kolom (A–F) dan nomor baris — kamu akan sering memakainya.

📄 **Sheet: Penjualan**

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

Rumus selalu diawali `=`. Lima fungsi pertama yang wajib hafal:

```
=SUM(F2:F11)      → menjumlahkan  (45.250.000)
=AVERAGE(F2:F11)  → rata-rata     (4.525.000)
=COUNT(F2:F11)    → banyaknya angka (10)
=MAX(F2:F11)      → terbesar      (14.000.000)
=MIN(F2:F11)      → terkecil      (750.000)
```

Referensi relatif vs absolut — ini yang paling sering bikin laporan salah:

- `F2` = **relatif**: saat rumus di-copy ke bawah, berubah jadi F3, F4, …
- `$J$1` = **absolut**: dikunci, tidak berubah walau di-copy.
- Tekan **F4** (atau Fn+F4) saat mengetik referensi untuk menambah `$`.

🇮🇩 **Penting untuk laptop Indonesia:** jika region-nya Indonesia, pemisah argumen adalah **titik koma**: `=SUM(A1;B1)` . Tutorial berbahasa Inggris memakai koma. Di aplikasi ini ditulis koma, dan jawaban isian menerima keduanya.

🧪 **Praktik di Excel:** buka latihan-excel-penjualan.xlsx → sheet *Bab1-Dasar*. Kerjakan tugasnya, lalu kembali ke sini untuk kuis.

🧠 Uji Diri — 10 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 Sheet: Penjualan

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

1. **Isi sel baru dianggap RUMUS oleh Excel jika diawali tanda…**

   A. #
   B. =
   C. @
   D. $

2. 📄 Data: Penjualan

   Lihat sheet di atas. Hasil rumus ini adalah...

   ```
   =SUM(D2:D11)
   ```

   A. 30
   B. 35
   C. 10
   D. 45.250.000

3. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =COUNT(D2:D11)
   ```

   A. 35
   B. 10
   C. 11
   D. 6

4. **Range F2:F11 artinya...**

   A. Hanya sel F2 dan F11
   B. Semua sel dari F2 sampai F11 (10 sel)
   C. Kolom F sampai kolom ke-11
   D. Sel F2 dikalikan F11

5. **Sel G2 berisi rumus =D2*E2. Kalau G2 di-copy ke G3, rumusnya otomatis menjadi...**

   A. =D2*E2
   B. =D3*E3
   C. =E3*F3
   D. =D3*E2

6. **Tarif pajak ada di sel J1. Kamu ingin setiap baris menghitung Total × pajak, lalu rumusnya di-copy ke bawah. Rumus yang AMAN:**

   A. =F2*J1
   B. =F2*$J$1
   C. =$F$2*J1
   D. =F2*J$2

7. **Saat mengetik referensi (misal A1), tombol keyboard untuk menambah tanda $ dengan cepat adalah F__ (isi angkanya saja)**

   Jawab: _____

8. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =AVERAGE(F2:F11)
   ```

   A. 4.525.000
   B. 45.250.000
   C. 4.700.000
   D. 3.500.000

9. 📄 Data: Penjualan

   Rumus untuk mencari Total TERBESAR di kolom F: =____(F2:F11)

   Jawab: _____

10. **Rumusmu =SUM(F2:F11, F13) ditolak Excel dengan pesan 'There's a problem with this formula'. Penyebab paling mungkin di laptop berbahasa/region Indonesia:**

    A. Pemisah argumen harus titik koma: =SUM(F2:F11; F13)
    B. SUM tidak boleh punya dua argumen
    C. Rumus harus ditulis huruf kecil
    D. Range harus memakai dua titik dua

Kunci & Pembahasan · Excel Bab 1

← Kembali ke soal · Awal bab

1. **B. =** — Excel hanya menghitung isi sel yang diawali '='. Tanpa itu, tulisan SUM(F2:F11) cuma dianggap teks biasa dan tidak dihitung.
   - **A:** '#' muncul di kode error seperti #N/A atau #REF!, bukan pembuka rumus.
   - **C:** '@' di Excel modern adalah operator khusus (implicit intersection), bukan pembuka rumus.
   - **D:** '$' dipakai untuk MENGUNCI referensi (contoh $A$1), bukan memulai rumus.
2. **B. 35** — Kolom D adalah Qty. SUM menjumlahkan semua angka di D2 sampai D11: 2+10+3+5+1+4+1+2+6+1 = 35.
   - **A:** Hitungan meleset — coba jumlahkan lagi pelan-pelan dari D2 sampai D11.
   - **C:** 10 adalah BANYAKNYA baris data. Itu hasil COUNT, bukan SUM.
   - **D:** 45.250.000 adalah total kolom F (Total), bukan kolom D (Qty).
3. **B. 10** — COUNT tidak menjumlahkan — ia MENGHITUNG berapa sel yang berisi angka. Dari D2 sampai D11 ada 10 sel berisi angka, jadi hasilnya 10.
   - **A:** 35 adalah hasil SUM (penjumlahan), bukan COUNT.
   - **C:** 11 kalau baris judul ikut dihitung — tapi D1 berisi teks 'Qty' dan memang tidak ada di range D2:D11.
   - **D:** 6 adalah jumlah kolom tabel, tidak ada hubungannya dengan rumus ini.
4. **B. Semua sel dari F2 sampai F11 (10 sel)** — Tanda titik dua (:) artinya 'sampai'. F2:F11 = F2, F3, F4, … F11 → 10 sel berurutan.
   - **A:** Untuk dua sel terpisah ditulis F2,F11 (pakai koma/titik koma), bukan titik dua.
   - **C:** Angka 11 adalah nomor BARIS, bukan kolom.
   - **D:** Perkalian ditulis dengan *, contoh =F2*F11.

5. **B. =D3*E3** — Ini disebut referensi RELATIF: saat rumus di-copy satu baris ke bawah, semua referensi tanpa $ ikut turun satu baris. D2→D3, E2→E3. Inilah yang membuat kamu cukup menulis rumus sekali lalu tarik ke bawah.
   - **A:** Akan tetap D2*E2 hanya jika referensinya dikunci: =$D$2*$E$2.
   - **C:** Kolom tidak bergeser karena kamu meng-copy ke BAWAH, bukan ke samping.
   - **D:** Kedua referensi relatif, jadi keduanya ikut bergeser — tidak mungkin hanya satu.
6. **B. =F2*$J$1** — $J$1 mengunci kolom J dan baris 1, jadi saat di-copy ke bawah semua baris tetap menunjuk tarif pajak di J1. F2 dibiarkan relatif supaya ikut berganti ke F3, F4, dst.
   - **A:** Saat di-copy ke bawah, J1 akan bergeser jadi J2, J3... yang kosong → hasilnya 0.
   - **C:** Ini mengunci F2, jadi SEMUA baris memakai Total baris 2 — salah besar.
   - **D:** J$2 mengunci baris 2, padahal tarif ada di baris 1.
7. **4** — F4 berputar: A1 → $A$1 → A$1 → $A1 → A1. Di banyak laptop tekan Fn+F4. Ini menghemat banyak waktu saat membuat laporan.
8. **A. 4.525.000** — AVERAGE = total ÷ banyaknya data. Total kolom F = 45.250.000, banyak data = 10, jadi 45.250.000 ÷ 10 = 4.525.000.
   - **B:** Itu TOTAL-nya (hasil SUM), belum dibagi 10.
   - **C:** 4.700.000 adalah total milik Bima saja.
   - **D:** Angka ini tidak muncul dari perhitungan mana pun di data.
9. **max** — MAX mengambil nilai terbesar. Di data ini hasilnya 14.000.000 (Andi, Laptop). Pasangannya MIN untuk nilai terkecil (750.000).

10. **A. Pemisah argumen harus titik koma: =SUM(F2:F11; F13)** — Pengaturan region Indonesia memakai koma sebagai tanda desimal (3,5), sehingga Excel memakai TITIK KOMA (;) untuk memisahkan argumen. Ini jebakan paling sering untuk pemula di Indonesia — tutorial YouTube berbahasa Inggris memakai koma.
    - **B:** SUM boleh menerima banyak argumen, contoh =SUM(A1;B5;C2:C9).
    - **C:** Excel tidak peduli huruf besar/kecil pada nama fungsi.
    - **D:** Satu titik dua sudah benar untuk menulis range.

EXCEL · BAB 2

SUMIFS & COUNTIFS — Menjumlah dan Menghitung dengan Syarat

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Pertanyaan atasan yang paling sering: *"Berapa penjualan Andi di Jakarta?"* Jawabannya: **SUMIFS**.

```
=SUMIFS(range_yang_dijumlah, range_syarat1, syarat1,
range_syarat2, syarat2, …)

=SUMIFS(F2:F11, A2:A11,"Andi")                    → 18.750.000
=SUMIFS(F2:F11, A2:A11,"Andi", B2:B11,"Jakarta")  → 18.000.000
```

COUNTIFS sama polanya, tapi *menghitung banyaknya baris* (tanpa range jumlah di depan):

```
=COUNTIFS(B2:B11,"Jakarta")                  → 4
=COUNTIFS(C2:C11,"Laptop", B2:B11,"<>Jakarta") → 2
```

Jenis syarat yang bisa dipakai:

- Teks persis: `"Andi"` · Tidak sama dengan: `"<>Jakarta"`
- Angka: `">=5000000"`, `"<1000000"` (pakai tanda kutip)
- Dari sel lain: `H2` (TANPA kutip) → laporan jadi dinamis
- Rentang: tulis kolom yang sama dua kali: `F2:F11,">=1000000", F2:F11,"<=5000000"`

AVERAGEIFS = rata-rata dengan syarat, polanya sama persis dengan SUMIFS.

> 💡 Hubungan dengan SQL: `SUMIFS` ≈ `SELECT SUM(total) … WHERE sales='Andi' AND wilayah='Jakarta'` . Kalau paham yang satu, yang lain jadi mudah.

> 🧪 **Praktik:** sheet *Bab2-SUMIFS* di file latihan. Buat tabel ringkasan total per sales memakai SUMIFS yang mengacu ke sel nama (bukan diketik manual).

🧠 Uji Diri — 11 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Penjualan**

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

1. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =SUMIF(A2:A11,"Andi",F2:F11)
   ```

   A. 14.000.000
   B. 18.750.000
   C. 4.000.000
   D. 3

2. **Apa beda utama SUMIF dan SUMIFS?**

   A. SUMIFS bisa banyak syarat, dan range yang dijumlah ditulis PALING DEPAN
   B. Tidak ada bedanya
   C. SUMIF untuk teks, SUMIFS untuk angka
   D. SUMIFS hanya ada di Google Sheets

3. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =SUMIFS(F2:F11, A2:A11,"Andi", B2:B11,"Jakarta")
   ```

   A. 18.750.000
   B. 14.000.000
   C. 18.000.000
   D. 4.000.000

4. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =COUNTIF(B2:B11,"Jakarta")
   ```

   A. 3
   B. 4
   C. 25.200.000
   D. 5

5. 📄 Data: Penjualan

   Hasil rumus ini adalah... (tanda <> artinya 'tidak sama dengan')

   ```
   =COUNTIFS(C2:C11,"Laptop", B2:B11,"<>Jakarta")
   ```

   A. 1
   B. 2
   C. 3
   D. 4

6. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =SUMIFS(F2:F11, F2:F11,">=5000000")
   ```

   A. 34.000.000
   B. 27.000.000
   C. 45.250.000
   D. 4

7. 📄 Data: Penjualan

   Lengkapi rumus untuk menjumlahkan Qty milik Bima: =SUMIFS(____, A2:A11,"Bima")

   Jawab: _____

8. 📄 Data: Penjualan

   Sel H2 berisi nama sales 'Citra'. Rumus SUMIFS yang benar memakai H2 sebagai syarat:

   A. =SUMIFS(F2:F11,A2:A11,"H2")
   B. =SUMIFS(F2:F11,A2:A11,H2)
   C. =SUMIFS(A2:A11,F2:F11,H2)
   D. =SUMIFS(F2:F11,H2,A2:A11)

9. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =SUMIFS(F2:F11, B2:B11,"Bandung", C2:C11,"<>Mouse")
   ```

   A. 10.500.000
   B. 9.000.000
   C. 1.500.000
   D. 7.000.000

10. 📄 Data: Penjualan

    Hasil rumus ini adalah...

    ```
    =AVERAGEIFS(F2:F11, C2:C11,"Laptop")
    ```

    A. 7.000.000
    B. 9.333.333
    C. 28.000.000
    D. 14.000.000

11. 📄 Data: Penjualan

    Kamu ingin menghitung transaksi yang Total-nya antara 1 juta dan 5 juta (termasuk batasnya). Rumus yang tepat:

    A. =COUNTIFS(F2:F11,">=1000000", F2:F11,"<=5000000")
    B. =COUNTIF(F2:F11,"1000000-5000000")
    C. =COUNTIFS(F2:F11,">=1000000 AND <=5000000")
    D. =SUMIFS(F2:F11,">=1000000","<=5000000")

Kunci & Pembahasan · Excel Bab 2

← Kembali ke soal · Awal bab

1. **B. 18.750.000** — SUMIF mencari baris yang kolom A-nya 'Andi' (baris 2, 5, dan 9), lalu menjumlahkan kolom F di baris-baris itu: 14.000.000 + 750.000 + 4.000.000 = 18.750.000.
   - **A:** Ini hanya transaksi Andi yang pertama. SUMIF menjumlahkan SEMUA baris yang cocok.
   - **C:** Ini hanya transaksi terakhir Andi (Monitor).
   - **D:** 3 adalah BANYAKNYA transaksi Andi — itu hasil COUNTIF, bukan SUMIF.

2. **A. SUMIFS bisa banyak syarat, dan range yang dijumlah ditulis PALING DEPAN** — SUMIF(range_syarat, syarat, range_jumlah) → satu syarat, range jumlah di BELAKANG. SUMIFS(range_jumlah, range_syarat1, syarat1, range_syarat2, syarat2, ...) → banyak syarat, range jumlah di DEPAN. Tips: biasakan pakai SUMIFS saja, karena bisa dipakai untuk satu maupun banyak syarat.
   - **B:** Urutan argumennya berbeda — inilah sumber error paling sering.
   - **C:** Keduanya bisa memakai syarat teks maupun angka.
   - **D:** SUMIFS ada di Excel sejak versi 2007.

3. **C. 18.000.000** — Dua syarat harus terpenui SEKALIGUS: Sales = Andi DAN Wilayah = Jakarta. Yang cocok hanya baris 2 (14.000.000) dan baris 9 (4.000.000). Baris 5 milik Andi tapi di Surabaya, jadi tidak ikut. Total = 18.000.000.
   - **A:** Itu total semua transaksi Andi — syarat wilayah Jakarta diabaikan.
   - **B:** Itu hanya baris 2. Baris 9 (Andi, Jakarta, Monitor) juga cocok.
   - **D:** Itu hanya baris 9. Baris 2 juga cocok.

4. **B. 4** — COUNTIF menghitung ada berapa sel di B2:B11 yang isinya 'Jakarta': baris 2, 4, 7, dan 9 → 4 transaksi.
   - **A:** Coba cek lagi baris 7 (Bima, Jakarta, Keyboard).
   - **C:** Itu total RUPIAH penjualan Jakarta (hasil SUMIF), bukan banyaknya transaksi.
   - **D:** Hanya ada 4 baris Jakarta.

5. **B. 2** — Laptop ada di baris 2 (Jakarta), 6 (Bandung), dan 8 (Surabaya). Syarat kedua membuang yang Jakarta, jadi tersisa 2 transaksi.
   - **A:** Ada dua Laptop di luar Jakarta: Bandung (Dewi) dan Surabaya (Citra).
   - **C:** 3 adalah semua transaksi Laptop — syarat <>Jakarta belum diterapkan.
   - **D:** Hanya ada 3 transaksi Laptop di seluruh data.

6. **A. 34.000.000** — Range syarat boleh sama dengan range yang dijumlah. Total yang ≥ 5 juta: 14.000.000 + 6.000.000 + 7.000.000 + 7.000.000 = 34.000.000. Syarat perbandingan ditulis dalam tanda kutip: ">=5000000".
   - **B:** Ada yang terlewat: 14 + 6 + 7 + 7, ada DUA transaksi 7 juta (Dewi & Citra).
   - **C:** Itu total semua transaksi tanpa syarat.
   - **D:** 4 adalah BANYAKNYA transaksi ≥ 5 juta — itu hasil COUNTIFS.

7. **d2:d11** — Argumen pertama SUMIFS selalu range yang dijumlah. Qty ada di kolom D, jadi D2:D11. Hasilnya 10 + 4 + 1 = 15.

8. **B. =SUMIFS(F2:F11,A2:A11,H2)** — Referensi sel ditulis TANPA tanda kutip. Dengan H2 sebagai syarat, kamu cukup mengganti isi H2 (misal jadi 'Dewi') dan hasilnya ikut berubah — ini dasar membuat laporan dinamis. Hasil untuk Citra: 13.000.000.
   - **A:** Dengan kutip, Excel mencari sales yang namanya benar-benar 'H2' → hasilnya 0.
   - **C:** Urutannya terbalik: yang dijumlah harus F (Total), bukan A (nama).
   - **D:** Urutan argumen salah: setelah range jumlah harus range syarat, lalu syaratnya.

9. **B. 9.000.000** — Bandung ada di baris 3 (Mouse, 1.500.000), 6 (Laptop, 7.000.000), dan 11 (Monitor, 2.000.000). Syarat <>Mouse membuang baris 3. Hasil: 7.000.000 + 2.000.000 = 9.000.000.
   - **A:** Itu total Bandung tanpa membuang Mouse.
   - **C:** Itu justru nilai Mouse yang seharusnya DIBUANG.
   - **D:** Monitor Bima di Bandung (2.000.000) juga ikut.

10. **B. 9.333.333** — AVERAGEIFS = rata-rata dengan syarat. Total Laptop: 14.000.000, 7.000.000, 7.000.000 → jumlah 28.000.000 ÷ 3 transaksi = 9.333.333 (dibulatkan).
    - **A:** 7 juta adalah harga satuan Laptop, bukan rata-rata total transaksinya.
    - **C:** Itu jumlahnya (SUMIFS), belum dibagi 3.
    - **D:** Itu hanya transaksi terbesar (Andi, 2 Laptop).
11. **A. =COUNTIFS(F2:F11,">=1000000", F2:F11,"<=5000000")** — Untuk rentang, tulis kolom yang SAMA dua kali dengan dua syarat: ≥ batas bawah dan ≤ batas atas. Yang cocok: 1.500.000, 4.000.000, 1.200.000, 1.800.000, 2.000.000 → hasilnya 5.
    - **B:** Excel tidak mengerti tanda strip sebagai rentang — ia mencari teks '1000000-5000000' persis.
    - **C:** AND tidak bisa ditulis di dalam teks syarat. Pecah jadi dua pasang argumen.
    - **D:** SUMIFS menjumlahkan (bukan menghitung), dan susunan argumennya juga salah.

XLOOKUP (dan VLOOKUP) — Mengambil Data dari Tabel Lain

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Di kantor, data jarang ada di satu tempat. Harga ada di **master produk**, transaksi di sheet lain. XLOOKUP menyatukannya.

📄 **Sheet: Produk**

	A	B	C
1	**Produk**	**Kategori**	**Harga**
2	Laptop	Komputer	7.000.000
3	Mouse	Aksesoris	150.000
4	Monitor	Komputer	2.000.000
5	Keyboard	Aksesoris	300.000

```
=XLOOKUP(yang_dicari, kolom_pencarian, kolom_hasil,
[jika_tidak_ketemu])

=XLOOKUP("Monitor", A2:A5, C2:C5)              → 2.000.000
=XLOOKUP("Mouse",   A2:A5, B2:B5)              → Aksesoris
=XLOOKUP("Printer", A2:A5, C2:C5, "Tidak ada") → Tidak ada
```

Mengambil dari sheet lain + di-copy ke bawah — kunci range tabelnya:

```
=XLOOKUP(C2, Produk!$A$2:$A$5, Produk!$B$2:$B$5)
```

VLOOKUP masih banyak dipakai di file kantor lama, jadi pahami juga:

```
=VLOOKUP("Mouse", A2:C5, 3, FALSE)   → 150.000
          dicari  tabel  kolom ke-3  cocok persis
```

- VLOOKUP hanya bisa mencari di kolom **paling kiri** dan mengambil ke **kanan**.
- Selalu tulis `FALSE`. Tanpa itu, VLOOKUP bisa mengambil data yang salah **tanpa error**.

> ℹ️ XLOOKUP ada di Excel 365/2021 ke atas dan Google Sheets. Kalau kantormu memakai Excel 2016, pakai VLOOKUP atau INDEX+MATCH.

🧪 **Praktik:** sheet *Bab3-XLOOKUP* — isi kolom Kategori dan Harga di sheet Penjualan dengan mengambil dari sheet Produk.

🧠 Uji Diri — 9 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Produk**

	A	B	C
1	**Produk**	**Kategori**	**Harga**
2	Laptop	Komputer	7.000.000
3	Mouse	Aksesoris	150.000
4	Monitor	Komputer	2.000.000
5	Keyboard	Aksesoris	300.000

1. **XLOOKUP dipakai untuk...**

   A. Mencari sebuah nilai di satu kolom, lalu mengambil data yang sejajar di kolom lain
   B. Menjumlah dengan syarat
   C. Mengurutkan data
   D. Membuat grafik

2. 📄 Data: Produk

   Hasil rumus ini adalah...

   ```
   =XLOOKUP("Monitor", A2:A5, C2:C5)
   ```

   A. Komputer
   B. 2.000.000
   C. Monitor
   D. 3

3. 📄 Data: Produk

   Hasil rumus ini adalah...

   ```
   =XLOOKUP("Mouse", A2:A5, B2:B5)
   ```

   A. 150.000
   B. Aksesoris
   C. Komputer
   D. #N/A

4. 📄 Data: Produk

   Hasil rumus ini adalah...

   ```
   =XLOOKUP("Printer", A2:A5, C2:C5)
   ```

   A. 0
   B. #N/A
   C. Printer
   D. Sel kosong

5. **Lengkapi agar muncul teks Tidak ada saat produk tidak ditemukan: =XLOOKUP(H2, A2:A5, C2:C5, ____)**

   Jawab: _____

6. **Kelebihan XLOOKUP dibanding VLOOKUP:**

   A. Bisa mencari ke kiri dan tidak perlu menghitung nomor kolom
   B. Argumennya lebih banyak sehingga lebih akurat
   C. VLOOKUP tidak bisa mencari teks
   D. Tidak ada bedanya

7. **Di sheet Penjualan, kamu mengisi Kategori di G2 dengan mengambil dari sheet Produk berdasarkan nama produk di C2, lalu rumusnya di-copy sampai G11. Rumus yang PALING AMAN:**

   A. =XLOOKUP(C2, Produk!$A$2:$A$5, Produk!$B$2:$B$5)
   B. =XLOOKUP(C2, Produk!A2:A5, Produk!B2:B5)
   C. =XLOOKUP(Produk!A2:A5, C2, Produk!B2:B5)
   D. =XLOOKUP(C2, Produk!$B$2:$B$5, Produk!$A$2:$A$5)

8. 📄 Data: Produk

   Hasil rumus VLOOKUP ini adalah...

   ```
   =VLOOKUP("Mouse", A2:C5, 3, FALSE)
   ```

   A. Aksesoris
   B. 150.000
   C. Mouse
   D. #REF!

9. **Kenapa VLOOKUP tanpa argumen ke-4 (atau dengan TRUE) berbahaya untuk mencari nama produk?**

   A. Ia memakai pencocokan perkiraan, sehingga bisa mengembalikan data yang SALAH tanpa memberi error
   B. Ia lebih lambat
   C. Ia tidak bisa membaca huruf kapital
   D. Ia selalu menghasilkan #N/A

Kunci & Pembahasan · Excel Bab 3

← Kembali ke soal · Awal bab

1. **A. Mencari sebuah nilai di satu kolom, lalu mengambil data yang sejajar di kolom lain** — Bayangkan buku telepon: cari NAMA (kolom pencarian), ambil NOMOR-nya (kolom hasil). XLOOKUP(yang_dicari, kolom_pencarian, kolom_hasil). Di dunia kerja ini dipakai untuk menggabungkan data dari dua sheet, misalnya mengambil harga produk dari master data.
   - **B:** Itu tugas SUMIFS.
   - **C:** Itu fungsi SORT atau menu Data → Sort.
   - **D:** Grafik dibuat dari menu Insert → Chart.
2. **B. 2.000.000** — Excel mencari 'Monitor' di A2:A5 → ketemu di baris 4. Lalu ia mengambil isi kolom hasil (C) di baris yang sama: C4 = 2.000.000.
   - **A:** Komputer ada di kolom B. Kolom hasil di rumus ini adalah C (Harga).
   - **C:** Itu nilai yang DICARI, bukan yang dikembalikan.
   - **D:** XLOOKUP mengembalikan isi sel, bukan nomor posisinya (itu tugas MATCH/XMATCH).
3. **B. Aksesoris** — Mouse ditemukan di baris 3. Kolom hasilnya B (Kategori), jadi yang diambil B3 = Aksesoris.
   - **A:** 150.000 ada di kolom C. Rumus ini mengambil dari kolom B.
   - **C:** Komputer adalah kategori Laptop dan Monitor, bukan Mouse.
   - **D:** #N/A hanya muncul kalau yang dicari tidak ditemukan. Mouse ada di tabel.
4. **B. #N/A** — Printer tidak ada di A2:A5, sehingga XLOOKUP mengembalikan error #N/A (Not Available). Solusinya: isi argumen ke-4 if_not_found, contoh =XLOOKUP("Printer",A2:A5,C2:C5,"Tidak ada").
   - **A:** XLOOKUP tidak mengubah 'tidak ketemu' menjadi 0 kecuali kamu memintanya di argumen ke-4.
   - **C:** XLOOKUP tidak mengembalikan nilai yang dicari.
   - **D:** Tanpa argumen ke-4 hasilnya error, bukan kosong.
5. **"tidak ada"** — Argumen ke-4 XLOOKUP adalah if_not_found. Karena isinya teks, wajib diapit kutip ganda: "Tidak ada". Laporan jadi rapi, tidak penuh #N/A.

6. **A. Bisa mencari ke kiri dan tidak perlu menghitung nomor kolom** — VLOOKUP hanya bisa mengambil kolom di KANAN kolom pencarian, dan kamu harus menulis nomor kolom (misal 3) — kalau ada kolom baru disisipkan, nomornya jadi salah. XLOOKUP langsung menunjuk kolom hasil, jadi lebih aman. Tapi tetap pelajari VLOOKUP: banyak file kantor lama masih memakainya.
   - **B:** Akurasi tidak ditentukan banyaknya argumen.
   - **C:** VLOOKUP bisa mencari teks.
   - **D:** Bedanya besar — lihat penjelasan.

7. **A. =XLOOKUP(C2, Produk!$A$2:$A$5, Produk!$B$2:$B$5)** — C2 dibiarkan relatif supaya berganti ke C3, C4... saat di-copy. Range tabel Produk DIKUNCI dengan $ supaya tidak ikut bergeser. Pola 'nilai relatif + tabel referensi absolut' ini dipakai di hampir semua laporan kantor.
   - **B:** Benar di G2, tapi saat di-copy ke bawah range ikut bergeser (A3:A6, A4:A7...) dan baris atas tabel tidak lagi ikut dicari → hasil salah atau #N/A.
   - **C:** Urutannya terbalik: argumen pertama adalah NILAI yang dicari (C2).
   - **D:** Kolom pencarian dan kolom hasil tertukar: ini mencari nama produk di kolom Kategori.

8. **B. 150.000** — VLOOKUP mencari 'Mouse' di kolom PERTAMA range (A), lalu mengambil kolom ke-3 dari range A:C, yaitu C (Harga) → 150.000. FALSE = harus cocok persis.
   - **A:** Aksesoris ada di kolom ke-2 range, bukan ke-3.
   - **C:** Kolom ke-1 adalah kolom pencarian itu sendiri.
   - **D:** #REF! muncul kalau nomor kolom melebihi lebar range (misal 4 untuk A:C).

9. **A. Ia memakai pencocokan perkiraan, sehingga bisa mengembalikan data yang SALAH tanpa memberi error** — Mode perkiraan (TRUE) dirancang untuk tabel bertingkat yang terurut, seperti tabel diskon. Untuk mencari nama atau kode, ia bisa mengambil baris tetangga tanpa error — laporan terlihat benar padahal salah. Selalu tulis FALSE di VLOOKUP. XLOOKUP aman karena default-nya pencocokan persis.
    - **B:** Masalahnya akurasi, bukan kecepatan.
    - **C:** VLOOKUP memang tidak membedakan huruf besar/kecil, tapi itu bukan bahayanya.
    - **D:** Justru bahayanya karena ia TIDAK memberi error.

IF Bertingkat, IFS, AND, OR

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

IF membuat keputusan otomatis: `=IF(syarat, jika_benar, jika_salah)`

```
=IF(F2>=5000000, "Target", "Belum")
```

IF bertingkat untuk lebih dari 2 kategori. Aturan emas: **urutkan dari batas terbesar**, karena Excel berhenti di syarat pertama yang benar.

```
=IF(F2>=5000000, "Tinggi",
   IF(F2>=1500000, "Sedang", "Rendah"))
```

Versi yang lebih mudah dibaca dengan **IFS** (TRUE = "selain itu"):

```
=IFS(F2>=5000000,"Tinggi", F2>=1500000,"Sedang", TRUE,"Rendah")
```

Gabungkan syarat:

- `AND(a, b)` → benar jika **semua** benar. Contoh bonus: Jakarta *dan* ≥ 5 juta.
- `OR(a, b)` → benar jika **salah satu** benar.

> ⚠️ Di dalam rumus, angka ditulis tanpa titik ribuan: `5000000`, bukan `5.000.000`. Teks hasil wajib diapit kutip ganda: `"Tinggi"`.

> 🧪 **Praktik:** sheet *Bab4-IF* — beri label Tinggi/Sedang/Rendah dan kolom Bonus.

🧠 Uji Diri — 10 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Penjualan**

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

1. 📄 Data: Penjualan

   Hasil rumus ini adalah...

   ```
   =IF(F2>=5000000, "Target", "Belum")
   ```

   A. Target
   B. Belum
   C. TRUE
   D. 14000000

2. 📄 Data: Penjualan

   Rumus yang sama dipakai di baris 5. Hasilnya...

   ```
   =IF(F5>=5000000, "Target", "Belum")
   ```

   A. Target
   B. Belum
   C. 750000
   D. #VALUE!

3. 📄 Data: Penjualan

   Hasil rumus IF bertingkat ini untuk baris 3 adalah...

   ```
   =IF(F3>=5000000, "Tinggi", IF(F3>=1500000, "Sedang",
   "Rendah"))
   ```

   A. Tinggi
   B. Sedang
   C. Rendah
   D. Error

4. 📄 Data: Penjualan

   Rumus yang sama untuk baris 7 menghasilkan...

   ```
   =IF(F7>=5000000, "Tinggi", IF(F7>=1500000, "Sedang",
   "Rendah"))
   ```

   A. Tinggi
   B. Sedang
   C. Rendah
   D. Belum

5. **Rumus ini memberi hasil yang salah. Kenapa?**

   ```
   =IF(F2>=1500000, "Sedang", IF(F2>=5000000, "Tinggi",
   "Rendah"))
   ```

   A. Nilai 14 juta jadi 'Sedang', karena syarat pertama sudah terpenuhi lebih dulu
   B. Excel tidak bisa membaca angka besar
   C. IF tidak boleh bersarang
   D. Kurang tanda $

6. **Rumus IFS yang setara dengan IF bertingkat Tinggi/Sedang/Rendah:**

   A. =IFS(F2>=5000000,"Tinggi", F2>=1500000,"Sedang", TRUE,"Rendah")
   B. =IFS(F2>=5000000,"Tinggi","Sedang","Rendah")
   C. =IFS("Tinggi",F2>=5000000,"Sedang",F2>=1500000)
   D. =IF(F2>=5000000,"Tinggi","Sedang","Rendah")

7. 📄 Data: Penjualan

   Berapa baris yang mendapat 'Bonus' dengan rumus ini?

   ```
   =IF(AND(B2="Jakarta", F2>=5000000), "Bonus", "-")
   ```

   A. 1
   B. 2
   C. 3
   D. 4

8. 📄 Data: Penjualan

   Berapa baris yang hasilnya 'Prioritas'?

   ```
   =IF(OR(C2="Laptop", D2>=5), "Prioritas", "Biasa")
   ```

   A. 3
   B. 5
   C. 6
   D. 7

9. **Lengkapi agar selain itu tertulis Eceran: =IF(D2>=5, "Grosir", ____)**

   Jawab: _____

10. **Rumus ini error. Kenapa?**

    ```
    =IF(F2>=5.000.000; "Tinggi"; "Rendah")
    ```

    A. Angka di dalam rumus ditulis tanpa pemisah ribuan: 5000000
    B. Pemisahnya harus koma
    C. IF tidak menerima teks
    D. Kurang tanda kurung

Kunci & Pembahasan · Excel Bab 4

← Kembali ke soal · Awal bab

1. **A. Target** — IF(syarat, jika_benar, jika_salah). F2 = 14.000.000, dan 14.000.000 ≥ 5.000.000 itu BENAR, jadi hasilnya teks pertama: Target.
   - **B:** 'Belum' hanya muncul jika syaratnya salah.
   - **C:** IF mengembalikan teks yang kamu tentukan, bukan TRUE/FALSE.
   - **D:** IF tidak mengembalikan nilai sel aslinya.

2. **B. Belum** — F5 = 750.000. Apakah 750.000 ≥ 5.000.000? Tidak. Syarat salah → yang dipakai argumen ketiga: Belum.
   - **A:** Syaratnya tidak terpenuhi untuk 750.000.
   - **C:** IF mengembalikan teks pilihanmu, bukan nilai selnya.
   - **D:** Tidak ada error: semua argumennya valid.

3. **B. Sedang** — F3 = 1.500.000. Syarat pertama (≥ 5 juta) salah → masuk ke IF kedua. Syarat kedua: 1.500.000 ≥ 1.500.000 → BENAR, karena ≥ termasuk 'sama dengan'. Hasil: Sedang.
   - **A:** 1,5 juta belum mencapai 5 juta.
   - **C:** Hati-hati: ≥ berarti 'lebih besar ATAU SAMA DENGAN'. 1.500.000 sama dengan batasnya.
   - **D:** Rumus ini valid: IF di dalam IF diperbolehkan.

4. **C. Rendah** — F7 = 1.200.000. Tidak ≥ 5 juta, tidak ≥ 1,5 juta → jatuh ke pilihan terakhir: Rendah.
   - **A:** 1,2 juta jauh di bawah 5 juta.
   - **B:** 1.200.000 masih di bawah batas Sedang (1.500.000).
   - **D:** 'Belum' tidak ada di rumus ini.

5. **A. Nilai 14 juta jadi 'Sedang', karena syarat pertama sudah terpenuhi lebih dulu** — Excel memeriksa syarat dari KIRI ke KANAN dan berhenti di syarat pertama yang benar. 14 juta ≥ 1,5 juta sudah benar, jadi hasilnya 'Sedang' dan syarat 'Tinggi' tidak pernah diperiksa. Aturan emas: untuk syarat ≥, urutkan dari batas TERBESAR ke terkecil.
   - **B:** Angka 14 juta tidak bermasalah untuk Excel.
   - **C:** IF bersarang diperbolehkan (sampai 64 tingkat).
   - **D:** $ tidak ada hubungannya dengan logika urutan syarat.

6. **A. =IFS(F2>=5000000,"Tinggi", F2>=1500000,"Sedang", TRUE,"Rendah")** — IFS ditulis berpasangan: syarat1, hasil1, syarat2, hasil2, ... Untuk 'selain itu', pakai TRUE sebagai syarat terakhir (TRUE selalu benar). Lebih mudah dibaca daripada IF bertumpuk.
   - **B:** 'Sedang' dan 'Rendah' tidak punya syarat — IFS butuh pasangan syarat-hasil.
   - **C:** Urutannya terbalik: syarat dulu, baru hasilnya.
   - **D:** IF hanya menerima 3 argumen.
7. **B. 2** — AND = SEMUA syarat harus benar. Baris Jakarta: 2 (14 jt ✓), 4 (6 jt ✓), 7 (1,2 jt ×), 9 (4 jt ×). Jadi hanya 2 baris yang mendapat Bonus.
   - **A:** Baris 4 (Citra, Jakarta, 6 juta) juga memenuhi kedua syarat.
   - **C:** Baris 9 (Andi, Jakarta) hanya 4 juta — belum ≥ 5 juta.
   - **D:** Itu banyaknya baris Jakarta. Syarat ≥ 5 juta belum diterapkan.
8. **C. 6** — OR = cukup SALAH SATU syarat benar. Laptop: baris 2, 6, 8. Qty ≥ 5: baris 3 (10), 5 (5), 10 (6). Tidak ada baris yang dihitung dua kali → 6 baris.
   - **A:** Itu hanya baris Laptop. Baris dengan Qty ≥ 5 juga ikut.
   - **B:** Cek lagi baris 5: Qty-nya tepat 5, dan ≥ 5 termasuk 5.
   - **D:** Hanya ada 6 baris yang memenuhi.
9. **"eceran"** — Argumen ketiga IF adalah hasil jika syarat SALAH. Karena berupa teks, tulis "Eceran" dengan kutip ganda.
10. **A. Angka di dalam rumus ditulis tanpa pemisah ribuan: 5000000** — Titik ribuan hanyalah TAMPILAN (format angka). Di dalam rumus, angka ditulis polos: 5000000. Titik koma di rumus ini justru benar untuk Excel berbahasa/region Indonesia.
    - **B:** Di Excel region Indonesia, titik koma adalah pemisah yang benar.
    - **C:** IF boleh mengembalikan teks.
    - **D:** Jumlah kurungnya sudah lengkap.

EXCEL · BAB 5

Pivot Table — Meringkas Ribuan Baris dalam Detik

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Pivot table = ringkasan otomatis. Yang di SUMIFS butuh puluhan rumus, di pivot cukup drag-and-drop.

Langkah membuat:

1. Klik satu sel di dalam data → **Ctrl+T** (jadikan Table, agar baris baru ikut terbaca).
2. **Insert → PivotTable** → New Worksheet → OK.
3. Seret kolom ke 4 area:
   - **Rows** → kelompok baris (mis. Sales) — mirip `GROUP BY` di SQL
   - **Columns** → kelompok kolom (mis. Wilayah) — membentuk tabel silang
   - **Values** → angka yang dihitung (mis. Sum of Total)
   - **Filters** → saring seluruh laporan (mis. hanya Laptop)

Contoh hasil Rows = Sales, Values = Sum of Total:

Sales	Sum of Total
Andi	18.750.000
Bima	4.700.000
Citra	13.000.000
Dewi	8.800.000
Grand Total	**45.250.000**

- Muncul **Count** padahal mau Sum? → Value Field Settings → Sum.
- Data berubah? → klik kanan → **Refresh**. Pivot tidak update sendiri.
- Selalu cocokkan **Grand Total** dengan =SUM data asli — kebiasaan analis yang teliti.

> 🧪 **Praktik:** sheet *Bab5-Pivot* — buat 3 pivot: total per sales, total per wilayah, dan tabel silang Sales × Wilayah.

🧠 Uji Diri — 10 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 Sheet: Penjualan

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

1. **Pivot table paling cocok untuk...**

   A. Meringkas data besar (total/jumlah per kategori) tanpa menulis rumus
   B. Mengedit data mentah
   C. Mencari satu nilai
   D. Membuat tabel kosong

2. **Menu untuk membuat pivot table di Excel:**

   A. Insert → PivotTable
   B. Home → Format
   C. Data → Sort
   D. View → Freeze Panes

3. **Kamu ingin melihat total penjualan PER SALES. Kolom 'Sales' diletakkan di area...**

   A. Rows
   B. Values
   C. Filters
   D. Tidak perlu dimasukkan

4. 📄 Data: Penjualan

   Pivot: Rows = Sales, Values = Sum of Total. Angka di baris 'Citra' adalah…

   A. 6.000.000
   B. 13.000.000
   C. 7.000.000
   D. 2

5. 📄 Data: Penjualan

   Pivot: Rows = Wilayah, Values = Sum of Total. Wilayah dengan nilai terbesar:

   A. Jakarta — 25.200.000
   B. Bandung — 10.500.000
   C. Surabaya — 9.550.000
   D. Jakarta — 4

6. 📄 Data: Penjualan

   Pivot: Rows = Produk, Values = Count of Produk. Angka untuk Monitor adalah…

   A. 3
   B. 6
   C. 12.000.000
   D. 1

7. 📄 Data: Penjualan

   Pivot: Rows = Sales, Columns = Wilayah, Values = Sum of Total. Isi sel pertemuan Andi × Jakarta adalah…

   A. 18.000.000
   B. 14.000.000
   C. 18.750.000
   D. 25.200.000

8. **Kamu menambah 5 baris transaksi baru di bawah data, tapi pivot belum berubah. Yang harus dilakukan:**

   A. Klik kanan pivot → Refresh (dan pastikan sumber datanya mencakup baris baru — paling aman ubah data jadi Table dengan Ctrl+T)
   B. Buat file Excel baru
   C. Ketik manual angkanya di pivot
   D. Tunggu, nanti update sendiri

9. **Pivot menampilkan 'Count of Total', padahal kamu ingin jumlah rupiah. Perbaikannya:**

   A. Value Field Settings → ganti ke Sum
   B. Hapus kolom Total
   C. Pindahkan Total ke Rows
   D. Ganti format angka menjadi Rupiah

10. **Shortcut untuk mengubah range data menjadi Excel Table (agar pivot otomatis mencakup baris baru): Ctrl + ____**

    Jawab: _____

Kunci & Pembahasan · Excel Bab 5

← Kembali ke soal · Awal bab

1. **A. Meringkas data besar (total/jumlah per kategori) tanpa menulis rumus** — Pivot table membuat ringkasan seperti 'total penjualan per sales per wilayah' hanya dengan drag-and-drop. Untuk data 10.000 baris, ini jauh lebih cepat daripada menulis ratusan SUMIFS. Ini skill yang hampir selalu dites saat seleksi data analyst atau admin.
   - **B:** Pivot hanya membaca data mentah; ia tidak mengubahnya.
   - **C:** Untuk mencari satu nilai, pakai XLOOKUP.
   - **D:** Pivot selalu dibuat dari data yang sudah ada.
2. **A. Insert → PivotTable** — Klik salah satu sel di dalam data → Insert → PivotTable → pilih letaknya (biasanya New Worksheet) → OK. Lalu seret kolom ke area Rows, Columns, Values, dan Filters.
   - **B:** Format hanya untuk tampilan sel.
   - **C:** Sort hanya mengurutkan data.
   - **D:** Freeze Panes hanya mengunci baris/kolom saat scroll.
3. **A. Rows** — Rows = apa yang mau dijadikan baris/kelompok (di sini nama sales). Values = angka yang mau dihitung (kolom Total, dengan Sum). Cara gampang mengingat: Rows itu seperti GROUP BY di SQL.
   - **B:** Values untuk angka yang dihitung (Total). Nama sales bukan angka untuk dijumlah.
   - **C:** Filters hanya untuk menyaring seluruh laporan (misal hanya Jakarta), bukan mengelompokkan.
   - **D:** Tanpa Sales di Rows, pivot hanya menampilkan satu angka total.
4. **B. 13.000.000** — Pivot menjumlahkan semua transaksi Citra: 6.000.000 (Monitor) + 7.000.000 (Laptop) = 13.000.000. Hasilnya sama persis dengan =SUMIFS(F2:F11,A2:A11,"Citra") — pivot adalah cara cepat membuat banyak SUMIFS sekaligus.
   - **A:** Itu hanya transaksi pertama Citra.
   - **C:** Itu hanya transaksi kedua Citra.
   - **D:** 2 adalah banyaknya transaksi (Count), bukan Sum.

5. **A. Jakarta — 25.200.000** — Jakarta: 14.000.000 + 6.000.000 + 1.200.000 + 4.000.000 = 25.200.000. Bandung 10.500.000, Surabaya 9.550.000. Totalnya 45.250.000 — selalu cek bahwa Grand Total pivot sama dengan total data aslinya.
   - **B:** Bandung di urutan kedua.
   - **C:** Surabaya justru paling kecil.
   - **D:** 4 adalah Count (banyaknya transaksi), bukan Sum.
6. **A. 3** — Count menghitung BANYAKNYA baris. Monitor muncul di baris 4, 9, dan 11 → 3 transaksi.
   - **B:** 6 adalah total Qty Monitor (3+2+1), bukan banyaknya transaksi.
   - **C:** 12.000.000 adalah Sum of Total Monitor.
   - **D:** Ada 3 transaksi Monitor.
7. **A. 18.000.000** — Columns menambah dimensi kedua (tabel silang). Andi di Jakarta: 14.000.000 + 4.000.000 = 18.000.000. Ini sama dengan SUMIFS dengan dua syarat yang kamu kerjakan di bab sebelumnya.
   - **B:** Itu hanya satu transaksi Andi di Jakarta — ada dua.
   - **C:** Itu total Andi di SEMUA wilayah (termasuk Surabaya).
   - **D:** Itu total Jakarta untuk semua sales.
8. **A. Klik kanan pivot → Refresh (dan pastikan sumber datanya mencakup baris baru — paling aman ubah data jadi Table dengan Ctrl+T)** — Pivot TIDAK otomatis update. Selalu Refresh setelah data berubah. Kalau sumbernya range biasa (A1:F11), baris baru di luar range tidak terbaca — itulah kenapa data sebaiknya diubah jadi Excel Table dulu, supaya range-nya ikut memanjang.
   - **B:** Tidak perlu: cukup Refresh.
   - **C:** Mengetik manual di pivot akan ditolak atau merusak laporan.
   - **D:** Pivot tidak pernah update sendiri tanpa Refresh.

9. **A. Value Field Settings → ganti ke Sum** — Excel memilih Count otomatis jika kolomnya mengandung sel kosong atau teks. Klik panah pada field di area Values → Value Field Settings → Sum. Sekalian cek apakah ada sel Total yang kosong atau berisi teks di data mentah.
    - **B:** Kolom Total justru yang kamu butuhkan.
    - **C:** Di Rows, Total akan jadi daftar kelompok, bukan dijumlah.
    - **D:** Format hanya mengubah tampilan, bukan cara menghitung.
10. **t** — Ctrl+T (Insert → Table). Table punya nama, range yang otomatis memanjang, dan filter bawaan. Ini kebiasaan yang membedakan pengguna Excel pemula dan menengah.

Conditional Formatting — Membuat Data Bicara

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Conditional formatting (CF) mewarnai sel otomatis sesuai nilainya. Nilainya tidak berubah, hanya tampilannya.

Home → Conditional Formatting, lalu pilih:

- **Highlight Cells Rules** → Greater Than, Less Than, Between, Text Contains, Duplicate Values
- **Top/Bottom Rules** → Top 3 Items, Above Average
- **Data Bars** → grafik batang mini di dalam sel
- **Color Scales** → gradasi warna (merah → hijau)
- **New Rule → Use a formula** → paling fleksibel

Teknik paling dicari: **mewarnai satu baris penuh** berdasarkan satu kolom.

```
Pilih A2:F11 → New Rule → Use a formula:
=$F2>=5000000
   ↑ kolom dikunci, baris bebas
```

Jika ada beberapa rule, urutannya diatur di **Manage Rules** — rule paling atas punya prioritas.

> 🎨 Gunakan warna seperlunya. Laporan yang semuanya berwarna sama buruknya dengan yang tidak berwarna. Tandai hanya yang perlu diperhatikan atasan.

> 🧪 **Praktik:** sheet *Bab6-CF* — warnai baris Total ≥ 5 juta, beri Data Bars di Qty, dan tandai nama sales tertentu.

🧠 Uji Diri — 9 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Penjualan**

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

1. **Conditional formatting berfungsi untuk...**

   A. Mengubah tampilan sel (warna, ikon, bar) otomatis berdasarkan nilainya
   B. Mengubah nilai sel
   C. Mengurutkan data
   D. Menghapus data duplikat

2. **Menu Conditional Formatting ada di tab...**

   A. Home
   B. Insert
   C. Data
   D. Review

3. 📄 Data: Penjualan

   Rule: Highlight Cells → Greater Than 5000000 diterapkan pada F2:F11. Berapa sel yang berwarna?

   A. 3
   B. 4
   C. 5
   D. 6

4. 📄 Data: Penjualan

   Rule 'Top 3 Items' pada F2:F11 akan mewarnai...

   A. 14.000.000, 7.000.000, 7.000.000
   B. 14.000.000, 7.000.000, 6.000.000
   C. Tiga baris pertama
   D. 14.000.000 saja

5. **Kamu ingin mewarnai SELURUH BARIS (A sampai F) jika Total ≥ 5 juta. Cara yang benar:**

   A. New Rule → 'Use a formula...': =$F2>=5000000, diterapkan ke A2:F11
   B. Highlight Cells → Greater Than pada A2:F11
   C. Data Bars pada kolom F
   D. Color Scales pada A2:A11

6. **Pada rumus rule =$F2>=5000000 untuk range A2:F11, kenapa F diberi $ tetapi angka 2 tidak?**

   A. Kolom dikunci agar SEMUA sel di baris itu mengecek kolom F; baris dibiarkan bergerak agar tiap baris mengecek dirinya sendiri
   B. Supaya rumusnya lebih pendek
   C. Tanda $ wajib di semua rumus CF
   D. Agar hanya baris 2 yang diwarnai

7. **Data Bars cocok dipakai untuk...**

   A. Melihat perbandingan besar-kecil nilai secara visual langsung di dalam sel
   B. Mewarnai teks jadi merah
   C. Menyembunyikan angka
   D. Menghitung rata-rata

8. 📄 Data: Penjualan

   Rule formula =$A2="Bima" diterapkan pada A2:F11. Berapa BARIS yang terwarnai?

   A. 1
   B. 3
   C. 4
   D. 10

9. **Ada dua rule: merah untuk Total < 1 juta, hijau untuk Total < 2 juta. Sel 750.000 cocok dengan keduanya. Warnanya apa, dan bagaimana mengaturnya?**

   A. Tergantung urutan di Manage Rules — rule paling atas menang; atur urutannya (bisa dibantu 'Stop If True')
   B. Selalu hijau
   C. Warnanya tercampur
   D. Excel menolak membuat rule kedua

Kunci & Pembahasan · Excel Bab 6

← Kembali ke soal · Awal bab

1. **A. Mengubah tampilan sel (warna, ikon, bar) otomatis berdasarkan nilainya** — Conditional formatting (CF) membuat data 'berbicara': transaksi besar langsung hijau, target yang belum tercapai merah. Nilai selnya tetap sama — hanya tampilannya yang berubah.
   - **B:** CF tidak pernah mengubah isi sel.
   - **C:** Mengurutkan memakai Sort.
   - **D:** Untuk itu pakai Data → Remove Duplicates (walau CF bisa MENANDAI duplikat).
2. **A. Home** — Home → Conditional Formatting. Di sana ada Highlight Cells Rules, Top/Bottom Rules, Data Bars, Color Scales, Icon Sets, dan New Rule untuk aturan berbasis rumus.
   - **B:** Tab Insert untuk tabel, grafik, dan gambar.
   - **C:** Tab Data untuk sort, filter, dan validasi.
   - **D:** Tab Review untuk komentar dan proteksi.
3. **B. 4** — Greater Than artinya LEBIH DARI (bukan sama dengan). Nilai > 5 juta: 14 juta, 6 juta, 7 juta, 7 juta → 4 sel.
   - **A:** Ada dua transaksi 7 juta (Dewi dan Citra).
   - **C:** Tidak ada nilai tepat 5 juta yang bisa menambah hitungan.
   - **D:** Hanya 4 nilai yang di atas 5 juta.
4. **A. 14.000.000, 7.000.000, 7.000.000** — Top 3 = tiga nilai terbesar: 14 juta, lalu dua nilai 7 juta. 6 juta ada di urutan ke-4, jadi tidak diwarnai. (Kalau ada nilai yang sama di batas, Excel bisa mewarnai lebih dari 3 sel.)
   - **B:** 6 juta berada di peringkat 4 — ada DUA nilai 7 juta di atasnya.
   - **C:** Top/Bottom melihat NILAI, bukan posisi baris.
   - **D:** Itu Top 1, bukan Top 3.

5. **A. New Rule → 'Use a formula...': =$F2>=5000000, diterapkan ke A2:F11** — Untuk mewarnai satu baris penuh berdasarkan satu kolom, CF harus memakai RUMUS. Pilih A2:F11 → New Rule → Use a formula → =$F2>=5000000 → pilih warna. Teknik ini sangat sering dipakai di laporan penjualan.
   - **B:** Ini membandingkan SETIAP sel dengan 5 juta, termasuk kolom nama dan Qty — tidak sesuai tujuan.
   - **C:** Data Bars hanya membuat bar di kolom F, bukan mewarnai baris.
   - **D:** Color Scales memberi gradasi warna, bukan aturan ≥ 5 juta.

6. **A. Kolom dikunci agar SEMUA sel di baris itu mengecek kolom F; baris dibiarkan bergerak agar tiap baris mengecek dirinya sendiri** — Excel menerapkan rumus ke setiap sel di A2:F11 seperti di-copy. Tanpa $ di depan F, sel B2 akan mengecek G2, C2 mengecek H2, dst. Tanpa baris yang bebas (angka 2 tanpa $), semua baris akan mengecek F2. $F2 = 'selalu kolom F, baris ikut'.
   - **B:** Panjang rumus bukan alasannya.
   - **C:** $ hanya dipakai bila perlu mengunci.
   - **D:** Justru karena baris TIDAK dikunci, setiap baris diperiksa sendiri-sendiri.

7. **A. Melihat perbandingan besar-kecil nilai secara visual langsung di dalam sel** — Data Bars membuat grafik batang mini di dalam sel. Cocok untuk kolom Total atau Qty, supaya atasan bisa langsung melihat mana yang besar tanpa membaca angka satu per satu.
   - **B:** Itu Highlight Cells Rules.
   - **C:** Data Bars justru memperjelas angka.
   - **D:** Rata-rata dihitung dengan AVERAGE.

8. **B. 3** — Rumus mengecek apakah kolom A di tiap baris berisi 'Bima': baris 3, 7, dan 11 → 3 baris penuh berwarna (A sampai F).
   - **A:** Bima punya 3 transaksi.
   - **C:** Hanya ada 3 baris Bima.
   - **D:** Hanya baris yang memenuhi syarat yang diwarnai.

9. **A. Tergantung urutan di Manage Rules — rule paling atas menang; atur urutannya (bisa dibantu 'Stop If True')** — Home → Conditional Formatting → Manage Rules menampilkan urutan prioritas. Rule teratas diterapkan lebih dulu; jika dua rule mengatur hal yang sama (warna isi), yang di atas menang. Letakkan aturan yang lebih spesifik (< 1 juta) di atas.
    - **B:** Tidak selalu — tergantung urutan rule.
    - **C:** Excel tidak mencampur warna.
    - **D:** Excel boleh punya banyak rule di range yang sama.

Teks & Membersihkan Data

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Di dunia kerja, data hampir tidak pernah datang dalam keadaan rapi. Data hasil export sistem, form Google, atau ketikan banyak orang biasanya berantakan. Analis yang bisa **membersihkan data dengan cepat** sangat dihargai.

📄 **Sheet: Pelanggan_Mentah**

	A	B	C	D	E
1	**Nama**	**Kota**	**Telepon**	**Email**	**Kode**
2	rina wulandari	jakarta	0812-3456-7890	Rina.W@Mail.com	INV-2026-001
3	TONO SAPUTRA	Bandung	081298765432	tono@mail.com	INV-2026-002
4	Sari Dewi	JAKARTA	0813-2222-1111	sari.dewi@mail.com	INV-2026-003
5	budi hartono	Surabaya	0857-1111-2222	budi@mail.com	INV-2026-004
6	Lia Amelia	bandung	0813-7777-8888	LIA@MAIL.COM	INV-2026-005
7	TONO SAPUTRA	Bandung	081298765432	tono@mail.com	INV-2026-002

Masalah di data ini: spasi berlebih (A2, A4, B3), huruf besar-kecil tidak konsisten, format telepon berbeda, dan baris 7 adalah **duplikat** baris 3.

```
=TRIM(A2)          → buang spasi berlebih     "rina wulandari"
=PROPER(A3)        → Kapital Tiap Kata         "Tono Saputra"
=UPPER(B2) / LOWER → HURUF BESAR / kecil
=LEN(E2)           → jumlah karakter          12
=LEFT(E2,3)        → 3 karakter dari kiri     "INV"
=RIGHT(E2,3)       → 3 karakter dari kanan    "001"
=MID(E2,5,4)       → mulai posisi 5, ambil 4  "2026"
=FIND("@",D3)      → posisi karakter "@"      5
=SUBSTITUTE(C2,"-","") → ganti teks          "081234567890"
=A2&" - "&B2       → sambung teks
```

Resep pembersihan yang aman:

1. Jangan menimpa data asli. Buat **kolom bantu**: `=PROPER(TRIM(A2))`, lalu tarik ke bawah.
2. Cek duplikat: `=COUNTIF($A$2:$A$7,A2)` → hasil > 1 artinya dobel.
3. Setelah rapi, salin kolom bantu → *Paste Special* → *Values*, lalu **Data → Remove Duplicates**.

4. Untuk memecah teks: **Data → Text to Columns**, atau ketik 1 contoh lalu **Ctrl+E (Flash Fill)**.

> ⚠️ **Jebakan klasik:** 'Bandung' dan 'Bandung ' (ada spasi di belakang) dianggap BERBEDA oleh COUNTIF, SUMIFS, dan Pivot Table. Kalau angka laporanmu tidak cocok, curigai spasi tersembunyi dan cek dengan `=LEN()`.

> 🧪 **Praktik di Excel:** file latihan → sheet *Bab7-Teks*.

🧠 Uji Diri — 16 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Pelanggan_Mentah**

	A	B	C	D	E
1	**Nama**	**Kota**	**Telepon**	**Email**	**Kode**
2	rina wulandari	jakarta	0812-3456-7890	Rina.W@Mail.com	INV-2026-001
3	TONO SAPUTRA	Bandung	081298765432	tono@mail.com	INV-2026-002
4	Sari Dewi	JAKARTA	0813-2222-1111	sari.dewi@mail.com	INV-2026-003
5	budi hartono	Surabaya	0857-1111-2222	budi@mail.com	INV-2026-004
6	Lia Amelia	bandung	0813-7777-8888	LIA@MAIL.COM	INV-2026-005
7	TONO SAPUTRA	Bandung	081298765432	tono@mail.com	INV-2026-002

1. 📄 Data: Pelanggan_Mentah

   Sel A2 berisi ' rina wulandari' (ada spasi di depan). Hasil rumus ini adalah...

   ```
   =TRIM(A2)
   ```

   A. rina wulandari
   B. Rina Wulandari
   C. RINA WULANDARI
   D. rina wulandari

2. **Hasil rumus ini adalah...**

```
=PROPER("budi hartono")
```

A. Budi hartono
B. Budi Hartono
C. BUDI HARTONO
D. budi Hartono

3. 📄 Data: Pelanggan_Mentah

Kolom A berisi nama yang berantakan: ada spasi berlebih, ada yang huruf besar semua, ada yang huruf kecil semua. Rumus kolom bantu (ditulis di F2 lalu ditarik ke bawah) yang paling tepat:

A. =PROPER(TRIM(A2))
B. =TRIM(A2)
C. =UPPER(TRIM(A2))
D. =PROPER(A2)

4. 📄 Data: Pelanggan_Mentah

Hasil rumus ini adalah...

```
=LEN(E2)
```

A. 3
B. 10
C. 12
D. 13

5. 📄 Data: Pelanggan_Mentah

Hasil rumus ini adalah...

```
=LEFT(E2,3)
```

A. INV
B. 001
C. 2026
D. INV-

6. 📄 Data: Pelanggan_Mentah

   Hasil rumus ini adalah...

   ```
   =RIGHT(E6,3)
   ```

   A. 005
   B. 5
   C. INV
   D. 2026

7. 📄 Data: Pelanggan_Mentah

   Kamu ingin mengambil TAHUN dari kode faktur di E2 ('INV-2026-001'). Rumus yang tepat:

   A. =MID(E2,5,4)
   B. =MID(E2,4,4)
   C. =LEFT(E2,4)
   D. =RIGHT(E2,4)

8. 📄 Data: Pelanggan_Mentah

   Nomor telepon di C2 ingin dibuat tanpa tanda strip. Hasil rumus ini...

   ```
   =SUBSTITUTE(C2,"-","")
   ```

   A. 081234567890
   B. 0812 3456 7890
   C. 0812-3456-7890
   D. 81234567890

9. **A2 = 'Rina' dan B2 = 'Jakarta'. Kamu ingin hasil: Rina - Jakarta. Rumus yang benar:**

   A. =A2&" - "&B2
   B. =A2+" - "+B2
   C. =A2&B2
   D. ="A2 - B2"

10. 📄 Data: Pelanggan_Mentah

D3 = 'tono@mail.com'. Hasil rumus ini adalah...

```
=LEFT(D3, FIND("@",D3)-1)
```

A. tono
B. tono@
C. mail.com
D. 5

11. 📄 Data: Pelanggan_Mentah

Di kolom B ada 3 data Bandung, tapi rumus ini hanya menghasilkan 1. Kenapa?

```
=COUNTIF(B2:B7,"Bandung")
```

A. B3 dan B7 berisi 'Bandung ' dengan spasi di belakang, sehingga dianggap teks yang berbeda
B. COUNTIF peka huruf besar/kecil, jadi 'bandung' tidak dihitung
C. COUNTIF hanya bisa menghitung angka
D. Range harus dikunci dengan $

12. 📄 Data: Pelanggan_Mentah

Rumus ini ditulis di F3. Hasilnya dan artinya adalah...

```
=COUNTIF($A$2:$A$7,A3)
```

A. 2 — nama di A3 muncul dua kali (ada duplikat)
B. 1 — tidak ada duplikat
C. 6 — ada 6 baris data
D. TRUE

13. **Fitur Excel untuk menghapus baris yang persis sama (duplikat) dengan cepat:**

A. Data → Remove Duplicates
B. Home → Delete Sheet
C. Ctrl+Z
D. Insert → Table

14. **Kolom A berisi nama lengkap ('Rina Wulandari'). Kamu ingin memisahkan nama depan dan belakang ke dua kolom tanpa rumus. Cara tercepat:**

    A. Data → Text to Columns (pemisah: spasi), atau ketik 1 contoh lalu Flash Fill (Ctrl+E)
    B. Ketik ulang satu per satu
    C. Pakai fungsi SUM
    D. Conditional Formatting

15. **Angka di kolom Qty rata KIRI dan ada segitiga hijau di pojok sel. =SUM(D2:D11) menghasilkan 0. Penyebabnya:**

    A. Angka tersimpan sebagai TEKS (biasanya hasil impor/copy dari sistem lain)
    B. Rumus SUM rusak
    C. Kolomnya terlalu sempit
    D. Excel belum di-update

16. **Fungsi untuk mengubah semua huruf menjadi KAPITAL: =_____(A2)**

    Jawab: _____

Kunci & Pembahasan · Excel Bab 7

← Kembali ke soal · Awal bab

1. **A. rina wulandari** — TRIM hanya membuang spasi berlebih: spasi di depan, di belakang, dan spasi ganda di tengah kalimat. Huruf besar/kecil TIDAK diubah, jadi hasilnya 'rina wulandari'.
   - **B:** Membuat huruf awal tiap kata jadi kapital adalah tugas PROPER, bukan TRIM.
   - **C:** Mengubah ke huruf besar semua adalah tugas UPPER.
   - **D:** Justru spasi di depan itulah yang dibuang TRIM.

2. **B. Budi Hartono** — PROPER membuat huruf pertama SETIAP kata menjadi kapital dan sisanya huruf kecil. Cocok untuk merapikan nama orang dan nama kota.
   - **A:** Itu gaya kalimat (kapital hanya di awal). PROPER mengkapitalkan setiap kata.
   - **C:** Itu hasil UPPER.
   - **D:** Kata pertama juga ikut dikapitalkan.

3. **A. =PROPER(TRIM(A2))** — Rumus bertingkat dikerjakan dari dalam ke luar: TRIM membuang spasi berlebih dulu, lalu PROPER merapikan huruf besar-kecil. Hasilnya 'Rina Wulandari', 'Tono Saputra', 'Sari Dewi', dan seterusnya. Jangan menimpa data asli — kerjakan di kolom bantu.
   - **B:** TRIM saja tidak memperbaiki 'TONO SAPUTRA' yang huruf besar semua.
   - **C:** Rapi, tapi semua nama jadi HURUF BESAR — bukan format nama yang lazim.
   - **D:** PROPER saja tidak membuang spasi ganda di 'Sari Dewi' dan spasi di depan 'rina'.

4. **C. 12** — LEN menghitung banyaknya karakter, termasuk tanda strip dan spasi. 'INV-2026-001' = I,N,V,-,2,0,2,6,-,0,0,1 → 12 karakter. LEN berguna untuk mendeteksi data aneh, misalnya nomor telepon yang kurang digit atau ada spasi tersembunyi.
   - **A:** 3 adalah panjang 'INV' saja.
   - **B:** Kalau tanda strip tidak dihitung hasilnya 10 — padahal LEN menghitung SEMUA karakter.
   - **D:** Tidak ada spasi tersembunyi di kode ini; panjangnya 12.

5. **A. INV** — LEFT(teks, n) mengambil n karakter dari KIRI. Tiga karakter pertama 'INV-2026-001' adalah 'INV'.
   - **B:** Mengambil dari kanan adalah tugas RIGHT.
   - **C:** Mengambil dari tengah adalah tugas MID.
   - **D:** Itu 4 karakter, padahal argumen keduanya 3.

6. **A. 005** — RIGHT mengambil karakter dari KANAN. E6 = 'INV-2026-005', tiga karakter terakhir = '005'. Hasil fungsi teks selalu berupa TEKS, jadi angka nol di depan tetap ada.
   - **B:** Nol di depan hilang hanya kalau hasilnya diubah menjadi angka (misal dengan VALUE). Hasil RIGHT berupa teks.
   - **C:** Itu hasil LEFT(E6,3).
   - **D:** Itu hasil MID(E6,5,4).

7. **A. =MID(E2,5,4)** — MID(teks, posisi_awal, jumlah). Hitung posisinya: I=1, N=2, V=3, '-'=4, '2'=5. Jadi mulai dari posisi 5 dan ambil 4 karakter → '2026'.
   - **B:** Posisi 4 adalah tanda strip, hasilnya '-202'.
   - **C:** LEFT 4 karakter menghasilkan 'INV-'.
   - **D:** RIGHT 4 karakter menghasilkan '-001'.

8. **A. 081234567890** — SUBSTITUTE(teks, yang_diganti, penggantinya). Semua '-' diganti dengan teks kosong "", jadi tanda strip hilang. Hasilnya tetap teks, sehingga angka 0 di depan aman.
   - **B:** Strip diganti dengan teks kosong, bukan spasi.
   - **C:** Itu teks aslinya — SUBSTITUTE sudah mengganti semua tanda strip.
   - **D:** Nol di depan tidak hilang karena hasilnya teks, bukan angka.

9. **A. =A2&" - "&B2** — Tanda & menyambung teks (sama seperti fungsi CONCAT). Teks tambahan seperti ' - ' ditulis dalam kutip — termasuk spasinya, supaya hasilnya tidak menempel.
   - **B:** Di Excel, + hanya untuk angka. Menjumlah teks menghasilkan #VALUE!.
   - **C:** Hasilnya 'RinaJakarta' — menempel tanpa pemisah.
   - **D:** Di dalam kutip, A2 dan B2 hanyalah tulisan, bukan alamat sel. Hasilnya teks 'A2 - B2'.

10. **A. tono** — Kerjakan dari dalam: FIND("@",D3) mencari posisi '@' → 5. Lalu 5−1 = 4, dan LEFT(D3,4) mengambil 4 karakter pertama → 'tono'. Pola FIND + LEFT/MID ini dipakai untuk memotong teks yang panjangnya berbeda-beda.
    - **B:** Tanpa −1, karakter '@' ikut terambil. Rumus ini sudah dikurangi 1.
    - **C:** Itu bagian SETELAH '@'. Untuk itu butuh MID atau RIGHT.
    - **D:** 5 adalah hasil FIND saja (posisi '@'), belum dipakai oleh LEFT.

11. **A. B3 dan B7 berisi 'Bandung ' dengan spasi di belakang, sehingga dianggap teks yang berbeda** — Spasi tersembunyi di belakang ('Bandung ') membuat teks dianggap berbeda. COUNTIF TIDAK peka huruf besar/kecil, jadi yang terhitung justru B6 'bandung'. Pelajarannya: bersihkan data dengan TRIM dulu sebelum membuat ringkasan. Inilah penyebab angka laporan yang 'tiba-tiba' tidak cocok.
    - **B:** Kebalikannya: COUNTIF tidak peka huruf besar/kecil — 'bandung' (B6) justru yang terhitung.
    - **C:** COUNTIF bisa menghitung teks dengan syarat teks.
    - **D:** Tanda $ hanya penting saat rumus di-copy; tidak mengubah hasil di sini.

12. **A. 2 — nama di A3 muncul dua kali (ada duplikat)** — COUNTIF menghitung berapa kali isi A3 ('TONO SAPUTRA') muncul di seluruh kolom A: baris 3 dan 7 → 2. Isi kolom bantu ini untuk semua baris; hasil > 1 berarti duplikat. Range dikunci dengan $ supaya tidak bergeser saat rumus ditarik ke bawah.
    - **B:** 'TONO SAPUTRA' ada di A3 dan A7.
    - **C:** COUNTIF tidak menghitung jumlah baris; ia menghitung yang sama dengan A3.
    - **D:** COUNTIF mengembalikan angka, bukan TRUE/FALSE.

13. **A. Data → Remove Duplicates** — Data → Remove Duplicates menghapus baris ganda berdasarkan kolom yang kamu pilih. Tips: rapikan dulu dengan TRIM/PROPER, karena 'TONO SAPUTRA' dan 'Tono Saputra ' dianggap berbeda sebelum dirapikan. Simpan salinan data mentah sebelum menghapus.
    - **B:** Itu menghapus seluruh sheet.
    - **C:** Ctrl+Z membatalkan aksi terakhir.
    - **D:** Table merapikan format & range, tidak menghapus duplikat.

14. **A. Data → Text to Columns (pemisah: spasi), atau ketik 1 contoh lalu Flash Fill (Ctrl+E)** — Text to Columns memecah teks berdasarkan pemisah (spasi, koma, titik koma). Flash Fill (Ctrl+E) menebak pola dari contoh yang kamu ketik. Di Excel 365 juga ada fungsi TEXTSPLIT.
    - **B:** Lambat dan rawan salah ketik untuk ribuan baris.
    - **C:** SUM untuk menjumlahkan angka.
    - **D:** Conditional Formatting hanya mewarnai sel.
15. **A. Angka tersimpan sebagai TEKS (biasanya hasil impor/copy dari sistem lain)** — Angka yang tersimpan sebagai teks tidak ikut dihitung SUM. Cirinya: rata kiri dan ada segitiga hijau. Perbaiki dengan klik ikon peringatan → Convert to Number, atau =VALUE(D2), atau Data → Text to Columns → Finish.
    - **B:** SUM bekerja normal untuk angka sungguhan.
    - **C:** Kolom sempit menampilkan ####, bukan hasil 0.
    - **D:** Ini masalah format data, bukan versi Excel.
16. **upper** — UPPER → 'RINA WULANDARI'. LOWER → huruf kecil semua. PROPER → kapital di awal tiap kata. Untuk kode atau kota yang akan dicocokkan, menyeragamkan huruf membantu menghindari data dobel.

Tanggal & Waktu — Analisis per Periode

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Laporan bisnis hampir selalu bertanya: *bulan ini berapa? telat berapa hari? jatuh tempo kapan?* Kuncinya satu: **Excel menyimpan tanggal sebagai angka hari** (1 Januari 2026 = 46023). Karena itu tanggal bisa dikurangkan, ditambah, dan dibandingkan.

📄 **Sheet: Transaksi**

	A	B	C	D	E
1	**Faktur**	**Tanggal**	**Sales**	**Total**	**Tgl_Bayar**
2	INV-01	03/01/2026	Andi	14.000.000	10/01/2026
3	INV-02	15/01/2026	Bima	1.500.000	20/02/2026
4	INV-03	28/01/2026	Citra	6.000.000	05/02/2026
5	INV-04	02/02/2026	Andi	750.000	02/02/2026
6	INV-05	14/02/2026	Dewi	7.000.000	30/03/2026
7	INV-06	27/02/2026	Bima	1.200.000	06/03/2026
8	INV-07	05/03/2026	Citra	7.000.000	12/03/2026
9	INV-08	18/03/2026	Andi	4.000.000	

```
=DAY(B2) / =MONTH(B2) / =YEAR(B2)   → 3 / 1 / 2026
=E3-B3                  → selisih hari (36)
=B2+30                  → jatuh tempo 30 hari
=EOMONTH(B6,0)          → akhir bulan: 28/02/2026
=TODAY()                → tanggal hari ini (selalu terbaru)
=NETWORKDAYS(awal,akhir) → hari kerja Senin–Jumat
=COUNTBLANK(E2:E9)      → faktur belum dibayar (1)
```

Total per bulan dengan SUMIFS dan rentang tanggal:

```
=SUMIFS(D2:D9, B2:B9,">="&DATE(2026,2,1), B2:B9,"
<"&DATE(2026,3,1))
→ 8.950.000 (semua transaksi Februari)
```

Perhatikan `">="&DATE(...)`: operator ditulis sebagai teks lalu disambung dengan nilai tanggal.

Status telat bayar: `=IF(E2-B2>30,"Telat","Tepat waktu")` → INV-02 (36 hari) dan INV-05 (44 hari) telat.

🌏 **Hati-hati format:** Indonesia dd/mm/yyyy, Amerika mm/dd/yyyy. 03/01/2026 bisa berarti 3 Januari atau 1 Maret. Tanggal yang tampil **rata kiri** biasanya masih teks — ubah dengan Text to Columns (format DMY). Saat bertukar data, pakai format ISO `2026-01-03`.

🧪 **Praktik di Excel:** file latihan → sheet *Bab8-Tanggal*.

🧠 Uji Diri — 14 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Transaksi**

	A	B	C	D	E
1	**Faktur**	**Tanggal**	**Sales**	**Total**	**Tgl_Bayar**
2	INV-01	03/01/2026	Andi	14.000.000	10/01/2026
3	INV-02	15/01/2026	Bima	1.500.000	20/02/2026
4	INV-03	28/01/2026	Citra	6.000.000	05/02/2026
5	INV-04	02/02/2026	Andi	750.000	02/02/2026
6	INV-05	14/02/2026	Dewi	7.000.000	30/03/2026
7	INV-06	27/02/2026	Bima	1.200.000	06/03/2026
8	INV-07	05/03/2026	Citra	7.000.000	12/03/2026
9	INV-08	18/03/2026	Andi	4.000.000	

1. **Kenapa di Excel kamu bisa mengurangkan dua tanggal, misalnya =E2-B2?**

   A. Excel menyimpan tanggal sebagai ANGKA (urutan hari sejak 1 Januari 1900); yang kamu lihat hanya formatnya
   B. Excel punya mode khusus untuk pengurangan tanggal
   C. Karena tanggal ditulis dengan garis miring
   D. Tidak bisa, hasilnya pasti error

2. 📄 Data: Transaksi

Hasil rumus ini adalah... (B4 = 28/01/2026, format hari/bulan/tahun)

```
=MONTH(B4)
```

A. 28
B. 1
C. 2026
D. Januari

3. 📄 Data: Transaksi

INV-02 dibayar berapa hari setelah tanggal faktur? Hasil rumus ini...

```
=E3-B3
```

A. 36
B. 5
C. 30
D. -36

4. 📄 Data: Transaksi

Rumus ini ditulis di F2 lalu ditarik sampai F8 (INV-01 s.d. INV-07). Berapa faktur yang berstatus "Telat"?

```
=IF(E2-B2>30,"Telat","Tepat waktu")
```

A. 1
B. 2
C. 3
D. 0

5. 📄 Data: Transaksi

   Hasil rumus ini adalah... (B6 = 14/02/2026)

   ```
   =EOMONTH(B6,0)
   ```

   A. 28/02/2026
   B. 29/02/2026
   C. 14/03/2026
   D. 01/03/2026

6. **Jatuh tempo faktur = 30 hari setelah tanggal faktur di B2. Rumusnya:**

   A. =B2+30
   B. =B2&30
   C. =DATE(B2+30)
   D. =MONTH(B2)+1

7. 📄 Data: Transaksi

   Hasil rumus ini adalah...

   ```
   =SUMIFS(D2:D9, B2:B9,">="&DATE(2026,2,1), B2:B9,"
   <"&DATE(2026,3,1))
   ```

   A. 8.950.000
   B. 21.500.000
   C. 11.000.000
   D. 41.450.000

8. **Kenapa syarat tanggal di SUMIFS ditulis ">="&DATE(2026,2,1), bukan ">=DATE(2026,2,1)"?**

   A. Syarat SUMIFS berupa teks; & menyambung operator ">=" dengan NILAI tanggal hasil DATE()
   B. Supaya rumus lebih pendek
   C. DATE tidak boleh dipakai di SUMIFS
   D. Tidak ada bedanya

9. 📄 Data: Transaksi

   Hasil rumus ini dan artinya:

   ```
   =COUNTBLANK(E2:E9)
   ```

   A. 1 — ada satu faktur yang belum dibayar (INV-08)
   B. 0 — semua sudah dibayar
   C. 8 — jumlah faktur
   D. 7 — jumlah faktur lunas

10. **=TODAY()-B9 dipakai untuk menghitung umur piutang INV-08. Apa yang perlu kamu ketahui tentang TODAY()?**

    A. Hasilnya berubah otomatis mengikuti tanggal hari ini setiap kali file dihitung ulang/dibuka
    B. Hasilnya tetap tanggal saat rumus pertama kali diketik
    C. TODAY() hanya ada di Google Sheets
    D. TODAY() butuh argumen tanggal

11. **Hasil rumus ini adalah... (2 Maret 2026 hari Senin, 6 Maret hari Jumat)**

    ```
    =NETWORKDAYS(DATE(2026,3,2), DATE(2026,3,6))
    ```

    A. 5
    B. 4
    C. 7
    D. 3

12. **Kolom tanggal hasil impor tampil rata KIRI dan =MONTH(B2) menghasilkan #VALUE!. Masalahnya:**

    A. Tanggal tersimpan sebagai TEKS, bukan tanggal sungguhan
    B. Fungsi MONTH salah ketik
    C. Tanggalnya di masa depan
    D. Kolomnya harus disembunyikan

13. **File dari kantor pusat di Amerika berisi tanggal 03/01/2026. Di laptopmu (region Indonesia) terbaca 3 Januari. Apa risikonya?**

    A. Di format Amerika (bulan/hari) itu artinya 1 Maret 2026 — tertukarnya bulan & hari bisa merusak laporan bulanan
    B. Tidak ada risiko, semua negara sama
    C. Excel selalu otomatis benar
    D. Tanggalnya akan terhapus

14. 📄 Data: Transaksi

    Hasil rumus ini di Excel berbahasa Inggris (B2 = 03/01/2026):

    ```
    =TEXT(B2,"mmm yyyy")
    ```

    A. Jan 2026
    B. 01 2026
    C. 3 Jan
    D. January 2026

Kunci & Pembahasan · Excel Bab 8

← Kembali ke soal · Awal bab

1. **A. Excel menyimpan tanggal sebagai ANGKA (urutan hari sejak 1 Januari 1900); yang kamu lihat hanya formatnya** — Di balik layar, 1 Januari 2026 adalah angka 46023. Karena tanggal adalah angka, kamu bisa menghitung selisih hari, menambah hari (=B2+30 untuk jatuh tempo), mengurutkan, dan menjumlah per rentang tanggal. Coba ubah format sel tanggal ke 'General' untuk melihat angkanya.
   - **B:** Tidak ada mode khusus; tanggal memang angka biasa.
   - **C:** Garis miring hanya tampilan format.
   - **D:** Bisa — dan ini salah satu hal paling berguna di Excel.

2. **B. 1** — MONTH mengambil nomor BULAN dari sebuah tanggal → 1 (Januari). Pasangannya: DAY → 28, YEAR → 2026. Kolom bantu 'Bulan' seperti ini memudahkan ringkasan per bulan dengan SUMIFS atau Pivot.
   - **A:** 28 adalah hasil DAY.
   - **C:** 2026 adalah hasil YEAR.
   - **D:** MONTH mengembalikan angka, bukan nama bulan. Untuk nama bulan pakai TEXT.

3. **A. 36** — 15 Januari → 31 Januari = 16 hari, ditambah 20 hari di Februari = 36 hari. Pastikan format sel hasilnya 'General' atau 'Number' — kalau masih format tanggal, angka 36 tampil sebagai 05/02/1900.
   - **B:** 5 hanya selisih tanggalnya (20 − 15), mengabaikan bedanya bulan.
   - **C:** Benar hanya kalau setiap bulan 30 hari. Excel menghitung hari yang sebenarnya.
   - **D:** Hasil negatif muncul kalau urutannya dibalik: =B3-E3.

4. **B. 2** — Selisih hari: INV-01 = 7, INV-02 = 36, INV-03 = 8, INV-04 = 0, INV-05 = 44, INV-06 = 7, INV-07 = 7. Yang lebih dari 30 hari: INV-02 dan INV-05 → 2 faktur. Ini analisis umur piutang (aging) sederhana yang sering diminta tim finance.
   - **A:** INV-05 (14/02 → 30/03) juga lebih dari 30 hari: 44 hari.
   - **C:** INV-03 hanya 8 hari (28/01 → 05/02), tidak telat.
   - **D:** INV-02 (36 hari) dan INV-05 (44 hari) jelas lebih dari 30 hari.

5. **A. 28/02/2026** — EOMONTH = End Of MONTH. Argumen 0 = akhir bulan yang sama → 28 Februari 2026 (2026 bukan tahun kabisat). EOMONTH(B6,1) = akhir bulan berikutnya (31/03/2026). Berguna untuk jatuh tempo 'akhir bulan' dan untuk mengelompokkan tanggal per bulan.
   - **B:** 2026 bukan tahun kabisat; Februari 2026 hanya 28 hari.
   - **C:** Itu tanggal + 1 bulan (fungsi EDATE), bukan akhir bulan.
   - **D:** Itu awal bulan berikutnya = EOMONTH(B6,0)+1.

6. **A. =B2+30** — Karena tanggal adalah angka hari, menambah 30 berarti maju 30 hari. Format hasilnya sebagai tanggal.
   - **B:** & menyambung teks: hasilnya '4602630' — bukan tanggal.
   - **C:** DATE butuh 3 argumen (tahun, bulan, hari).
   - **D:** Itu nomor bulan berikutnya, bukan tanggal jatuh tempo.

7. **A. 8.950.000** — Dua syarat pada kolom tanggal yang sama membentuk rentang: ≥ 1 Februari DAN < 1 Maret = semua transaksi Februari: INV-04 (750.000) + INV-05 (7.000.000) + INV-06 (1.200.000) = 8.950.000. Memakai '< 1 Maret' lebih aman daripada '<= 28 Februari' karena kamu tidak perlu tahu jumlah hari tiap bulan.
   - **B:** Itu total Januari.
   - **C:** Itu total Maret.
   - **D:** Itu total semua transaksi tanpa syarat.

8. **A. Syarat SUMIFS berupa teks; & menyambung operator ">=" dengan NILAI tanggal hasil DATE()** — Semua yang ada di dalam kutip dianggap teks apa adanya, jadi ">=DATE(2026,2,1)" tidak pernah dihitung. Dengan &, DATE dihitung dulu menjadi angka tanggal (46054), lalu digabung menjadi syarat ">=46054". Pola yang sama dipakai untuk syarat dari sel: ">="&H1.
   - **B:** Malah lebih panjang — tapi wajib supaya benar.
   - **C:** DATE boleh, asalkan di luar tanda kutip.
   - **D:** Bedanya besar: versi dalam kutip tidak akan cocok dengan data apa pun.

9. **A. 1 — ada satu faktur yang belum dibayar (INV-08)** — COUNTBLANK menghitung sel KOSONG. Hanya E9 (INV-08) yang belum berisi tanggal bayar. Kebalikannya, COUNTA(E2:E9) = 7 faktur yang sudah dibayar.
   - **B:** E9 kosong — INV-08 belum dibayar.
   - **C:** 8 adalah jumlah semua faktur (COUNTA kolom A).
   - **D:** 7 adalah jumlah sel yang TERISI (COUNTA), bukan yang kosong.

10. **A. Hasilnya berubah otomatis mengikuti tanggal hari ini setiap kali file dihitung ulang/dibuka** — TODAY() mengambil tanggal dari komputer dan selalu diperbarui. Cocok untuk laporan yang harus 'terkini'. Kalau butuh tanggal tetap (misal tanggal input data), tekan Ctrl+; untuk mengetik tanggal hari ini sebagai nilai mati.
    - **B:** Itu perilaku Ctrl+; (nilai tetap), bukan TODAY().
    - **C:** TODAY() ada di Excel dan Google Sheets.
    - **D:** TODAY() ditulis tanpa argumen: =TODAY().

11. **A. 5** — NETWORKDAYS menghitung hari KERJA (Senin–Jumat) di antara dua tanggal, termasuk tanggal awal dan akhir: Senin–Jumat = 5. Argumen ke-3 bisa diisi daftar tanggal libur nasional. Dipakai untuk SLA, misalnya 'pesanan dikirim maksimal 3 hari kerja'.
    - **B:** Itu hasil pengurangan biasa (6 − 2). NETWORKDAYS menghitung tanggal awal DAN akhir.
    - **C:** Rentang ini tidak mencakup akhir pekan, dan NETWORKDAYS tidak pernah menghitung Sabtu–Minggu.
    - **D:** Terlalu sedikit; Senin sampai Jumat ada 5 hari kerja.

12. **A. Tanggal tersimpan sebagai TEKS, bukan tanggal sungguhan** — Tanggal sungguhan (angka) tampil rata kanan; teks rata kiri dan tidak bisa dihitung. Perbaiki dengan Data → Text to Columns → pilih format DMY, atau rakit ulang dengan =DATE(RIGHT(B2,4), MID(B2,4,2), LEFT(B2,2)) untuk teks berformat dd/mm/yyyy.
    - **B:** Kalau salah ketik, error-nya #NAME?, bukan #VALUE!.
    - **C:** Tanggal masa depan tetap bisa dihitung.
    - **D:** Menyembunyikan kolom tidak mengubah isinya.

13. **A. Di format Amerika (bulan/hari) itu artinya 1 Maret 2026 — tertukarnya bulan & hari bisa merusak laporan bulanan** — AS memakai mm/dd/yyyy, Indonesia dd/mm/yyyy. Tanda data tertukar: tanggal ≤ 12 'terlihat' benar padahal salah, dan tanggal > 12 malah tersimpan sebagai teks. Saat bertukar data, pakai format ISO yyyy-mm-dd (2026-01-03), lalu cek dengan =MONTH() setelah impor.
    - **B:** Format tanggal berbeda antarnegara.
    - **C:** Excel menebak berdasarkan pengaturan region komputer — bisa salah.
    - **D:** Tanggal tidak hilang, tapi maknanya berubah.
14. **A. Jan 2026** — TEXT mengubah tanggal menjadi TEKS berformat: 'mmm' = nama bulan singkat, 'mmmm' = nama bulan lengkap, 'yyyy' = tahun 4 digit. Catatan: di Excel berbahasa lain, kode format bisa berbeda, dan hasilnya berupa teks (tidak bisa dihitung lagi). Untuk ringkasan per bulan, Pivot Table → Group → Months sering lebih praktis.
    - **B:** '01' muncul dengan kode 'mm' (dua huruf m), bukan 'mmm'.
    - **C:** Format ini tidak menampilkan tanggal (d).
    - **D:** Nama bulan lengkap muncul dengan 'mmmm' (empat huruf m).

EXCEL · BAB 9

INDEX-MATCH & Dynamic Array (FILTER, UNIQUE, SORT)

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Bab ini melengkapi XLOOKUP dengan dua hal: **INDEX-MATCH** (wajib untuk Excel versi lama dan tes kerja) dan **dynamic array** (Excel 365/Google Sheets) yang membuat laporan otomatis.

📄 **Sheet: Produk**

	A	B	C
1	**Produk**	**Kategori**	**Harga**
2	Laptop	Komputer	7.000.000
3	Mouse	Aksesoris	150.000
4	Monitor	Komputer	2.000.000
5	Keyboard	Aksesoris	300.000

MATCH mencari POSISI, **INDEX** mengambil ISI pada posisi itu:

```
=MATCH("Monitor", A2:A5, 0)          → 3   (urutan ke-3; 0 = cari yang persis)
=INDEX(C2:C5, 3)                     → 2.000.000
=INDEX(C2:C5, MATCH("Monitor",A2:A5,0))  → 2.000.000
```

Kelebihannya dibanding VLOOKUP: bisa mencari **ke kiri**, tidak rusak saat kolom disisipkan, dan bisa **dua arah**:

```
=INDEX(B2:D5, MATCH("Citra",A2:A5,0), MATCH("Feb",B1:D1,0))
```

Tarif bertingkat (komisi, diskon, grade) tanpa IF panjang — cari batas bawah terdekat:

📄 **Sheet: Komisi**

	A	B
1	**Batas_Bawah**	**Komisi**
2	0	2%
3	5.000.000	3%
4	10.000.000	5%

```
=XLOOKUP(7000000, A2:A4, B2:B4, , -1)   → 3%
=VLOOKUP(7000000, A2:B4, 2, TRUE)       → 3%  (versi lama, batas harus urut naik)
```

Dynamic array — satu rumus, banyak hasil yang "tumpah" (spill) ke bawah:

```
=UNIQUE(A2:A11)                 → Andi, Bima, Citra, Dewi
=SORT(UNIQUE(C2:C11))           → Keyboard, Laptop, Monitor, Mouse
=FILTER(A2:F11, B2:B11="Bandung")  → 3 baris Bandung
=COUNTA(UNIQUE(B2:B11))         → 3 wilayah unik
```

⚠️ **#SPILL!** artinya area tempat hasil akan tumpah tidak kosong. Kosongkan sel penghalang. Dan ingat: VLOOKUP tanpa argumen ke-4 memakai pencocokan *perkiraan* — selalu tulis `FALSE` untuk pencarian persis.

🧪 **Praktik di Excel:** file latihan → sheet *Bab9-Lookup*.

🧠 Uji Diri — 14 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Produk**

	A	B	C
1	**Produk**	**Kategori**	**Harga**
2	Laptop	Komputer	7.000.000
3	Mouse	Aksesoris	150.000
4	Monitor	Komputer	2.000.000
5	Keyboard	Aksesoris	300.000

📄 **Sheet: Komisi**

	A	B
1	**Batas_Bawah**	**Komisi**
2	0	2%
3	5.000.000	3%
4	10.000.000	5%

📄 **Sheet: Penjualan**

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

1. 📄 Data: Produk

   Hasil rumus ini adalah...

   ```
   =MATCH("Monitor", A2:A5, 0)
   ```

   A. 3
   B. 4
   C. 2.000.000
   D. Komputer

2. 📄 Data: Produk

   Hasil rumus ini adalah...

   ```
   =INDEX(C2:C5, 2)
   ```

   A. 150.000
   B. 7.000.000
   C. Mouse
   D. 2

3. 📄 Data: Produk

   Hasil rumus ini adalah...

   ```
   =INDEX(C2:C5, MATCH("Keyboard", A2:A5, 0))
   ```

   A. 300.000
   B. 4
   C. Aksesoris
   D. #N/A

4. **XLOOKUP sudah ada. Kenapa INDEX-MATCH tetap layak dipelajari?**

   A. Banyak kantor masih memakai Excel 2016/2019 yang belum punya XLOOKUP, dan INDEX-MATCH sering muncul di tes kerja
   B. INDEX-MATCH selalu lebih cepat dari semua fungsi
   C. XLOOKUP akan dihapus Microsoft
   D. INDEX-MATCH bisa membuat grafik

5. 📄 Data: Produk

   Kamu ingin mencari produk berdasarkan HARGA (kolom C) dan mengambil NAMA-nya (kolom A, di sebelah kiri). Mana yang TIDAK bisa melakukannya secara langsung?

   A. VLOOKUP
   B. INDEX-MATCH
   C. XLOOKUP
   D. FILTER

6. 📄 Data: Komisi

   Total penjualan Citra 7.000.000. Hasil rumus ini adalah...

   ```
   =XLOOKUP(7000000, A2:A4, B2:B4, , -1)
   ```

   A. 3%
   B. 2%
   C. 5%
   D. #N/A

7. 📄 Data: Produk

   Rumus =VLOOKUP("Mouse", A2:C5, 3) — tanpa argumen ke-4 — kadang mengambil harga produk lain. Kenapa?

   A. Tanpa argumen ke-4, VLOOKUP memakai pencocokan PERKIRAAN yang mengharuskan data terurut; untuk pencarian persis tulis FALSE
   B. VLOOKUP tidak bisa mencari teks
   C. Kolom ke-3 tidak ada
   D. 'Mouse' harus ditulis huruf kecil

8. 📄 Data: Penjualan

   Rumus ini ditulis di H2 (Excel 365). Hasilnya...

   ```
   =UNIQUE(A2:A11)
   ```

   A. Daftar nama sales tanpa duplikat (Andi, Bima, Citra, Dewi) yang otomatis 'tumpah' ke H2:H5
   B. Hanya 'Andi' di H2
   C. Angka 4
   D. 10 nama termasuk yang dobel

9. 📄 Data: Penjualan

   Berapa BARIS yang dihasilkan rumus ini?

   ```
   =FILTER(A2:F11, B2:B11="Bandung")
   ```

   A. 3
   B. 4
   C. 10
   D. 1

10. 📄 Data: Penjualan

    Hasil rumus ini adalah...

    ```
    =FILTER(A2:A11, F2:F11>=7000000)
    ```

    A. Andi, Dewi, Citra
    B. Andi saja
    C. Andi, Citra, Dewi, Bima
    D. Dewi, Citra

11. 📄 Data: Penjualan

    Nilai PERTAMA (paling atas) hasil rumus ini adalah...

    ```
    =SORT(UNIQUE(C2:C11))
    ```

    A. Keyboard
    B. Laptop
    C. Mouse
    D. Monitor

12. **Rumus =UNIQUE(A2:A11) di H2 menampilkan #SPILL!. Penyebab paling umum:**

    A. Ada sel yang tidak kosong di area H3:H5 yang akan diisi hasilnya
    B. Fungsi UNIQUE salah ketik
    C. Data A2:A11 kosong
    D. Excel kehabisan memori

13. 📄 Data: Penjualan

    Hasil rumus ini adalah...

    ```
    =COUNTA(UNIQUE(B2:B11))
    ```

    A. 3
    B. 10
    C. 4
    D. 1

14. **Tabel target: nama sales di A2:A5, bulan Jan–Mar di B1:D1, angka target di B2:D5. Rumus untuk mengambil target 'Citra' di bulan 'Feb':**

    A. =INDEX(B2:D5, MATCH("Citra",A2:A5,0), MATCH("Feb",B1:D1,0))
    B. =VLOOKUP("Citra","Feb",B2:D5)
    C. =INDEX(B2:D5, "Citra", "Feb")
    D. =MATCH("Citra","Feb")

Kunci & Pembahasan · Excel Bab 9

← Kembali ke soal · Awal bab

1. **A. 3** — MATCH mengembalikan POSISI (urutan ke berapa) di dalam range, bukan nomor baris sheet. Di A2:A5: Laptop = 1, Mouse = 2, Monitor = 3. Angka 0 di akhir artinya cari yang persis sama (exact match).
   - **B:** 4 adalah nomor BARIS di sheet. MATCH menghitung posisi di dalam range A2:A5, dimulai dari 1.
   - **C:** MATCH tidak mengambil isi kolom lain; itu tugas INDEX atau XLOOKUP.
   - **D:** Itu isi kolom B, bukan posisi.
2. **A. 150.000** — INDEX(range, n) mengambil isi ke-n dari range. Isi ke-2 dari C2:C5 adalah C3 = 150.000 (harga Mouse).
   - **B:** 7.000.000 adalah isi ke-1 (Laptop).
   - **C:** Mouse ada di kolom A, sedangkan range-nya kolom C.
   - **D:** INDEX mengembalikan isi sel, bukan angka posisinya.
3. **A. 300.000** — Kerjakan dari dalam: MATCH mencari 'Keyboard' → posisi 4. INDEX(C2:C5, 4) mengambil isi ke-4 kolom Harga → 300.000. Inilah 'XLOOKUP versi lama': MATCH mencari posisi, INDEX mengambil isinya.
   - **B:** 4 hanya hasil MATCH; INDEX lalu mengambil isinya.
   - **C:** Aksesoris ada di kolom B, sedangkan INDEX mengambil dari kolom C.
   - **D:** Keyboard ada di tabel, jadi tidak muncul #N/A.
4. **A. Banyak kantor masih memakai Excel 2016/2019 yang belum punya XLOOKUP, dan INDEX-MATCH sering muncul di tes kerja** — XLOOKUP hanya ada di Excel 2021/365 dan Google Sheets. Di Excel 2016/2019 pilihannya VLOOKUP atau INDEX-MATCH. INDEX-MATCH lebih fleksibel daripada VLOOKUP: bisa mencari ke kiri dan tidak rusak saat kolom disisipkan.
   - **B:** Perbedaan kecepatannya jarang terasa di data sehari-hari.
   - **C:** Tidak ada rencana seperti itu.
   - **D:** Grafik dibuat dari menu Insert → Chart.

5. **A. VLOOKUP** — VLOOKUP hanya bisa mencari di kolom PALING KIRI tabel lalu mengambil ke kanan. Mengambil ke kiri tidak bisa tanpa trik. INDEX-MATCH, XLOOKUP, dan FILTER bebas arah.
   - **B:** Bisa: =INDEX(A2:A5, MATCH(2000000, C2:C5, 0)).
   - **C:** Bisa: =XLOOKUP(2000000, C2:C5, A2:A5).
   - **D:** Bisa: =FILTER(A2:A5, C2:C5=2000000).

6. **A. 3%** — match_mode −1 = cari yang persis; kalau tidak ada, ambil nilai lebih KECIL terdekat. 7.000.000 tidak ada di tabel, dan nilai lebih kecil terdekat adalah 5.000.000 → 3%. Inilah cara membuat tarif bertingkat (komisi, diskon, grade) tanpa IF bertingkat yang panjang. Versi lama: =VLOOKUP(7000000, A2:B4, 2, TRUE) dengan batas bawah diurutkan naik.
   - **B:** 2% berlaku untuk penjualan di bawah 5 juta.
   - **C:** 5% baru berlaku mulai 10 juta.
   - **D:** Tanpa match_mode memang #N/A, tapi rumus ini memakai −1.

7. **A. Tanpa argumen ke-4, VLOOKUP memakai pencocokan PERKIRAAN yang mengharuskan data terurut; untuk pencarian persis tulis FALSE** — Default VLOOKUP adalah approximate match. Pada data yang tidak diurutkan, hasilnya bisa salah TANPA pesan error — berbahaya karena tidak kelihatan. Selalu tulis =VLOOKUP("Mouse",A2:C5,3,FALSE). XLOOKUP lebih aman karena default-nya exact match.
   - **B:** VLOOKUP bisa mencari teks.
   - **C:** Range A2:C5 punya 3 kolom, jadi kolom ke-3 (Harga) ada.
   - **D:** Pencarian di Excel tidak peka huruf besar/kecil.

8. **A. Daftar nama sales tanpa duplikat (Andi, Bima, Citra, Dewi) yang otomatis 'tumpah' ke H2:H5** — UNIQUE adalah dynamic array: satu rumus menghasilkan banyak nilai yang otomatis mengisi sel di bawahnya (spill). Hasilnya cocok dipakai sebagai daftar baris laporan, lalu di sebelahnya ditulis SUMIFS per nama.
   - **B:** Dynamic array mengisi beberapa sel sekaligus, bukan satu.
   - **C:** Angka 4 adalah hasil COUNTA(UNIQUE(A2:A11)).
   - **D:** UNIQUE justru membuang yang dobel.

9. **A. 3** — FILTER mengambil semua baris yang memenuhi syarat. Wilayah Bandung ada di baris 3 (Bima, Mouse), 6 (Dewi, Laptop), dan 11 (Bima, Monitor) → 3 baris lengkap dengan 6 kolom. Hasilnya ikut berubah saat data sumber berubah — beda dengan copy-paste hasil filter manual.
    - **B:** Jakarta yang punya 4 baris.
    - **C:** 10 adalah semua baris tanpa syarat.
    - **D:** FILTER mengembalikan SEMUA baris yang cocok, bukan hanya yang pertama.
10. **A. Andi, Dewi, Citra** — Total ≥ 7 juta ada di baris 2 (Andi, 14 juta), 6 (Dewi, 7 juta), dan 8 (Citra, 7 juta). Urutan hasil mengikuti urutan data asli. Bungkus dengan SORT jika ingin diurutkan.
    - **B:** ≥ berarti 7 juta juga termasuk.
    - **C:** Bima tidak punya transaksi ≥ 7 juta.
    - **D:** Andi (14 juta) juga memenuhi syarat.
11. **A. Keyboard** — UNIQUE menghasilkan Laptop, Mouse, Monitor, Keyboard (urutan kemunculan). SORT lalu mengurutkan A→Z: Keyboard, Laptop, Monitor, Mouse.
    - **B:** Laptop paling atas sebelum diurutkan. SORT mengurutkan A→Z.
    - **C:** Mouse justru paling akhir secara abjad.
    - **D:** Monitor ada di urutan ke-3 setelah diurutkan.
12. **A. Ada sel yang tidak kosong di area H3:H5 yang akan diisi hasilnya** — Dynamic array butuh ruang kosong untuk 'tumpah'. Kalau ada isi di jalurnya (bahkan satu spasi), muncul #SPILL!. Klik sel error untuk melihat garis putus-putus area yang dibutuhkan, lalu kosongkan sel penghalangnya.
    - **B:** Salah ketik nama fungsi menghasilkan #NAME?.
    - **C:** Data kosong menghasilkan hasil kosong/#CALC!, bukan #SPILL!.
    - **D:** Ini masalah ruang di sheet, bukan memori.
13. **A. 3** — Wilayah unik: Jakarta, Bandung, Surabaya → 3. Pola 'hitung nilai unik' ini setara dengan COUNT(DISTINCT wilayah) di SQL.
    - **B:** 10 adalah semua baris termasuk yang dobel.
    - **C:** Ada 4 sales unik, tapi wilayah unik hanya 3.
    - **D:** UNIQUE mengembalikan semua nilai unik, bukan satu.

14. **A. =INDEX(B2:D5, MATCH("Citra",A2:A5,0), MATCH("Feb",B1:D1,0))** — INDEX bisa menerima nomor BARIS dan nomor KOLOM. MATCH pertama mencari baris Citra, MATCH kedua mencari kolom Feb. Ini disebut pencarian dua arah (two-way lookup) — sering dipakai untuk tabel target vs realisasi.
    - **B:** Susunan argumen VLOOKUP tidak seperti itu, dan VLOOKUP tidak bisa mencari kolom berdasarkan judul tanpa MATCH.
    - **C:** INDEX butuh ANGKA posisi, bukan teks. Teks harus diubah jadi posisi dengan MATCH.
    - **D:** MATCH hanya mengembalikan posisi, dan argumennya pun salah.

Error, IFERROR & Data yang Rapi

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Error di Excel bukan musuh — ia **alarm** yang memberi tahu apa yang salah. Kenali tujuh yang paling sering:

Error	Artinya	Biasanya karena
`#DIV/0!`	bagi dengan nol	pembagi kosong/0
`#N/A`	tidak ditemukan	lookup gagal, spasi tersembunyi
`#NAME?`	nama tidak dikenal	salah ketik fungsi, teks tanpa kutip
`#REF!`	referensi hilang	kolom/baris yang dirujuk dihapus
`#VALUE!`	jenis data salah	teks ikut dihitung ("2 unit")
`#SPILL!`	tidak bisa tumpah	area hasil dynamic array tidak kosong
`#####`	bukan error	kolom terlalu sempit

Menangani error dengan bijak:

```
=IFERROR(D2/C2, 0)                          → semua error diganti
0
=IFNA(XLOOKUP(H2,A2:A5,C2:C5), "Tidak ada")  → hanya #N/A yang
diganti
=XLOOKUP(H2, A2:A5, C2:C5, "Tidak ada")      → paling ringkas
```

⚠️ Jangan bungkus semua rumus dengan `IFERROR(…,0)`. Kesalahan rumus sungguhan ikut tersembunyi, dan angka 0 palsu masuk ke total.

Kebiasaan data rapi yang membuat laporanmu jarang error:

- **Satu sel, satu informasi**: Qty = 2 dan Satuan = unit di kolom berbeda.
- **Data Validation** (dropdown) agar input konsisten.
- **Excel Table** (Ctrl+T) → `=SUM(DataPenjualan[Total])` otomatis ikut bertambah.
- **ROUND** jika nilai memang harus dibulatkan; format desimal hanya mengubah tampilan.
- **Freeze Panes** agar judul kolom tetap terlihat.

🧪 **Praktik di Excel:** file latihan → sheet *Bab10-Error*.

🧠 Uji Diri — 14 soal

1. **Rumus =D2/C2 menampilkan #DIV/0!. Artinya…**

   A. Pembagi (C2) bernilai 0 atau kosong
   B. Nama fungsi salah ketik
   C. Data yang dicari tidak ditemukan
   D. Kolomnya terlalu sempit

2. **Fungsi lookup (XLOOKUP/VLOOKUP/MATCH) tidak menemukan nilai yang dicari. Error yang muncul:**

   A. #N/A
   B. #REF!
   C. #DIV/0!
   D. #NUM!

3. **=SUMIFF(F2:F11, A2:A11, "Andi") menampilkan #NAME?. Penyebabnya:**

   A. Nama fungsi salah ketik (seharusnya SUMIFS atau SUMIF)
   B. Data Andi tidak ada
   C. Pemisahnya harus titik koma
   D. Range-nya terlalu besar

4. **Kemarin rumus =SUM(F2:F11)/H1 normal. Hari ini muncul #REF! setelah kamu menghapus kolom H. Kenapa?**

   A. Sel yang dirujuk rumus (H1) sudah terhapus, jadi referensinya hilang
   B. Kolom F ikut terhapus
   C. SUM tidak boleh dibagi
   D. File perlu disimpan ulang

5. **=D2*E2 menghasilkan #VALUE! hanya di satu baris. Ternyata D2 berisi '2 unit'. Penyebabnya…**

   A. D2 berisi teks ('2 unit'), jadi tidak bisa dikalikan
   B. E2 terlalu besar
   C. Perkalian harus memakai PRODUCT
   D. Baris itu disembunyikan

6. **Jika C2 kosong, hasil rumus ini adalah...**

```
=IFERROR(D2/C2, 0)
```

   A. 0
   B. #DIV/0!
   C. Sel kosong
   D. Isi D2

7. **Kenapa analis berpengalaman tidak membungkus SEMUA rumus dengan IFERROR(..., 0)?**

   A. IFERROR menyembunyikan semua error, termasuk kesalahan rumus sungguhan — angka 0 palsu bisa masuk ke total tanpa ketahuan
   B. IFERROR membuat file lebih besar
   C. IFERROR tidak bisa dipakai bersama XLOOKUP
   D. Angka 0 tidak boleh muncul di laporan

8. **Untuk membungkus lookup, IFNA lebih aman daripada IFERROR karena...**

   A. IFNA hanya menangkap #N/A (tidak ketemu); error lain seperti #REF! atau #VALUE! tetap terlihat
   B. IFNA lebih cepat dihitung
   C. IFNA bisa mengganti nilai yang benar
   D. IFERROR sudah tidak didukung

9. **Fungsi yang HANYA menangkap error #N/A: =____(XLOOKUP(H2,A2:A5,C2:C5),"Tidak ada")**

   Jawab: _____

10. **Supaya orang tidak salah ketik nama wilayah ('Jakrta', 'jakarta '), fitur yang tepat:**

    A. Data → Data Validation → List (dropdown pilihan)
    B. Conditional Formatting
    C. Freeze Panes
    D. Mengganti warna huruf

11. **Data diubah menjadi Excel Table (Ctrl+T) bernama DataPenjualan. Keuntungan =SUM(DataPenjualan[Total]) dibanding =SUM(F2:F11):**

    A. Range otomatis ikut bertambah saat ada baris baru, dan nama kolomnya mudah dibaca
    B. Hasilnya lebih akurat
    C. File bisa dibuka tanpa Excel
    D. Tidak ada bedanya

12. **Tiga sel berisi 1,4; 1,4; 1,4 dengan format 0 desimal, sehingga TAMPIL 1, 1, 1. SUM-nya tampil 4. Kenapa?**

    A. Format hanya mengubah TAMPILAN; nilai aslinya 1,4 sehingga totalnya 4,2 (tampil 4). Pakai ROUND jika nilainya memang harus dibulatkan
    B. Excel salah hitung
    C. SUM selalu membulatkan ke atas
    D. Ada sel tersembunyi

13. **Saat menggulir ke baris 500, judul kolom tidak terlihat lagi. Fitur agar baris judul selalu tampil:**

    A. View → Freeze Panes → Freeze Top Row
    B. Data → Sort
    C. Insert → Header
    D. Home → Wrap Text

14. **Sebuah sel menampilkan ######. Paling sering artinya:**

    A. Kolom terlalu sempit untuk menampilkan angka/tanggal — lebarkan kolomnya
    B. Rumusnya error
    C. Datanya terhapus
    D. File rusak

Kunci & Pembahasan · Excel Bab 10

← Kembali ke soal · Awal bab

1. **A. Pembagi (C2) bernilai 0 atau kosong** — #DIV/0! = pembagian dengan nol. Sering terjadi saat menghitung persentase atau rata-rata dan datanya belum lengkap.
   - **B:** Salah ketik nama fungsi menghasilkan #NAME?.
   - **C:** Data lookup yang tidak ditemukan menghasilkan #N/A.
   - **D:** Kolom sempit menampilkan #####.

2. **A. #N/A** — #N/A = Not Available (tidak tersedia). Biasanya karena nilainya memang tidak ada, salah ketik, atau ada spasi tersembunyi.
   - **B:** #REF! muncul saat sel yang dirujuk terhapus.
   - **C:** #DIV/0! muncul saat membagi dengan nol.
   - **D:** #NUM! muncul pada perhitungan angka yang tidak mungkin, misal akar dari bilangan negatif.

3. **A. Nama fungsi salah ketik (seharusnya SUMIFS atau SUMIF)** — #NAME? berarti Excel tidak mengenali sebuah nama: fungsi salah ketik, atau teks di dalam rumus lupa diberi tanda kutip (=COUNTIF(A2:A11, Andi)).
   - **B:** Kalau datanya tidak ada, SUMIFS menghasilkan 0, bukan error.
   - **C:** Pemisah yang salah biasanya ditolak saat mengetik rumus, bukan menghasilkan #NAME?.
   - **D:** Ukuran range tidak menyebabkan #NAME?.

4. **A. Sel yang dirujuk rumus (H1) sudah terhapus, jadi referensinya hilang** — #REF! = referensi tidak valid. Excel mengganti H1 dengan #REF! karena kolomnya sudah tidak ada. Perbaiki rumusnya atau batalkan penghapusan (Ctrl+Z). Pelajaran: cek dulu apakah ada rumus yang bergantung pada kolom yang akan dihapus.
   - **B:** Kolom F masih ada; yang hilang adalah H.
   - **C:** Hasil SUM boleh dibagi seperti angka biasa.
   - **D:** Menyimpan ulang tidak mengembalikan referensi yang hilang.

5. **A. D2 berisi teks ('2 unit'), jadi tidak bisa dikalikan** — #VALUE! = jenis data salah, misalnya teks ikut dihitung. Simpan angka dan satuan di kolom terpisah: Qty = 2, Satuan = 'unit'. Aturan data rapi: satu sel, satu jenis informasi.
   - **B:** Angka besar tidak menyebabkan #VALUE!.
   - **C:** Tanda * boleh dipakai; PRODUCT hanyalah alternatif.
   - **D:** Baris tersembunyi tetap dihitung normal.

6. **A. 0** — IFERROR(nilai, nilai_jika_error): kalau perhitungannya menghasilkan error APA PUN, tampilkan nilai pengganti. D2/C2 dengan C2 kosong → #DIV/0! → diganti 0.
   - **B:** Error itulah yang ditangkap IFERROR dan diganti 0.
   - **C:** Penggantinya 0, bukan kosong. Untuk kosong tulis "".
   - **D:** IFERROR mengganti dengan argumen kedua (0).

7. **A. IFERROR menyembunyikan semua error, termasuk kesalahan rumus sungguhan — angka 0 palsu bisa masuk ke total tanpa ketahuan** — Error adalah alarm. Kalau semua alarm dimatikan, salah ketik range atau data yang hilang ikut tersembunyi. Pakai IFERROR/IFNA secukupnya (misalnya hanya untuk lookup yang memang mungkin tidak ketemu), dan isi penggantinya dengan teks yang jelas seperti 'Tidak ada'.
   - **B:** Dampaknya ke ukuran file sangat kecil.
   - **C:** IFERROR bisa membungkus XLOOKUP.
   - **D:** 0 boleh muncul — masalahnya adalah 0 yang PALSU.

8. **A. IFNA hanya menangkap #N/A (tidak ketemu); error lain seperti #REF! atau #VALUE! tetap terlihat** — Lookup yang gagal menemukan data menghasilkan #N/A — itu yang wajar ditangani. Error lain menandakan rumusmu rusak dan sebaiknya tetap terlihat. Di XLOOKUP, kamu bahkan bisa memakai argumen ke-4 (if_not_found).
   - **B:** Perbedaan kecepatannya tidak berarti.
   - **C:** Tidak ada fungsi yang mengganti nilai yang benar.
   - **D:** IFERROR masih didukung penuh.

9. **ifna** — IFNA(nilai, pengganti) hanya bereaksi pada #N/A. Error lain tetap terlihat sehingga kesalahan rumus tidak tersembunyi.

10. **A. Data → Data Validation → List (dropdown pilihan)** — Data Validation membatasi isi sel: dropdown daftar wilayah, angka minimal/maksimal, atau rentang tanggal. Mencegah data kotor sejak awal jauh lebih murah daripada membersihkannya belakangan.
    - **B:** Conditional Formatting hanya mewarnai, tidak mencegah salah ketik.
    - **C:** Freeze Panes mengunci tampilan baris/kolom saat menggulir.
    - **D:** Warna huruf tidak membatasi isi sel.

11. **A. Range otomatis ikut bertambah saat ada baris baru, dan nama kolomnya mudah dibaca** — Structured reference terbaca seperti kalimat dan tidak perlu diedit setiap ada data baru. Pivot Table yang sumbernya Table juga otomatis mencakup baris baru setelah Refresh.
    - **B:** Hasilnya sama selama datanya sama; bedanya ada di perawatan saat data bertambah.
    - **C:** Tetap butuh Excel atau aplikasi yang kompatibel.
    - **D:** Bedanya terasa saat data bertambah setiap minggu.

12. **A. Format hanya mengubah TAMPILAN; nilai aslinya 1,4 sehingga totalnya 4,2 (tampil 4). Pakai ROUND jika nilainya memang harus dibulatkan** — Selisih 'receh' di laporan keuangan sering berasal dari sini. Jika aturan bisnisnya memang dibulatkan per baris, bulatkan NILAINYA dengan =ROUND(x,0) sebelum dijumlah.
    - **B:** Excel menghitung nilai asli dengan benar; tampilanlah yang menipu.
    - **C:** SUM tidak membulatkan apa pun.
    - **D:** Tidak perlu sel tersembunyi untuk menjelaskan selisih ini.

13. **A. View → Freeze Panes → Freeze Top Row** — Freeze Panes mengunci baris (dan/atau kolom) supaya tetap terlihat saat menggulir. Kecil, tapi sangat membantu saat memeriksa data besar.
    - **B:** Sort mengurutkan data.
    - **C:** Header di menu Insert untuk cetak halaman, bukan untuk mengunci tampilan.
    - **D:** Wrap Text membuat teks panjang turun ke baris berikutnya di dalam sel.

14. **A. Kolom terlalu sempit untuk menampilkan angka/tanggal — lebarkan kolomnya** — ##### bukan error rumus. Klik dua kali garis batas judul kolom untuk melebarkan otomatis. (Kasus langka: tanggal bernilai negatif juga tampil #####.)
    - **B:** Error rumus selalu diawali # lalu namanya, misal #N/A atau #REF!.
    - **C:** Isinya masih ada, hanya tidak muat ditampilkan.
    - **D:** File baik-baik saja.

BAGIAN 2

SQL

★ Prioritas #1 — Dipakai di hampir semua lowongan data, dan Odoo menyimpan datanya di PostgreSQL.

JOIN, GROUP BY, subquery, CASE, CTE, window function

8 bab · 84 soal

SELECT, WHERE, ORDER BY — Mengambil Data

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Semua latihan SQL memakai database toko kecil dengan 3 tabel:

🗄 Tabel: `pelanggan`

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

🗄 Tabel: `produk`

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

🗄 Tabel: `pesanan`

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

Pola dasar yang dipakai setiap hari:

```
SELECT kolom1, kolom2      -- kolom yang ditampilkan (* = semua)
FROM tabel                 -- dari tabel mana
WHERE syarat               -- saring baris
ORDER BY kolom DESC        -- urutkan (ASC kecil→besar, DESC
besar→kecil)
LIMIT 5;                   -- batasi jumlah baris
```

Operator di WHERE:

- `= <> > < >= <=` — teks pakai kutip tunggal: `'Jakarta'`

- `AND`, `OR`, `NOT`
- `IN ('Bandung','Surabaya')` — salah satu dari daftar
- `BETWEEN 100000 AND 500000`, `LIKE 'Ra%'` (diawali "Ra")
- `IS NULL` — bukan `= NULL`

> 🧪 **Praktik langsung:** buka **Lab SQL** di beranda dan kerjakan tantangan 1–2. Kamu menulis query sungguhan dan hasilnya dicek otomatis.

🧠 Uji Diri — 8 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

🗄 **Tabel: `pelanggan`**

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

🗄 **Tabel: `produk`**

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

1. **Perintah SQL untuk MENGAMBIL data dari tabel adalah...**

   A. GET
   B. SELECT
   C. FETCH
   D. SHOW DATA

2. 📄 Data: pelanggan

   Hasil query ini adalah...

   ```
   SELECT nama FROM pelanggan WHERE kota = 'Jakarta';
   ```

   A. Rina, Sari
   B. Rina, Sari, Lia
   C. Tono, Lia
   D. Rina

3. 📄 Data: produk

   Hasil query ini adalah...

   ```
   SELECT nama FROM produk WHERE harga > 1000000;
   ```

   A. Laptop, Monitor
   B. Laptop
   C. Laptop, Monitor, Keyboard
   D. Mouse, Keyboard

4. 📄 Data: produk

   Hasil query ini adalah...

   ```
   SELECT nama FROM produk ORDER BY harga DESC LIMIT 1;
   ```

   A. Laptop
   B. Mouse
   C. Monitor
   D. Keyboard

5. **Kata kunci untuk menyaring baris berdasarkan syarat:**
   SELECT * FROM produk ____ kategori = 'Aksesoris';

   Jawab: _____

6. 📄 Data: pelanggan

   Hasil query ini adalah...

   ```
   SELECT COUNT(*) FROM pelanggan WHERE kota IN ('Bandung',
   'Surabaya');
   ```

   A. 2
   B. 3
   C. 4
   D. 5

7. **Nilai teks di SQL ditulis dengan...**

   A. Tanda kutip tunggal: 'Jakarta'
   B. Tanpa kutip: Jakarta
   C. Kurung: (Jakarta)
   D. Pagar: #Jakarta#

8. 📄 Data: produk

   Hasil query ini adalah...

   ```
   SELECT nama FROM produk
   WHERE kategori = 'Komputer' AND harga < 5000000;
   ```

   A. Monitor
   B. Laptop, Monitor
   C. Laptop
   D. Tidak ada

Kunci & Pembahasan · SQL Bab 1

← Kembali ke soal · Awal bab

1. **B. SELECT** — SELECT adalah perintah membaca data. Pola dasarnya: SELECT kolom FROM tabel WHERE syarat. Sekitar 80% pekerjaan data analyst dimulai dari SELECT.
   - **A:** GET bukan perintah SQL (itu istilah di HTTP/API).
   - **C:** FETCH ada, tapi untuk cursor tingkat lanjut, bukan membaca data biasa.
   - **D:** SHOW dipakai di MySQL untuk melihat struktur (SHOW TABLES), bukan mengambil data.

2. **A. Rina, Sari** — WHERE menyaring baris: hanya yang kotanya 'Jakarta'. Di tabel pelanggan itu adalah Rina (id 1) dan Sari (id 3). SELECT nama hanya menampilkan kolom nama.
   - **B:** Lia tinggal di Bandung.
   - **C:** Tono dan Lia tinggal di Bandung.
   - **D:** Sari (id 3) juga tinggal di Jakarta.

3. **A. Laptop, Monitor** — Harga > 1.000.000: Laptop (7.000.000) dan Monitor (2.000.000). Mouse (150.000) dan Keyboard (300.000) tidak lolos.
   - **B:** Monitor 2.000.000 juga lebih dari 1 juta.
   - **C:** Keyboard hanya 300.000.
   - **D:** Itu justru produk yang harganya DI BAWAH 1 juta.

4. **A. Laptop** — ORDER BY harga DESC = urutkan dari harga terbesar (DESCending). LIMIT 1 = ambil baris pertama saja. Jadi hasilnya produk termahal: Laptop. Pola ini sering dipakai untuk 'top 1', 'top 5', dsb.
   - **B:** Mouse paling murah — itu hasilnya jika ORDER BY harga ASC.
   - **C:** Monitor ada di urutan kedua.
   - **D:** Keyboard ada di urutan ketiga.

5. **where** — WHERE diletakkan setelah FROM untuk menyaring baris. Hasil query ini: Mouse dan Keyboard.

6. **B. 3** — IN ('Bandung','Surabaya') sama dengan kota = 'Bandung' OR kota = 'Surabaya'. Yang cocok: Tono (Bandung), Budi (Surabaya), dan Lia (Bandung) → 3.
   - **A:** Lia (id 5) juga tinggal di Bandung.
   - **C:** Hanya 3 pelanggan yang tinggal di Bandung atau Surabaya.
   - **D:** 5 adalah jumlah semua pelanggan — syarat WHERE belum diterapkan.
7. **A. Tanda kutip tunggal: 'Jakarta'** — Standar SQL memakai kutip TUNGGAL untuk teks. Tanpa kutip, database mengira Jakarta adalah nama kolom → error 'no such column'. (Beberapa database menerima kutip ganda, tapi biasakan kutip tunggal.)
   - **B:** Tanpa kutip, Jakarta dianggap nama kolom → error.
   - **C:** Kurung dipakai untuk daftar IN atau subquery.
   - **D:** Format pagar hanya di MS Access untuk tanggal.
8. **A. Monitor** — AND = kedua syarat harus benar. Kategori Komputer: Laptop (7 jt) dan Monitor (2 jt). Yang juga < 5 juta hanya Monitor.
   - **B:** Laptop 7 juta tidak < 5 juta.
   - **C:** Laptop gagal di syarat harga.
   - **D:** Monitor memenuhi kedua syarat.

Agregat, GROUP BY & HAVING — Meringkas Data

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Fungsi agregat meringkas banyak baris menjadi satu angka: `COUNT(*)`, `SUM()`, `AVG()`, `MIN()`, `MAX()`.

GROUP BY = ringkasan *per kelompok*. Ini adalah **pivot table versi SQL**:

```
SELECT pelanggan_id,
       COUNT(*)  AS jumlah_pesanan,
       SUM(qty)  AS total_barang
FROM pesanan
GROUP BY pelanggan_id;
```

pelanggan_id	jumlah_pesanan	total_barang
1	3	5
2	2	3
3	2	3
4	1	5

Aturan wajib: setiap kolom di SELECT harus ada di GROUP BY, atau dibungkus fungsi agregat.

WHERE vs HAVING:

- `WHERE` menyaring **baris** sebelum dikelompokkan.
- `HAVING` menyaring **kelompok** setelah dikelompokkan → `HAVING COUNT(*) >= 2`.

```
Urutan tulis: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
→ LIMIT
```

> 🧪 **Lab SQL:** tantangan 3–4.

🧠 Uji Diri — 10 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

🗄️ Tabel: pesanan

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

🗄️ Tabel: produk

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

1. **Fungsi agregat untuk MENJUMLAHKAN nilai adalah...**

   A. COUNT()
   B. SUM()
   C. AVG()
   D. ADD()

2. 📄 Data: pesanan

   Hasil query ini adalah...

   ```
   SELECT COUNT(*) FROM pesanan;
   ```

   A. 5
   B. 8
   C. 13
   D. 16

3. 📄 Data: pesanan

Hasil query ini adalah...

```
SELECT SUM(qty) FROM pesanan;
```

A. 8
B. 16
C. 13
D. 1

4. 📄 Data: pesanan

Pada hasil query ini, baris untuk pelanggan_id = 1 berisi angka...

```
SELECT pelanggan_id, COUNT(*)
FROM pesanan
GROUP BY pelanggan_id;
```

A. 1
B. 3
C. 5
D. 8

5. 📄 Data: pesanan

Hasil query ini adalah...

```
SELECT produk_id, SUM(qty) AS total_qty
FROM pesanan
GROUP BY produk_id
ORDER BY total_qty DESC
LIMIT 1;
```

A. produk_id 2 — 8
B. produk_id 1 — 2
C. produk_id 3 — 3
D. produk_id 2 — 2

6. **Kenapa query ini ditolak di banyak database (PostgreSQL, SQL Server)?**

```
SELECT pelanggan_id, produk_id, COUNT(*)
FROM pesanan
GROUP BY pelanggan_id;
```

   A. produk_id tidak ada di GROUP BY dan tidak dibungkus fungsi agregat
   B. COUNT(*) tidak boleh dipakai bersama GROUP BY
   C. GROUP BY harus ditulis sebelum FROM
   D. Kurang ORDER BY

7. **Apa beda WHERE dan HAVING?**

   A. WHERE menyaring baris SEBELUM dikelompokkan; HAVING menyaring kelompok SETELAH GROUP BY
   B. Sama saja
   C. HAVING hanya untuk teks
   D. WHERE hanya untuk angka

8. 📄 Data: pesanan

   Hasil query ini adalah...

```
SELECT pelanggan_id
FROM pesanan
GROUP BY pelanggan_id
HAVING COUNT(*) >= 2;
```

   A. 1, 2, 3
   B. 1
   C. 1, 2, 3, 4
   D. 4

9. **Lengkapi urutan klausa yang benar: SELECT ... FROM ... WHERE ... GROUP BY ... ____ ... ORDER BY ...**

   Jawab: _____

10. 📄 Data: produk

Pada hasil query ini, nilai untuk kategori Aksesoris adalah...

```
SELECT kategori, AVG(harga)
FROM produk
GROUP BY kategori;
```

A. 225000
B. 450000
C. 150000
D. 4500000

Kunci & Pembahasan · SQL Bab 2

← Kembali ke soal · Awal bab

1. **B. SUM()** — SUM() menjumlahkan. Keluarga agregat: COUNT (banyaknya baris), SUM (jumlah), AVG (rata-rata), MIN, MAX. Persis seperti di Excel.
   - **A:** COUNT menghitung banyaknya baris, bukan menjumlah nilainya.
   - **C:** AVG menghitung rata-rata.
   - **D:** ADD() bukan fungsi agregat SQL.

2. **B. 8** — COUNT(*) menghitung banyaknya BARIS di tabel pesanan: id 101 sampai 108 → 8 pesanan.
   - **A:** 5 adalah jumlah pelanggan, bukan pesanan.
   - **C:** Tidak ada perhitungan di data yang menghasilkan 13.
   - **D:** 16 adalah total kolom qty — itu hasil SUM(qty).

3. **B. 16** — SUM(qty) menjumlahkan kolom qty: 1+3+2+1+2+5+1+1 = 16 barang terjual.
   - **A:** 8 adalah banyaknya baris (COUNT).
   - **C:** Coba jumlahkan lagi kolom qty.
   - **D:** 1 adalah qty pesanan pertama saja.

4. **B. 3** — GROUP BY mengelompokkan baris yang pelanggan_id-nya sama, lalu COUNT(*) dihitung PER KELOMPOK. Pelanggan 1 punya pesanan 101, 102, dan 107 → 3. Ini setara pivot Excel: Rows = pelanggan_id, Values = Count.
   - **A:** Pelanggan 1 punya lebih dari satu pesanan.
   - **C:** 5 adalah SUM(qty) pelanggan 1 (1+3+1), bukan COUNT.
   - **D:** 8 adalah semua pesanan tanpa dikelompokkan.

5. **A. produk_id 2 — 8** — Total qty per produk: produk 1 = 1+1 = 2, produk 2 = 3+5 = 8, produk 3 = 2+1 = 3, produk 4 = 2+1 = 3. Diurutkan terbesar lalu ambil 1 → produk 2 (Mouse) dengan 8. AS total_qty memberi nama (alias) ke kolom hasil.
   - **B:** Produk 1 justru terkecil (2).
   - **C:** Produk 3 total 3, kalah dari produk 2.
   - **D:** Produk 2 memang benar, tapi qty-nya 8, bukan 2 (2 adalah COUNT pesanannya).

6. **A. produk_id tidak ada di GROUP BY dan tidak dibungkus fungsi agregat** — Aturan GROUP BY: setiap kolom di SELECT harus (1) ada di GROUP BY, atau (2) dibungkus agregat (SUM, COUNT, ...). Pelanggan 1 punya 3 produk berbeda — database tidak tahu produk_id mana yang harus ditampilkan di satu baris. (SQLite dan MySQL lama membiarkannya, tapi hasilnya acak — jangan biasakan.)
   - **B:** COUNT(*) dengan GROUP BY justru pasangan paling umum.
   - **C:** Urutan yang benar: SELECT ... FROM ... WHERE ... GROUP BY ...
   - **D:** ORDER BY opsional.

7. **A. WHERE menyaring baris SEBELUM dikelompokkan; HAVING menyaring kelompok SETELAH GROUP BY** — WHERE bekerja di baris mentah (misal: hanya pesanan qty ≥ 2). HAVING bekerja di hasil kelompok (misal: hanya pelanggan yang COUNT(*) ≥ 2). Agregat seperti COUNT(*) tidak boleh ditaruh di WHERE.
   - **B:** Urutan eksekusinya berbeda — lihat penjelasan.
   - **C:** HAVING bisa untuk angka maupun teks.
   - **D:** WHERE bisa untuk teks maupun angka.

8. **A. 1, 2, 3** — Jumlah pesanan per pelanggan: 1 → 3, 2 → 2, 3 → 2, 4 → 1. HAVING COUNT(*) >= 2 menyisakan pelanggan 1, 2, dan 3. Contoh bisnis: mencari pelanggan yang sudah repeat order.
   - **B:** Pelanggan 2 dan 3 juga punya 2 pesanan (≥ 2 termasuk 2).
   - **C:** Pelanggan 4 hanya punya 1 pesanan.
   - **D:** Pelanggan 4 justru yang TIDAK lolos.

9. **having** — Urutan tulis: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT. Hafalkan urutan ini — query salah urutan langsung error.

10. **A. 225000** — Aksesoris berisi Mouse (150.000) dan Keyboard (300.000). Rata-ratanya (150.000 + 300.000) ÷ 2 = 225.000.
    - **B:** 450.000 adalah jumlahnya (SUM), belum dibagi 2.
    - **C:** Itu harga Mouse saja.
    - **D:** 4.500.000 adalah rata-rata kategori Komputer.

JOIN — Menggabungkan Tabel

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Tabel pesanan hanya menyimpan `pelanggan_id` dan `produk_id`. Untuk laporan yang bisa dibaca manusia, gabungkan tabel dengan JOIN:

```
SELECT o.id, p.nama, pr.nama AS produk, o.qty
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr  ON o.produk_id    = pr.id;
```

Jenis JOIN yang paling sering dipakai:

- **INNER JOIN** (cukup ditulis `JOIN`) → hanya baris yang **punya pasangan** di kedua tabel. Lia (tanpa pesanan) hilang.
- **LEFT JOIN** → **semua** baris tabel kiri tetap ada; yang tidak punya pasangan diisi `NULL`.

Pola emas — mencari yang **tidak punya** pasangan (pelanggan pasif):

```
SELECT p.nama
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id
WHERE o.id IS NULL;      -- hasil: Lia
```

> 💡 Gabungkan JOIN + GROUP BY untuk pertanyaan bisnis: "berapa total belanja tiap pelanggan?" → `SUM(o.qty * pr.harga) … GROUP BY p.nama`.

> 🧪 **Lab SQL:** tantangan 5–7.

🧠 Uji Diri — 10 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

Tabel: pelanggan

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

Tabel: pesanan

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

Tabel: produk

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

1. **JOIN dipakai untuk...**

   A. Menggabungkan baris dari dua tabel berdasarkan kolom yang berhubungan
   B. Menggabungkan dua database
   C. Menambah baris baru
   D. Menyalin tabel

2. 📄 Data: pelanggan, pesanan

Hasil query ini adalah...

```
SELECT p.nama
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
WHERE o.id = 105;
```

A. Rina
B. Sari
C. Tono
D. Budi

3. 📄 Data: pelanggan, pesanan

Hasil query ini adalah...

```
SELECT COUNT(*)
FROM pelanggan p
JOIN pesanan o ON o.pelanggan_id = p.id;
```

A. 5
B. 8
C. 9
D. 4

4. 📄 Data: pelanggan, pesanan

Hasil query ini adalah...

```
SELECT COUNT(*)
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id;
```

A. 8
B. 9
C. 5
D. 13

5. **Query yang benar untuk mencari pelanggan yang BELUM PERNAH memesan:**

   A. SELECT p.nama FROM pelanggan p LEFT JOIN pesanan o ON o.pelanggan_id = p.id WHERE o.id IS NULL;
   B. SELECT p.nama FROM pelanggan p JOIN pesanan o ON o.pelanggan_id = p.id WHERE o.id IS NULL;
   C. SELECT nama FROM pelanggan WHERE id = NULL;
   D. SELECT nama FROM pesanan WHERE qty = 0;

6. 📄 Data: pelanggan, pesanan

   Siapa nama pelanggan yang dihasilkan query 'belum pernah memesan' tadi?

   Jawab: _____

7. **Kata kunci untuk menulis syarat penghubung dua tabel pada JOIN:**

   A. ON
   B. WHERE
   C. WITH
   D. LINK

8. 📄 Data: pelanggan, pesanan, produk

   Hasil query ini adalah...

```
SELECT p.nama, SUM(o.qty * pr.harga) AS belanja
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr ON o.produk_id = pr.id
GROUP BY p.nama
ORDER BY belanja DESC
LIMIT 1;
```

   A. Rina — 9450000
   B. Sari — 7600000
   C. Rina — 9000000
   D. Tono — 4300000

9. 📄 Data: pelanggan, pesanan, produk

   Query sebelumnya ditambah WHERE pr.kategori = 'Aksesoris' (sebelum GROUP BY). Siapa juaranya?

   A. Budi — 750000
   B. Sari — 600000
   C. Rina — 450000
   D. Budi — 5

10. **Kenapa kita memakai alias seperti p, o, dan pr?**

    A. Supaya query lebih pendek dan jelas kolom berasal dari tabel mana (p.id vs o.id)
    B. Karena wajib di semua query
    C. Supaya query 10x lebih cepat
    D. Untuk mengganti nama tabel secara permanen

Kunci & Pembahasan · SQL Bab 3

← Kembali ke soal · Awal bab

1. **A. Menggabungkan baris dari dua tabel berdasarkan kolom yang berhubungan** — Tabel pesanan hanya menyimpan pelanggan_id (angka). Untuk tahu NAMA pelanggannya, gabungkan dengan tabel pelanggan lewat pesanan.pelanggan_id = pelanggan.id. Konsepnya sama seperti XLOOKUP di Excel, tapi untuk banyak kolom sekaligus.
   - **B:** JOIN bekerja antar tabel di database yang sama.
   - **C:** Menambah baris memakai INSERT.
   - **D:** Menyalin tabel memakai CREATE TABLE ... AS SELECT.
2. **B. Sari** — Pesanan 105 punya pelanggan_id = 3. JOIN mencocokkannya dengan pelanggan id 3 = Sari. 'o' dan 'p' adalah alias (nama pendek) untuk tabel pesanan dan pelanggan.
   - **A:** Rina adalah pelanggan id 1.
   - **C:** Tono adalah pelanggan id 2.
   - **D:** Budi adalah pelanggan id 4.
3. **B. 8** — (INNER) JOIN hanya menampilkan pasangan yang COCOK di kedua tabel. Setiap pesanan punya tepat satu pelanggan → 8 baris. Lia tidak punya pesanan, jadi Lia tidak muncul.
   - **A:** 5 adalah jumlah pelanggan, tapi satu pelanggan bisa muncul berkali-kali.
   - **C:** 9 adalah hasil LEFT JOIN (8 + Lia yang tanpa pesanan).
   - **D:** 4 adalah banyaknya pelanggan yang pernah memesan.
4. **B. 9** — LEFT JOIN mempertahankan SEMUA baris tabel kiri (pelanggan), walaupun tidak punya pasangan. 8 baris pesanan yang cocok + 1 baris Lia (kolom pesanannya NULL) = 9.
   - **A:** 8 adalah hasil INNER JOIN — Lia belum dihitung.
   - **C:** Setiap pelanggan bisa muncul lebih dari sekali.
   - **D:** 13 = 5 + 8, tapi JOIN tidak menumpuk tabel seperti itu.

5. **A. SELECT p.nama FROM pelanggan p LEFT JOIN pesanan o ON o.pelanggan_id = p.id WHERE o.id IS NULL;** — Pola klasik 'anti-join': LEFT JOIN mempertahankan semua pelanggan, lalu WHERE o.id IS NULL menyisakan yang tidak punya pasangan pesanan. Di bisnis ini dipakai untuk mencari pelanggan pasif yang perlu di-follow up tim sales.
   - **B:** INNER JOIN sudah membuang pelanggan tanpa pesanan, jadi IS NULL tidak akan menemukan apa pun.
   - **C:** Membandingkan dengan NULL harus memakai IS NULL, bukan = NULL. Lagi pula id pelanggan tidak ada yang NULL.
   - **D:** Tabel pesanan tidak punya kolom nama, dan tidak ada qty 0.
6. **lia** — pelanggan_id yang muncul di pesanan: 1, 2, 3, 4. Id 5 (Lia) tidak pernah muncul → Lia belum pernah memesan.
7. **A. ON** — JOIN pelanggan p ON o.pelanggan_id = p.id. ON menjelaskan kolom mana yang menghubungkan kedua tabel. Lupa menulis ON (atau salah kolom) bisa membuat hasilnya berlipat ganda.
   - **B:** WHERE untuk menyaring hasil, bukan menghubungkan tabel (walau versi lama SQL memakai cara itu).
   - **C:** WITH dipakai untuk CTE (query bernama).
   - **D:** LINK bukan kata kunci SQL.
8. **A. Rina — 9450000** — Tiga tabel digabung: pesanan → pelanggan (nama) dan pesanan → produk (harga). Belanja Rina: 1×7.000.000 + 3×150.000 + 1×2.000.000 = 9.450.000. Sari: 7.000.000 + 2×300.000 = 7.600.000. Rina terbesar. Inilah 'pelanggan paling berharga' — query yang sangat sering diminta tim sales.
   - **B:** Sari di posisi kedua.
   - **C:** Hampir — jangan lupa 3 Mouse milik Rina (3 × 150.000 = 450.000).
   - **D:** Tono di posisi ketiga.

9. **A. Budi — 750000** — Hanya pesanan Aksesoris yang dihitung: Rina 3 Mouse = 450.000; Sari 2 Keyboard = 600.000; Budi 5 Mouse = 750.000; Tono 1 Keyboard = 300.000. Budi juara — padahal di semua kategori Budi paling kecil. Filter bisa mengubah cerita.
    - **B:** Sari kedua dengan 600.000.
    - **C:** Rina ketiga di kategori ini.
    - **D:** 5 adalah qty Budi, bukan nilai rupiahnya.
10. **A. Supaya query lebih pendek dan jelas kolom berasal dari tabel mana (p.id vs o.id)** — Tabel pelanggan dan pesanan sama-sama punya kolom id. Tanpa awalan tabel, database bingung ('ambiguous column'). Alias membuat penulisan awalan itu singkat.
    - **B:** Alias opsional — tapi sangat membantu saat JOIN.
    - **C:** Alias tidak memengaruhi kecepatan.
    - **D:** Alias hanya berlaku selama query itu dijalankan.

Subquery — Query di Dalam Query

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Subquery menjawab pertanyaan kecil dulu (di dalam kurung), lalu hasilnya dipakai query luar.

1. Subquery satu nilai — dibandingkan dengan `= > <`:

```
SELECT nama FROM produk
WHERE harga > (SELECT AVG(harga) FROM produk);   -- Laptop
```

2. Subquery daftar — dipakai dengan `IN` / `NOT IN`:

```
SELECT nama FROM pelanggan
WHERE id IN (SELECT pelanggan_id FROM pesanan WHERE qty >= 3);
```

3. Subquery di FROM — tabel sementara yang diberi nama:

```
SELECT AVG(jml) FROM (
  SELECT pelanggan_id, COUNT(*) AS jml
  FROM pesanan GROUP BY pelanggan_id
) AS t;
```

4. Correlated subquery — memakai kolom dari query luar (dihitung per baris):

```
SELECT nama FROM produk p
WHERE harga = (SELECT MAX(harga) FROM produk
               WHERE kategori = p.kategori);
```

🤔 Kapan subquery, kapan JOIN? Banyak masalah bisa diselesaikan dengan keduanya. Pilih yang paling mudah dibaca timmu. Di wawancara, menjelaskan alasan pilihanmu sering lebih dinilai daripada jawabannya.

🧪 **Lab SQL:** tantangan 8–10.

🧠 Uji Diri — 9 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

Tabel: produk

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

Tabel: pelanggan

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

Tabel: pesanan

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

1. **Subquery adalah...**

   A. Query di dalam query lain, ditulis di dalam tanda kurung
   B. Query yang dijalankan dua kali
   C. Tabel cadangan
   D. Query tanpa FROM

2. 📄 Data: produk

Hasil query ini adalah...

```
SELECT nama FROM produk
WHERE harga > (SELECT AVG(harga) FROM produk);
```

A. Laptop
B. Laptop, Monitor
C. Monitor
D. Semua produk

3. 📄 Data: pelanggan, pesanan

Hasil query ini adalah...

```
SELECT nama FROM pelanggan
WHERE id IN (SELECT pelanggan_id FROM pesanan WHERE qty >=
3);
```

A. Rina, Budi
B. Budi
C. Rina, Tono, Budi
D. Lia

4. 📄 Data: pelanggan, pesanan

Hasil query ini adalah...

```
SELECT nama FROM pelanggan
WHERE id NOT IN (SELECT pelanggan_id FROM pesanan);
```

A. Lia
B. Budi
C. Tono, Lia
D. Kosong

5. 📄 Data: pesanan, produk

   Hasil query ini adalah...

```
SELECT id FROM pesanan
WHERE produk_id IN (
  SELECT id FROM produk WHERE kategori = 'Komputer'
);
```

   A. 101, 103, 104, 107
   B. 101, 104
   C. 102, 105, 106, 108
   D. 103, 107

6. 📄 Data: pesanan

   Hasil query ini adalah...

```
SELECT AVG(jml) FROM (
  SELECT pelanggan_id, COUNT(*) AS jml
  FROM pesanan
  GROUP BY pelanggan_id
) AS t;
```

   A. 2
   B. 1.6
   C. 8
   D. 4

7. **Operator untuk mengecek apakah sebuah nilai ADA dalam daftar hasil subquery: WHERE id ____ (SELECT ...)**

   Jawab: _____

8. **Subquery berkorelasi (correlated subquery) artinya...**

   A. Subquery yang memakai kolom dari query luar, sehingga dihitung ulang untuk tiap baris
   B. Subquery yang berada di FROM
   C. Subquery yang selalu menghasilkan satu nilai
   D. Subquery yang error

9. 📄 Data: produk

Hasil query ini adalah...

```
SELECT nama FROM produk p
WHERE harga = (
  SELECT MAX(harga) FROM produk
  WHERE kategori = p.kategori
);
```

A. Laptop, Keyboard
B. Laptop
C. Laptop, Monitor
D. Mouse, Monitor

Kunci & Pembahasan · SQL Bab 4

← Kembali ke soal · Awal bab

1. **A. Query di dalam query lain, ditulis di dalam tanda kurung** — Subquery menjawab 'pertanyaan kecil' dulu, lalu hasilnya dipakai query luar. Contoh: produk yang harganya di atas (rata-rata harga semua produk). Bagian dalam kurung dihitung lebih dulu.
   - **B:** Subquery biasa dijalankan sekali (kecuali jenis correlated).
   - **C:** Subquery tidak menyimpan tabel apa pun.
   - **D:** Subquery biasanya tetap punya FROM.

2. **A. Laptop** — Subquery dihitung dulu: rata-rata harga = (7.000.000 + 150.000 + 2.000.000 + 300.000) ÷ 4 = 2.362.500. Produk dengan harga > 2.362.500 hanya Laptop. Monitor (2.000.000) masih di bawah rata-rata.
   - **B:** Monitor 2.000.000 masih di bawah rata-rata 2.362.500.
   - **C:** Monitor di bawah rata-rata.
   - **D:** Tidak mungkin semua produk di atas rata-ratanya sendiri.

3. **A. Rina, Budi** — Subquery: pesanan dengan qty ≥ 3 adalah 102 (pelanggan 1, qty 3) dan 106 (pelanggan 4, qty 5) → daftar (1, 4). Query luar mengambil nama pelanggan id 1 dan 4: Rina dan Budi.
   - **B:** Rina juga punya pesanan qty 3 (≥ 3 termasuk 3).
   - **C:** Pesanan Tono hanya qty 2 dan 1.
   - **D:** Lia tidak punya pesanan sama sekali.

4. **A. Lia** — Subquery menghasilkan (1, 2, 3, 4) — semua pelanggan yang pernah memesan. NOT IN mengambil yang tidak ada di daftar itu: Lia (id 5). Ini cara kedua (selain LEFT JOIN … IS NULL) untuk mencari pelanggan pasif.
   - **B:** Budi pernah memesan (pesanan 106).
   - **C:** Tono pernah memesan (103, 108).
   - **D:** Lia cocok dengan syaratnya.

5. **A. 101, 103, 104, 107** — Subquery: produk kategori Komputer = id 1 (Laptop) dan 3 (Monitor). Pesanan dengan produk_id 1 atau 3: 101, 103, 104, 107.
   - **B:** Itu hanya pesanan Laptop. Monitor (produk 3) juga Komputer.
   - **C:** Itu justru pesanan Aksesoris.
   - **D:** Itu hanya pesanan Monitor.

6. **A. 2** — Subquery di FROM membuat tabel sementara: jumlah pesanan per pelanggan = 3, 2, 2, 1. Lalu AVG = (3 + 2 + 2 + 1) ÷ 4 = 2. Lia tidak ikut, karena ia tidak ada di tabel pesanan.
   - **B:** 1.6 = 8 ÷ 5, seolah-olah Lia (0 pesanan) ikut dihitung. Tapi subquery hanya membaca tabel pesanan.
   - **C:** 8 adalah total pesanan, belum dirata-rata.
   - **D:** 4 adalah banyaknya pelanggan di tabel sementara.

7. **in** — IN mengecek keanggotaan dalam daftar. Lawannya NOT IN. Untuk subquery yang menghasilkan satu nilai saja, kamu juga bisa memakai =, >, <.

8. **A. Subquery yang memakai kolom dari query luar, sehingga dihitung ulang untuk tiap baris** — Contoh: WHERE harga = (SELECT MAX(harga) FROM produk WHERE kategori = p.kategori). Bagian p.kategori berasal dari query luar, jadi untuk setiap produk, MAX dihitung untuk kategorinya sendiri. Ampuh, tapi bisa lambat di data besar.
   - **B:** Subquery di FROM disebut derived table.
   - **C:** Itu disebut scalar subquery.
   - **D:** Correlated subquery adalah teknik yang valid.

9. **A. Laptop, Keyboard** — Untuk setiap produk, subquery mencari harga termahal DI KATEGORINYA SENDIRI. Komputer → max 7.000.000 (Laptop). Aksesoris → max 300.000 (Keyboard). Hasilnya: produk termahal per kategori.
   - **B:** Keyboard juga termahal di kategori Aksesoris.
   - **C:** Monitor bukan yang termahal di kategori Komputer.
   - **D:** Itu justru yang termurah/bukan termahal di kategorinya.

CASE WHEN — Logika IF di dalam SQL

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

CASE WHEN adalah IF bertingkat versi SQL. Syarat diperiksa dari atas, yang **pertama benar** dipakai.

```
SELECT nama, harga,
  CASE WHEN harga >= 2000000 THEN 'Mahal'
       WHEN harga >= 200000  THEN 'Sedang'
       ELSE 'Murah'
  END AS label
FROM produk;
```

Tiga kegunaan yang paling sering dipakai di kantor:

1. **Memberi label/segmen** — rentang harga, segmen pelanggan (VIP/Reguler/Baru), status (Telat/Tepat).
2. **Conditional aggregation** — membuat tabel ala pivot di SQL:

```
SELECT p.nama,
  SUM(CASE WHEN pr.kategori = 'Komputer'  THEN o.qty ELSE 0
END) AS qty_komputer,
  SUM(CASE WHEN pr.kategori = 'Aksesoris' THEN o.qty ELSE 0
END) AS qty_aksesoris
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr  ON o.produk_id = pr.id
GROUP BY p.nama;
```

3. **Urutan khusus** — `ORDER BY CASE WHEN kota = 'Bandung' THEN 0 ELSE 1 END`.

> ⚠️ Dua jebakan: (1) urutan WHEN — tulis syarat paling ketat lebih dulu; (2) tanpa ELSE, baris yang tidak cocok menjadi NULL.

> 🧪 **Lab SQL:** tantangan 11 dan 18.

🧠 Uji Diri — 11 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

Tabel: `produk`

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

Tabel: `pesanan`

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

Tabel: `pelanggan`

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

1. **CASE WHEN di SQL paling mirip dengan fungsi apa di Excel?**

   A. IF / IFS
   B. SUMIFS
   C. XLOOKUP
   D. CONCAT

2. 📄 Data: produk

Label untuk Monitor (harga 2.000.000) adalah...

```
SELECT nama,
  CASE WHEN harga >= 2000000 THEN 'Mahal'
       ELSE 'Murah' END AS label
FROM produk;
```

A. Mahal
B. Murah
C. NULL
D. Error

3. 📄 Data: produk

Hasil query ini adalah...

```
SELECT CASE WHEN harga >= 2000000 THEN 'Mahal'
            ELSE 'Murah' END AS label,
       COUNT(*) AS jumlah
FROM produk
GROUP BY label;
```

A. Mahal 2, Murah 2
B. Mahal 1, Murah 3
C. Mahal 3, Murah 1
D. Error: GROUP BY tidak boleh memakai alias

4. 📄 Data: pesanan

Pesanan 106 punya qty 5. Nilai kolom ukuran untuknya adalah...

```
SELECT id, qty,
  CASE WHEN qty >= 1 THEN 'Kecil'
       WHEN qty >= 5 THEN 'Besar'
  END AS ukuran
FROM pesanan WHERE id = 106;
```

A. Kecil
B. Besar
C. NULL
D. Kecil dan Besar

5. 📄 Data: pelanggan

Hasil query ini untuk Budi (kota Surabaya) adalah...

```
SELECT nama,
  CASE WHEN kota = 'Jakarta' THEN 'Pusat' END AS wilayah
FROM pelanggan WHERE nama = 'Budi';
```

A. NULL
B. Pusat
C. '' (teks kosong)
D. Cabang

6. 📄 Data: pesanan, produk

Produk id 1 & 3 = Komputer, id 2 & 4 = Aksesoris. Hasil query ini adalah...

```
SELECT
  SUM(CASE WHEN produk_id IN (1,3) THEN qty ELSE 0 END) AS
qty_komputer,
  SUM(CASE WHEN produk_id IN (2,4) THEN qty ELSE 0 END) AS
qty_aksesoris
FROM pesanan;
```

A. 5 dan 11
B. 11 dan 5
C. 16 dan 16
D. 4 dan 4

7. 📄 Data: pesanan

Hasil query ini adalah...

```
SELECT COUNT(CASE WHEN qty >= 2 THEN 1 END) AS pesanan_besar
FROM pesanan;
```

A. 4
B. 8
C. 12
D. 3

8. **Setiap CASE harus ditutup dengan kata kunci ___**

Jawab: _____

9. 📄 Data: pelanggan

Nama di baris PERTAMA hasil query ini adalah...

```
SELECT nama, kota
FROM pelanggan
ORDER BY CASE WHEN kota = 'Bandung' THEN 0 ELSE 1 END, nama;
```

A. Lia
B. Tono
C. Budi
D. Rina

10. 📄 Data: pelanggan, pesanan, produk

Berapa pelanggan yang masuk segmen VIP? (Belanja: Rina 9,45 jt; Sari 7,6 jt; Tono 4,3 jt; Budi 0,75 jt)

```
SELECT p.nama,
  SUM(o.qty * pr.harga) AS belanja,
  CASE WHEN SUM(o.qty * pr.harga) >= 5000000 THEN 'VIP'
       WHEN SUM(o.qty * pr.harga) >= 1000000 THEN 'Reguler'
       ELSE 'Baru' END AS segmen
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr  ON o.produk_id = pr.id
GROUP BY p.nama;
```

A. 2
B. 1
C. 3
D. 4

11. **Mana yang BUKAN kegunaan CASE WHEN?**

A. Menggabungkan dua tabel berdasarkan id
B. Memberi label/kategori (misal Mahal/Murah)
C. Membuat kolom ala pivot dengan SUM(CASE ...)
D. Membuat urutan khusus di ORDER BY

Kunci & Pembahasan · SQL Bab 5

← Kembali ke soal · Awal bab

1. **A. IF / IFS** — CASE WHEN syarat THEN hasil ... ELSE hasil_lain END bekerja seperti IF bertingkat atau IFS: syarat diperiksa berurutan, yang pertama benar yang dipakai.
   - **B:** SUMIFS menjumlah dengan syarat; di SQL padanannya SUM + WHERE/GROUP BY (atau SUM(CASE...)).
   - **C:** XLOOKUP padanannya JOIN.
   - **D:** CONCAT untuk menyambung teks.

2. **A. Mahal** — Syaratnya >= (lebih besar ATAU SAMA DENGAN). 2.000.000 >= 2.000.000 bernilai benar, jadi labelnya 'Mahal'.
   - **B:** Kalau syaratnya > (tanpa =), barulah Monitor jadi 'Murah'.
   - **C:** NULL hanya muncul jika tidak ada syarat yang cocok DAN tidak ada ELSE.
   - **D:** Sintaksnya benar: CASE ... WHEN ... THEN ... ELSE ... END.

3. **A. Mahal 2, Murah 2** — CASE membuat kolom kategori baru, lalu GROUP BY menghitung per kategori. Mahal: Laptop & Monitor. Murah: Mouse & Keyboard. Ini cara membuat 'bucket' seperti rentang harga atau segmen pelanggan.
   - **B:** Monitor (tepat 2 juta) masuk 'Mahal' karena syaratnya >=.
   - **C:** Hanya Laptop dan Monitor yang ≥ 2 juta.
   - **D:** SQLite, MySQL, dan PostgreSQL mengizinkan alias di GROUP BY. (SQL Server tidak — di sana tulis ulang CASE-nya di GROUP BY.)

4. **A. Kecil** — CASE berhenti di syarat PERTAMA yang benar. 5 >= 1 sudah benar, jadi hasilnya 'Kecil' dan WHEN kedua tidak pernah diperiksa. Aturan: tulis syarat paling ketat lebih dulu (qty >= 5 di atas qty >= 1). Sama persis dengan IF bertingkat di Excel.
   - **B:** Seharusnya 'Besar' — tapi urutan WHEN-nya terbalik, jadi WHEN kedua tidak pernah tercapai.
   - **C:** Ada syarat yang cocok (qty >= 1), jadi tidak NULL.
   - **D:** CASE hanya mengembalikan satu nilai.

5. **A. NULL** — Tanpa ELSE, baris yang tidak cocok dengan WHEN mana pun bernilai NULL. Biasakan selalu menulis ELSE supaya tidak ada nilai yang 'hilang' di laporan.
   - **B:** Budi tinggal di Surabaya, bukan Jakarta.
   - **C:** NULL berbeda dengan teks kosong.
   - **D:** Tidak ada kata 'Cabang' di query.

6. **A. 5 dan 11** — Ini 'conditional aggregation': CASE memilih qty yang dihitung, sisanya 0. Komputer: 1+2+1+1 = 5 (pesanan 101, 103, 104, 107). Aksesoris: 3+2+5+1 = 11. Teknik ini membuat tabel ala pivot (kategori jadi kolom) langsung di SQL.
   - **B:** Kebalikannya — Mouse (id 2) terjual paling banyak, dan Mouse itu Aksesoris.
   - **C:** 16 adalah total semua qty; CASE memisahkannya per kategori.
   - **D:** 4 adalah BANYAKNYA pesanan per kategori, bukan jumlah qty-nya.

7. **A. 4** — Untuk baris dengan qty < 2, CASE tanpa ELSE menghasilkan NULL — dan COUNT(kolom) tidak menghitung NULL. Jadi yang terhitung hanya pesanan dengan qty ≥ 2: 102 (3), 103 (2), 105 (2), 106 (5) → 4. Mirip COUNTIF di Excel.
   - **B:** 8 adalah semua pesanan — COUNT(*) yang akan menghasilkan 8.
   - **C:** 12 adalah jumlah qty dari pesanan ≥ 2 (3+2+2+5), bukan banyaknya pesanan.
   - **D:** Ada empat pesanan dengan qty ≥ 2.

8. **end** — Bentuk lengkapnya: CASE WHEN ... THEN ... ELSE ... END. Lupa END adalah syntax error paling sering saat menulis CASE.

9. **A. Lia** — CASE di ORDER BY membuat urutan khusus: Bandung diberi angka 0 (paling depan), kota lain 1. Lalu di dalam kelompok yang sama diurutkan berdasarkan nama. Bandung: Lia, Tono → Lia paling atas.
   - **B:** Tono juga Bandung, tapi di dalam kelompoknya diurutkan berdasarkan nama: L sebelum T.
   - **C:** Budi bukan Bandung, jadi mendapat angka 1 dan berada setelah semua pelanggan Bandung.
   - **D:** Rina bukan Bandung.

10. **A. 2** — Belanja ≥ 5 juta: Rina (9,45 jt) dan Sari (7,6 jt) → 2 VIP. Tono = Reguler, Budi = Baru. Segmentasi berbasis aturan seperti ini adalah versi sederhana dari segmentasi di skripsimu (yang memakai clustering).
    - **B:** Sari (7,6 jt) juga ≥ 5 jt.
    - **C:** Tono (4,3 jt) masuk Reguler, belum VIP.
    - **D:** Budi hanya 750 ribu → Baru.
11. **A. Menggabungkan dua tabel berdasarkan id** — Menggabungkan tabel adalah tugas JOIN. CASE dipakai untuk label, conditional aggregation (pivot), dan urutan khusus.
    - **B:** Ini kegunaan paling umum CASE.
    - **C:** Ini disebut conditional aggregation.
    - **D:** CASE bisa dipakai di ORDER BY.

NULL & Fungsi Bawaan (Teks, Angka, Tanggal)

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

NULL = nilainya tidak diketahui. Bukan 0, bukan teks kosong. Tiga aturan yang wajib diingat:

- Cek dengan `IS NULL` / `IS NOT NULL` — `= NULL` tidak pernah benar.
- NULL merambat: `NULL + 5 = NULL`.
- `COUNT(*)` menghitung baris; `COUNT(kolom)` **tidak** menghitung NULL.

Jebakan klasik setelah LEFT JOIN — hitung pesanan per pelanggan, termasuk yang 0:

```
SELECT p.nama,
       COUNT(o.id)              AS jumlah_pesanan,  -- Lia = 0 ✓
(COUNT(*) → 1 ✗)
       COALESCE(SUM(o.qty), 0)  AS total_qty        -- Lia = 0 ✓
(tanpa COALESCE → NULL)
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id
GROUP BY p.nama;
```

Fungsi bawaan (padanannya di Excel):

SQL	Excel	Contoh
`UPPER / LOWER`	UPPER / LOWER	UPPER('jakarta') → JAKARTA
`LENGTH`	LEN	LENGTH('Keyboard') → 8
`SUBSTR(x, 6, 2)`	MID	'2026-01-12' → '01'
`TRIM`	TRIM	buang spasi tepi
`||` atau `CONCAT()`	&	nama || ' (' || kota || ')'
`ROUND(x, 2)`	ROUND	pembulatan
`COALESCE(x, 0)`	IFERROR/IF kosong	NULL → 0
`COUNT(DISTINCT x)`	COUNTA(UNIQUE())	jumlah nilai unik

Fungsi tanggal berbeda per database — ambil bulan: MySQL `MONTH(tgl)`, PostgreSQL `EXTRACT(MONTH FROM tgl)`, SQLite `strftime('%m', tgl)`. Saat tes kerja, tanyakan databasenya; logikanya tetap sama.

> 🧪 **Lab SQL:** tantangan 12–13.

🧠 Uji Diri — 13 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

🗄 Tabel: `pelanggan`

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

🗄 Tabel: `pesanan`

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

🗄 Tabel: `produk`

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

1. **Di SQL, NULL artinya...**

   A. Nilai tidak diketahui / tidak ada — bukan 0 dan bukan teks kosong
   B. Angka nol
   C. Teks kosong ''
   D. Error

2. **Seandainya ada pelanggan yang kotanya NULL, berapa baris hasil query ini?**

```
SELECT nama FROM pelanggan WHERE kota = NULL;
```

A. Selalu 0 baris
B. Semua pelanggan yang kotanya NULL
C. Semua pelanggan
D. Error

3. 📄 Data: pelanggan, pesanan

Lia belum pernah memesan. Nilai jml untuk Lia adalah…

```
SELECT p.nama, COUNT(*) AS jml
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id
GROUP BY p.nama;
```

A. 1
B. 0
C. NULL
D. Lia tidak muncul

4. 📄 Data: pelanggan, pesanan

Tanpa COALESCE, nilai total_qty untuk Lia adalah… (dengan COALESCE(SUM(o.qty), 0) nilainya jadi 0)

```
SELECT p.nama, SUM(o.qty) AS total_qty
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id
GROUP BY p.nama;
```

A. NULL
B. 0
C. 1
D. Error

5. **Hasil query ini adalah...**

```
SELECT NULL + 5;
```

A. NULL
B. 5
C. 0
D. Error

6. **Hasil query ini adalah...**

```
SELECT UPPER('jakarta'), LENGTH('Keyboard');
```

A. JAKARTA dan 8
B. Jakarta dan 8
C. JAKARTA dan 7
D. jakarta dan 9

7. **Hasil query ini adalah...**

```
SELECT SUBSTR('2026-01-12', 6, 2);
```

A. 01
B. 20
C. 12
D. -0

8. 📄 Data: pelanggan

Hasil query ini (SQLite/PostgreSQL) adalah...

```
SELECT nama || ' (' || kota || ')'
FROM pelanggan WHERE id = 1;
```

A. Rina (Jakarta)
B. Rina||(Jakarta)
C. RinaJakarta
D. Error

9. 📄 Data: pesanan

   Hasil query ini adalah...

   ```
   SELECT COUNT(DISTINCT pelanggan_id) FROM pesanan;
   ```

   A. 4
   B. 8
   C. 5
   D. 3

10. 📄 Data: produk

    Hasil query ini adalah...

    ```
    SELECT COUNT(*) FROM produk WHERE nama LIKE 'M%';
    ```

    A. 2
    B. 1
    C. 3
    D. 0

11. **Fungsi untuk mengambil BULAN dari kolom tanggal berbeda-beda per database. Yang benar untuk MySQL:**

    A. MONTH(tanggal)
    B. strftime('%m', tanggal)
    C. EXTRACT(MONTH FROM tanggal) — hanya ada di MySQL
    D. DATEPART(tanggal)

12. 📄 Data: produk

    Hasil query ini adalah...

    ```
    SELECT AVG(harga) FROM produk;
    ```

    A. 2.362.500
    B. 9.450.000
    C. 2.000.000
    D. 1.150.000

13. **Fungsi yang mengambil nilai pertama yang BUKAN NULL, misalnya ____(SUM(qty), 0)**

    Jawab: _____

Kunci & Pembahasan · SQL Bab 6

← Kembali ke soal · Awal bab

1. **A. Nilai tidak diketahui / tidak ada — bukan 0 dan bukan teks kosong** — NULL = 'tidak ada nilainya'. Karena itu NULL tidak bisa dibandingkan dengan = dan merambat dalam perhitungan (NULL + 5 = NULL).
   - **B:** 0 adalah nilai yang diketahui (nol).
   - **C:** '' adalah teks yang diketahui (kosong).
   - **D:** NULL adalah nilai yang sah, bukan error.

2. **A. Selalu 0 baris** — Membandingkan apa pun dengan NULL memakai = hasilnya 'tidak diketahui', bukan benar — sehingga tidak ada baris yang lolos. Cara yang benar: WHERE kota IS NULL (atau IS NOT NULL).
   - **B:** Itulah yang diinginkan, tapi harus ditulis IS NULL.
   - **C:** WHERE tetap menyaring; hanya saja tidak ada yang lolos.
   - **D:** Query ini tidak error — justru itu bahayanya: diam-diam menghasilkan 0 baris.

3. **A. 1** — LEFT JOIN tetap menghasilkan SATU baris untuk Lia (kolom pesanannya NULL). COUNT(*) menghitung baris, jadi Lia = 1 — salah secara bisnis! Pakai COUNT(o.id): COUNT(kolom) tidak menghitung NULL, sehingga Lia = 0. Ini jebakan yang sering muncul di tes SQL.
   - **B:** 0 adalah hasil yang BENAR secara bisnis, tapi butuh COUNT(o.id), bukan COUNT(*).
   - **C:** COUNT tidak pernah menghasilkan NULL.
   - **D:** LEFT JOIN mempertahankan semua pelanggan, termasuk Lia.

4. **A. NULL** — SUM dari nilai yang semuanya NULL hasilnya NULL. COALESCE(nilai, pengganti) mengambil nilai pertama yang bukan NULL, jadi COALESCE(SUM(o.qty), 0) mengubah NULL menjadi 0 — jauh lebih enak dibaca di laporan.
   - **B:** 0 baru muncul setelah dibungkus COALESCE.
   - **C:** 1 adalah hasil COUNT(*), bukan SUM.
   - **D:** Query ini valid.

5. **A. NULL** — NULL merambat: operasi aritmetika dengan NULL hasilnya NULL. Kalau sebuah kolom diskon bisa NULL, tulis harga - COALESCE(diskon, 0) supaya hasilnya tidak ikut NULL.
   - **B:** NULL tidak dianggap 0 dalam aritmetika.
   - **C:** NULL bukan 0.
   - **D:** Query ini valid.

6. **A. JAKARTA dan 8** — UPPER mengubah ke huruf besar (seperti UPPER di Excel). LENGTH menghitung karakter (seperti LEN di Excel): K-e-y-b-o-a-r-d = 8.
   - **B:** Kapital di awal saja bukan hasil UPPER.
   - **C:** 'Keyboard' terdiri dari 8 huruf.
   - **D:** UPPER mengubah semua huruf menjadi kapital.

7. **A. 01** — SUBSTR(teks, posisi_awal, panjang) — seperti MID di Excel. Posisi 1–4 = '2026', posisi 5 = '-', posisi 6–7 = '01' (bulan).
   - **B:** '20' adalah dua karakter pertama (posisi 1).
   - **C:** '12' ada di posisi 9–10 (tanggalnya).
   - **D:** Posisi 5 adalah '-', tapi query mulai dari posisi 6.

8. **A. Rina (Jakarta)** — Di SQLite dan PostgreSQL, || menyambung teks (seperti & di Excel). Di MySQL pakai CONCAT(nama, ' (', kota, ')'). Tiap database punya dialek sedikit berbeda — tapi konsepnya sama.
   - **B:** || adalah operator, bukan teks yang ikut tampil.
   - **C:** Spasi dan tanda kurung ditulis di dalam kutip, jadi ikut tampil.
   - **D:** Query valid di SQLite dan PostgreSQL.

9. **A. 4** — DISTINCT membuang duplikat sebelum dihitung. pelanggan_id di pesanan: 1,1,2,3,3,4,1,2 → unik: 1, 2, 3, 4 = 4 pelanggan yang pernah memesan. Setara =COUNTA(UNIQUE(...)) di Excel.
   - **B:** 8 adalah jumlah pesanan (COUNT(*)).
   - **C:** Ada 5 pelanggan, tapi Lia belum pernah memesan.
   - **D:** Ada 4 pelanggan_id berbeda.

10. **A. 2** — LIKE 'M%' = diawali huruf M, lalu apa saja (% = karakter apa pun berapa pun banyaknya). Cocok: Mouse dan Monitor.
    - **B:** Mouse DAN Monitor sama-sama diawali M.
    - **C:** Hanya dua produk yang diawali M.
    - **D:** Ada produk yang diawali M.
11. **A. MONTH(tanggal)** — MySQL: MONTH(tanggal). PostgreSQL: EXTRACT(MONTH FROM tanggal) atau DATE_TRUNC('month', tanggal). SQLite (dipakai Lab di aplikasi ini): strftime('%m', tanggal). SQL Server: MONTH() atau DATEPART(month, tanggal). Saat tes kerja, tanyakan database apa yang dipakai.
    - **B:** Itu versi SQLite.
    - **C:** EXTRACT adalah standar SQL dan ada di PostgreSQL; MySQL juga mendukungnya, tapi bukan 'hanya ada di MySQL'.
    - **D:** DATEPART milik SQL Server dan butuh argumen bagian tanggal: DATEPART(month, tanggal).
12. **A. 2.362.500** — (7.000.000 + 150.000 + 2.000.000 + 300.000) ÷ 4 = 9.450.000 ÷ 4 = 2.362.500. Rata-rata ini 'tertarik' ke atas oleh harga Laptop — di bab statistik kamu akan belajar kapan median lebih jujur.
    - **B:** Itu jumlahnya (SUM), belum dibagi 4.
    - **C:** 2.000.000 adalah harga Monitor.
    - **D:** Itu median-nya ((300.000 + 2.000.000) ÷ 2), bukan rata-rata.
13. **coalesce** — COALESCE(a, b, c, ...) mengembalikan argumen pertama yang tidak NULL. Paling sering dipakai untuk mengganti NULL dengan 0 atau 'Tidak ada'. MySQL juga punya IFNULL, SQL Server punya ISNULL.

CTE (WITH) & UNION — Query Bertahap yang Rapi

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Query nyata sering butuh beberapa langkah. **CTE** membuat langkah-langkah itu bernama dan terbaca dari atas ke bawah — seperti kolom bantu di Excel.

```
WITH belanja AS (                          -- langkah 1: total per
pelanggan
  SELECT p.nama, SUM(o.qty * pr.harga) AS total
  FROM pesanan o
  JOIN pelanggan p ON o.pelanggan_id = p.id
  JOIN produk pr  ON o.produk_id = pr.id
  GROUP BY p.nama
),
rata AS (                                  -- langkah 2: rata-
ratanya
  SELECT AVG(total) AS nilai FROM belanja
)
SELECT b.nama, b.total                     -- langkah 3: yang di
atas rata-rata
FROM belanja b, rata r
WHERE b.total > r.nilai;                   -- hasil: Rina, Sari
```

- `WITH` ditulis sekali; CTE berikutnya dipisah koma.
- CTE hanya berlaku untuk satu query itu.
- Debug: jalankan `SELECT * FROM belanja` untuk mengecek tiap langkah.

UNION menumpuk hasil query ke **bawah** (JOIN menyambung ke samping):

```
SELECT nama, 'pelanggan' AS jenis FROM pelanggan
UNION ALL
SELECT nama, 'produk' FROM produk;
```

`UNION` membuang duplikat, `UNION ALL` tidak. Syarat: jumlah kolom sama dan tipenya cocok per posisi. Contoh nyata: menggabungkan tabel penjualan 2025 dan 2026.

> 🧪 **Lab SQL:** tantangan 14.

🧠 Uji Diri — 11 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

🗄 Tabel: `pesanan`

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

🗄 Tabel: `produk`

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

🗄 Tabel: `pelanggan`

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

1. **Kegunaan utama CTE (WITH nama AS (...) SELECT ...) adalah...**

   A. Memecah query panjang menjadi langkah-langkah bernama yang mudah dibaca
   B. Membuat tabel permanen di database
   C. Mempercepat semua query secara otomatis
   D. Menghapus data duplikat

2. 📄 Data: pesanan, produk

Hasil query ini adalah...

```
WITH belanja AS (
  SELECT o.pelanggan_id, SUM(o.qty * pr.harga) AS total
  FROM pesanan o
  JOIN produk pr ON o.produk_id = pr.id
  GROUP BY o.pelanggan_id
)
SELECT AVG(total) FROM belanja;
```

A. 5.525.000
B. 4.420.000
C. 22.100.000
D. 2.762.500

3. **Cara menulis DUA CTE dalam satu query yang benar:**

A. WITH a AS (...), b AS (...) SELECT ...
B. WITH a AS (...) WITH b AS (...) SELECT ...
C. WITH a, b AS (...) SELECT ...
D. CTE hanya boleh satu per query

4. 📄 Data: pelanggan, pesanan, produk

Siapa saja hasil query ini?

```
WITH belanja AS (
  SELECT p.nama, SUM(o.qty * pr.harga) AS total
  FROM pesanan o
  JOIN pelanggan p ON o.pelanggan_id = p.id
  JOIN produk pr  ON o.produk_id = pr.id
  GROUP BY p.nama
)
SELECT nama FROM belanja
WHERE total > (SELECT AVG(total) FROM belanja);
```

A. Rina dan Sari
B. Rina saja
C. Rina, Sari, dan Tono
D. Semua pelanggan

5. 📄 Data: pesanan, produk

Hasil query ini adalah...

```
WITH ringkas AS (
  SELECT produk_id, SUM(qty) AS total_qty
  FROM pesanan GROUP BY produk_id
)
SELECT pr.nama
FROM ringkas r JOIN produk pr ON pr.id = r.produk_id
ORDER BY r.total_qty DESC
LIMIT 1;
```

A. Mouse
B. Laptop
C. Monitor
D. Keyboard

6. **Apa beda CTE dengan subquery?**

A. Hasilnya bisa sama; CTE lebih mudah dibaca dan bisa dipakai berkali-kali dalam query yang sama
B. CTE selalu jauh lebih cepat
C. Subquery tidak bisa dipakai di WHERE
D. CTE hanya ada di Excel

7. **Beda UNION dan UNION ALL:**

A. UNION membuang baris duplikat, UNION ALL mempertahankan semua baris
B. UNION ALL membuang duplikat
C. UNION menggabungkan kolom ke samping
D. Tidak ada bedanya

8. 📄 Data: pelanggan

   Berapa baris hasil query ini?

   ```
   SELECT kota FROM pelanggan
   UNION
   SELECT kota FROM pelanggan;
   ```

   A. 3
   B. 10
   C. 5
   D. 6

9. **Syarat agar dua query bisa digabung dengan UNION:**

   A. Jumlah kolomnya sama dan tipe datanya cocok per posisi
   B. Nama tabelnya harus sama
   C. Harus ada JOIN
   D. Harus ada GROUP BY

10. **Query dengan 3 CTE hasilnya aneh. Cara debug paling efektif:**

    A. Jalankan tiap CTE satu per satu (SELECT * FROM cte_1, lalu cte_2, ...) dan cek hasil antaranya
    B. Tulis ulang semuanya dari nol
    C. Tambahkan DISTINCT di mana-mana
    D. Hapus semua WHERE

11. **Kata kunci untuk memulai CTE adalah ____**

    Jawab: _____

Kunci & Pembahasan · SQL Bab 7

← Kembali ke soal · Awal bab

1. **A. Memecah query panjang menjadi langkah-langkah bernama yang mudah dibaca** — CTE (Common Table Expression) seperti kolom bantu di Excel: hitung sesuatu dulu, beri nama, lalu pakai di langkah berikutnya. Hanya hidup selama satu query itu.
   - **B:** CTE bersifat sementara; tabel permanen dibuat dengan CREATE TABLE.
   - **C:** Tujuannya keterbacaan, bukan kecepatan.
   - **D:** Itu tugas DISTINCT.

2. **A. 5.525.000** — Langkah 1 (CTE): total belanja per pelanggan → 9.450.000, 4.300.000, 7.600.000, 750.000 (4 pelanggan). Langkah 2: rata-ratanya = 22.100.000 ÷ 4 = 5.525.000. Lia tidak ikut karena tidak punya pesanan.
   - **B:** Itu kalau dibagi 5 (termasuk Lia). Lia tidak ada di CTE karena tidak punya pesanan.
   - **C:** Itu total semua belanja, belum dirata-rata.
   - **D:** Itu rata-rata per PESANAN (22,1 jt ÷ 8), bukan per pelanggan.

3. **A. WITH a AS (...), b AS (...) SELECT ...** — Kata WITH ditulis sekali, lalu CTE dipisah dengan koma. CTE kedua boleh memakai CTE pertama — sehingga query bisa dibaca seperti resep langkah demi langkah.
   - **B:** WITH kedua menyebabkan syntax error.
   - **C:** Setiap CTE perlu definisinya sendiri.
   - **D:** Boleh banyak, dipisah koma.

4. **A. Rina dan Sari** — Rata-rata belanja = 5.525.000. Yang di atasnya: Rina (9.450.000) dan Sari (7.600.000). Perhatikan CTE 'belanja' dipakai DUA kali — tanpa CTE, query join yang panjang itu harus ditulis dua kali.
   - **B:** Sari (7,6 jt) juga di atas rata-rata 5,525 jt.
   - **C:** Tono (4,3 jt) di bawah rata-rata.
   - **D:** WHERE menyaring yang di atas rata-rata saja.

5. **A. Mouse** — Total qty per produk: Laptop 2, Mouse 8 (3 + 5), Monitor 3, Keyboard 3. Diurutkan menurun dan diambil 1 → Mouse. Paling laku secara jumlah unit, walau pendapatannya kecil — beda 'terlaris' dan 'paling menguntungkan' sering ditanyakan saat interview.
   - **B:** Laptop paling besar pendapatannya, tapi qty-nya hanya 2.
   - **C:** Monitor terjual 3 unit.
   - **D:** Keyboard terjual 3 unit.
6. **A. Hasilnya bisa sama; CTE lebih mudah dibaca dan bisa dipakai berkali-kali dalam query yang sama** — Secara hasil, banyak CTE bisa ditulis ulang sebagai subquery. Bedanya di keterbacaan: logika dibaca dari atas ke bawah, bukan dari dalam ke luar. Di tes kerja, query yang rapi dan mudah dicek memberi nilai tambah.
   - **B:** Kecepatan tergantung database dan query-nya; tidak selalu berbeda.
   - **C:** Subquery justru sering dipakai di WHERE.
   - **D:** CTE adalah fitur SQL.
7. **A. UNION membuang baris duplikat, UNION ALL mempertahankan semua baris** — Keduanya menumpuk hasil query ke BAWAH (seperti Append di Power Query). UNION menghapus duplikat (lebih lambat), UNION ALL tidak. Pakai UNION ALL jika kamu yakin tidak ada duplikat atau duplikat memang perlu dihitung.
   - **B:** Kebalikannya.
   - **C:** Menggabungkan ke samping adalah tugas JOIN.
   - **D:** Bedanya ada di penanganan duplikat.
8. **A. 3** — Kedua query menghasilkan 5 baris, ditumpuk jadi 10, lalu UNION membuang duplikat → tersisa kota unik: Jakarta, Bandung, Surabaya = 3.
   - **B:** 10 adalah hasil UNION ALL.
   - **C:** 5 adalah hasil satu query saja.
   - **D:** Hanya ada 3 kota berbeda.

9. **A. Jumlah kolomnya sama dan tipe datanya cocok per posisi** — UNION menumpuk berdasarkan POSISI kolom: kolom ke-1 dengan kolom ke-1, dan seterusnya. Nama kolom hasil diambil dari query pertama.
    - **B:** Tabelnya boleh berbeda, misal data penjualan 2025 dan 2026.
    - **C:** UNION dan JOIN adalah operasi yang berbeda.
    - **D:** GROUP BY tidak diperlukan.
10. **A. Jalankan tiap CTE satu per satu (SELECT * FROM cte_1, lalu cte_2, ...) dan cek hasil antaranya** — Sama seperti memeriksa kolom bantu di Excel: cek hasil tiap langkah. Biasanya masalahnya ketahuan di langkah yang jumlah barisnya tiba-tiba melonjak (JOIN dobel) atau hilang (INNER JOIN yang seharusnya LEFT JOIN).
    - **B:** Membuang waktu dan bisa mengulang kesalahan yang sama.
    - **C:** DISTINCT sering hanya menutupi masalah JOIN yang dobel.
    - **D:** Menghapus syarat malah mengubah arti query.
11. **with** — WITH nama_cte AS (SELECT ...) SELECT ... FROM nama_cte; — CTE hanya berlaku untuk satu query yang mengikutinya.

Window Function — Peringkat, Total Berjalan & Pertumbuhan

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Window function menghitung per kelompok **tanpa** menggabungkan baris. Ini topik yang membedakan analis pemula dan analis siap kerja, dan sering muncul di tes teknis.

```
fungsi() OVER (PARTITION BY kelompok ORDER BY urutan)
```

Kebutuhan bisnis	Window function
Peringkat sales/pelanggan	`RANK() OVER (ORDER BY belanja DESC)`
Top 1 per kategori	`ROW_NUMBER() OVER (PARTITION BY kategori ORDER BY harga DESC)` lalu `WHERE rn = 1` (di CTE)
Total berjalan	`SUM(x) OVER (ORDER BY tanggal)`
Porsi dari total	`x * 100.0 / SUM(x) OVER ()`
Bulan lalu (untuk growth)	`LAG(x) OVER (ORDER BY bulan)`
Rata-rata bergerak 7 hari	`AVG(x) OVER (ORDER BY tgl ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`

Tiga fungsi peringkat untuk nilai 300, 200, 200, 100:

```
ROW_NUMBER → 1, 2, 3, 4   (selalu unik)
RANK       → 1, 2, 2, 4   (kembar sama, lalu meloncat)
DENSE_RANK → 1, 2, 2, 3   (kembar sama, tanpa loncatan)
```

⚠️ Window function dihitung **setelah WHERE**. Untuk menyaring hasilnya (misal `rn = 1`), bungkus dulu dengan CTE. Dan ingat: `5 / 16 = 0` di SQLite/PostgreSQL — pakai `100.0`.

🧪 **Lab SQL:** tantangan 15–17.

🧠 Uji Diri — 12 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

🗄️ Tabel: pesanan

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

🗄️ Tabel: produk

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

1. **Beda utama window function (... OVER (...)) dengan GROUP BY:**

   A. Window function menghitung per kelompok TANPA menggabungkan baris — setiap baris tetap tampil
   B. Window function lebih cepat
   C. GROUP BY tidak bisa menghitung SUM
   D. Window function hanya untuk tanggal

2. **Kata kunci yang menandai sebuah window function, misal SUM(qty) ____ (PARTITION BY pelanggan_id)**

   Jawab: _____

3. 📄 Data: pesanan

   Berapa nilai rnk untuk pesanan dengan qty 1?

```
SELECT id, qty,
  RANK() OVER (ORDER BY qty DESC) AS rnk
FROM pesanan;
```

   A. 5
   B. 4
   C. 2
   D. 8

4. **Untuk nilai 300, 200, 200, 100 (diurutkan menurun), DENSE_RANK menghasilkan...**

   A. 1, 2, 2, 3
   B. 1, 2, 2, 4
   C. 1, 2, 3, 4
   D. 1, 1, 1, 1

5. 📄 Data: pesanan

   Nilai kumulatif pada baris tanggal 2026-01-08 adalah...

```
SELECT tanggal, qty,
  SUM(qty) OVER (ORDER BY tanggal) AS kumulatif
FROM pesanan;
```

   A. 9
   B. 2
   C. 7
   D. 16

6. 📄 Data: pesanan

   Nilai total_pelanggan untuk pesanan 101 (milik pelanggan 1) adalah...

```
SELECT id, pelanggan_id, qty,
  SUM(qty) OVER (PARTITION BY pelanggan_id) AS total_pelanggan
FROM pesanan;
```

   A. 5
   B. 1
   C. 16
   D. 3

7. 📄 Data: pesanan

Nilai persen untuk pesanan 106 (qty 5) adalah...

```
SELECT id, qty,
  qty * 100.0 / SUM(qty) OVER () AS persen
FROM pesanan;
```

A. 31.25
B. 5
C. 100
D. 31

8. **Kenapa query persen ditulis qty * 100.0, bukan qty * 100?**

A. Di SQLite/PostgreSQL, bilangan bulat ÷ bilangan bulat dibulatkan ke bawah (500 / 16 = 31); 100.0 memaksa hasil desimal
B. Supaya hasilnya lebih besar
C. 100.0 lebih cepat dihitung
D. Tidak ada bedanya di database mana pun

9. 📄 Data: produk

Untuk kategori Aksesoris, produk apa yang muncul di hasil query ini?

```
WITH r AS (
  SELECT kategori, nama, harga,
    ROW_NUMBER() OVER (PARTITION BY kategori ORDER BY harga
DESC) AS rn
  FROM produk
)
SELECT kategori, nama FROM r WHERE rn = 1;
```

A. Keyboard
B. Mouse
C. Laptop
D. Keyboard dan Mouse

10. **Kenapa query sebelumnya memakai CTE, bukan langsung WHERE ROW_NUMBER() OVER (...) = 1?**

    A. Window function dihitung SETELAH WHERE, jadi tidak bisa dipakai di WHERE pada query yang sama
    B. CTE wajib untuk semua query
    C. ROW_NUMBER hanya bisa dipakai di CTE
    D. Supaya hasilnya diurutkan

11. 📄 Data: pesanan

    Nilai qty_sebelumnya pada baris tanggal 2026-01-05 adalah...

    ```
    SELECT tanggal, qty,
      LAG(qty) OVER (ORDER BY tanggal) AS qty_sebelumnya
    FROM pesanan;
    ```

    A. 3
    B. 2
    C. 1
    D. NULL

12. **AVG(pendapatan) OVER (ORDER BY tanggal ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) menghitung...**

    A. Rata-rata bergerak 7 hari (hari ini + 6 hari sebelumnya)
    B. Rata-rata seluruh tabel
    C. Rata-rata 6 hari ke depan
    D. Peringkat pendapatan

Kunci & Pembahasan · SQL Bab 8

← Kembali ke soal · Awal bab

1. **A. Window function menghitung per kelompok TANPA menggabungkan baris — setiap baris tetap tampil** — GROUP BY meringkas 8 pesanan menjadi 4 baris (per pelanggan). Window function tetap menampilkan 8 baris, tapi setiap baris bisa 'melihat' kelompoknya: total pelanggan, peringkat, total berjalan, dan nilai baris sebelumnya.
   - **B:** Bukan soal kecepatan, tapi bentuk hasil.
   - **C:** GROUP BY jelas bisa SUM.
   - **D:** Window function bisa untuk data apa pun.

2. **over** — OVER (...) menentukan 'jendela' baris: PARTITION BY = kelompoknya, ORDER BY = urutannya. OVER () yang kosong berarti jendelanya seluruh tabel.

3. **A. 5** — Urutan qty: 5, 3, 2, 2, 1, 1, 1, 1. RANK memberi peringkat sama untuk nilai kembar lalu MELONCAT: 1, 2, 3, 3, 5, 5, 5, 5. Jadi qty 1 berperingkat 5 (peringkat 4 dilewati karena ada dua pesanan di peringkat 3).
   - **B:** 4 adalah hasil DENSE_RANK, yang tidak meloncat.
   - **C:** Peringkat 2 milik qty 3.
   - **D:** 8 adalah hasil ROW_NUMBER untuk baris terakhir.

4. **A. 1, 2, 2, 3** — Tiga fungsi peringkat: ROW_NUMBER = selalu unik (1, 2, 3, 4). RANK = kembar sama, lalu meloncat (1, 2, 2, 4) — seperti klasemen olahraga. DENSE_RANK = kembar sama, tanpa loncatan (1, 2, 2, 3).
   - **B:** Itu RANK (meloncat ke 4).
   - **C:** Itu ROW_NUMBER (tidak peduli nilai kembar).
   - **D:** Hanya terjadi jika semua nilainya sama.

5. **A. 9** — SUM ... OVER (ORDER BY tanggal) = total berjalan (running total): 01-03: 1 → 01-04: 4 → 01-05: 6 → 01-07: 7 → 01-08: 9. Dipakai untuk grafik 'penjualan kumulatif bulan ini vs target'.
   - **B:** 2 adalah qty di tanggal itu saja.
   - **C:** 7 adalah kumulatif sampai 01-07.
   - **D:** 16 adalah total di baris terakhir.

6. **A. 5** — PARTITION BY = hitung per kelompok, tapi baris tetap utuh. Pelanggan 1 punya pesanan 101 (1), 102 (3), dan 107 (1) → 5. Nilai 5 ini muncul di ketiga baris milik pelanggan 1. Berguna untuk menghitung 'porsi pesanan ini dari total pelanggannya'.
   - **B:** 1 adalah qty pesanan 101 saja.
   - **C:** 16 adalah total semua pesanan (OVER () tanpa PARTITION).
   - **D:** 3 adalah qty pesanan 102.
7. **A. 31.25** — SUM(qty) OVER () = total seluruh tabel = 16, tanpa menggabungkan baris. 5 × 100 ÷ 16 = 31,25%. Pola 'porsi dari total' ini sangat sering dipakai di laporan (kontribusi produk, kontribusi sales).
   - **B:** 5 adalah qty-nya, belum dibagi total.
   - **C:** 100 hanya jika pesanan ini satu-satunya baris (misalnya karena WHERE id = 106 ikut menyaring jendelanya).
   - **D:** 31 adalah hasil pembagian bilangan bulat — itulah alasan query memakai 100.0, bukan 100.
8. **A. Di SQLite/PostgreSQL, bilangan bulat ÷ bilangan bulat dibulatkan ke bawah (500 / 16 = 31); 100.0 memaksa hasil desimal** — Pembagian integer adalah jebakan klasik: 1 / 3 = 0 di SQLite dan PostgreSQL. Ubah salah satu angka jadi desimal (100.0) atau pakai CAST(qty AS REAL). (MySQL kebetulan selalu menghasilkan desimal.)
   - **B:** Nilainya sama; bedanya hanya di bagian desimal yang tidak hilang.
   - **C:** Kecepatannya praktis sama.
   - **D:** Bedanya nyata di SQLite dan PostgreSQL.
9. **A. Keyboard** — ROW_NUMBER dimulai dari 1 di setiap kategori, diurutkan dari harga termahal. Aksesoris: Keyboard (300.000) = 1, Mouse (150.000) = 2. Mengambil rn = 1 = 'produk termahal per kategori'. Pola 'top-N per kelompok' ini SANGAT sering muncul di tes SQL.
   - **B:** Mouse lebih murah, jadi rn = 2.
   - **C:** Laptop adalah juara kategori Komputer.
   - **D:** rn = 1 hanya satu baris per kategori.

10. **A. Window function dihitung SETELAH WHERE, jadi tidak bisa dipakai di WHERE pada query yang sama** — Urutan logis SQL: FROM → WHERE → GROUP BY → HAVING → window function → SELECT → ORDER BY. Karena WHERE dijalankan lebih dulu, ia belum 'tahu' nilai ROW_NUMBER. Solusinya: hitung di CTE/subquery, lalu saring di query luar.
    - **B:** CTE tidak wajib untuk semua query.
    - **C:** ROW_NUMBER bisa dipakai di SELECT mana pun; yang tidak bisa adalah menyaringnya di WHERE yang sama.
    - **D:** Pengurutan adalah tugas ORDER BY.
11. **A. 3** — LAG mengambil nilai dari baris SEBELUMNYA. Baris sebelum 01-05 adalah 01-04 dengan qty 3. Baris pertama (01-03) bernilai NULL karena tidak punya baris sebelumnya. LAG adalah alat utama untuk menghitung pertumbuhan bulan-ke-bulan: (bulan_ini − LAG(bulan_ini)) / LAG(bulan_ini).
    - **B:** 2 adalah qty di baris 01-05 itu sendiri.
    - **C:** 1 adalah qty di baris 01-03.
    - **D:** NULL hanya untuk baris PERTAMA.
12. **A. Rata-rata bergerak 7 hari (hari ini + 6 hari sebelumnya)** — Frame ROWS BETWEEN ... menentukan baris mana yang ikut dihitung. Moving average 7 hari meratakan naik-turun harian (misal efek akhir pekan) sehingga tren lebih terlihat di dashboard.
    - **B:** Rata-rata seluruh tabel ditulis AVG(x) OVER ().
    - **C:** PRECEDING artinya baris SEBELUMNYA.
    - **D:** Peringkat memakai RANK/ROW_NUMBER.

Tantangan Lab SQL

Tulis query-nya di kertas atau di Lab SQL aplikasi, lalu cocokkan dengan jawaban di bawah. Tabel yang dipakai:

Tabel: `pelanggan`

id	nama	kota
1	Rina	Jakarta
2	Tono	Bandung
3	Sari	Jakarta
4	Budi	Surabaya
5	Lia	Bandung

Tabel: `produk`

id	nama	kategori	harga
1	Laptop	Komputer	7000000
2	Mouse	Aksesoris	150000
3	Monitor	Komputer	2000000
4	Keyboard	Aksesoris	300000

Tabel: `pesanan`

id	pelanggan_id	produk_id	qty	tanggal
101	1	1	1	2026-01-03
102	1	2	3	2026-01-04
103	2	3	2	2026-01-05
104	3	1	1	2026-01-07
105	3	4	2	2026-01-08
106	4	2	5	2026-01-09
107	1	3	1	2026-01-10
108	2	4	1	2026-01-12

1. **Pelanggan dari Bandung. Tampilkan kolom nama dan kota dari semua pelanggan yang tinggal di Bandung.**
2. **Produk dari termahal. Tampilkan nama dan harga semua produk, diurutkan dari harga termahal ke termurah.**
3. **Jumlah pesanan per pelanggan. Untuk setiap pelanggan_id di tabel pesanan, tampilkan berapa banyak pesanannya.**
4. **Produk laris (HAVING). Tampilkan produk_id dan total qty yang terjual, hanya untuk produk dengan total qty minimal 3.**

5. **Pesanan + nama pelanggan. Tampilkan id pesanan dan nama pelanggan untuk setiap pesanan.**

6. **Total belanja per pelanggan. Tampilkan nama pelanggan dan total belanjanya (qty × harga), urut dari belanja terbesar. Hanya pelanggan yang pernah memesan.**

7. **Pelanggan pasif. Tampilkan nama pelanggan yang belum pernah memesan sama sekali.**

8. **Produk di atas rata-rata. Tampilkan nama dan harga produk yang harganya lebih tinggi dari rata-rata harga semua produk.**

9. **Pembeli produk Komputer. Tampilkan nama pelanggan (tanpa duplikat) yang pernah membeli produk berkategori Komputer.**

10. **Pendapatan per kota. Tampilkan kota pelanggan dan total pendapatan dari kota itu (qty × harga), urut dari terbesar.**

11. **Label harga (CASE). Tampilkan nama, harga, dan kolom label untuk semua produk: 'Premium' jika harga ≥ 2.000.000, selain itu 'Reguler'.**

12. **Semua pelanggan + jumlah pesanan. Tampilkan nama SEMUA pelanggan dan jumlah pesanannya — pelanggan yang belum pernah memesan harus tampil dengan angka 0.**

13. **Total barang, 0 jika belum pesan. Tampilkan nama SEMUA pelanggan dan total qty yang pernah mereka pesan. Yang belum pernah memesan harus bernilai 0, bukan NULL.**

14. **Pelanggan di atas rata-rata (CTE). Tampilkan nama dan total belanja (qty × harga) pelanggan yang belanjanya di atas rata-rata belanja pelanggan yang pernah memesan.**

15. **Peringkat pelanggan (RANK). Tampilkan nama, total belanja, dan peringkat (1 = belanja terbesar) untuk pelanggan yang pernah memesan, diurutkan dari peringkat 1.**

16. **Total berjalan per tanggal. Tampilkan tanggal, nilai pesanan (qty × harga), dan total berjalan (kumulatif) nilai pesanan, diurutkan dari tanggal paling awal.**

17. **Produk termahal per kategori. Tampilkan kategori, nama, dan harga produk TERMAHAL di setiap kategori (satu baris per kategori).**

18. **Pivot qty per kategori (CASE). Untuk setiap pelanggan yang pernah memesan, tampilkan nama, qty_komputer, dan qty_aksesoris (jumlah qty per kategori produk).**

Jawaban Lab SQL

1. ```
   SELECT nama, kota
   FROM pelanggan
   WHERE kota = 'Bandung';
   ```

   WHERE menyaring baris sebelum ditampilkan. Hasilnya Tono dan Lia.

2. ```
   SELECT nama, harga
   FROM produk
   ORDER BY harga DESC;
   ```

   DESC = descending (menurun). Tantangan ini mengecek URUTAN baris, karena urutan adalah inti pertanyaannya.

3. ```
   SELECT pelanggan_id, COUNT(*) AS jumlah_pesanan
   FROM pesanan
   GROUP BY pelanggan_id;
   ```

   GROUP BY mengelompokkan baris dengan pelanggan_id yang sama, lalu COUNT(*) dihitung per kelompok — seperti pivot table dengan Rows = pelanggan_id.

4. ```
   SELECT produk_id, SUM(qty) AS total_qty
   FROM pesanan
   GROUP BY produk_id
   HAVING SUM(qty) >= 3;
   ```

   WHERE tidak bisa memakai SUM() karena dijalankan sebelum pengelompokan. HAVING menyaring setelah GROUP BY. Hasil: produk 2 (8), 3 (3), dan 4 (3).

5. ```
   SELECT o.id, p.nama
   FROM pesanan o
   JOIN pelanggan p ON o.pelanggan_id = p.id;
   ```

   JOIN menyambungkan pelanggan_id di tabel pesanan dengan id di tabel pelanggan — seperti XLOOKUP untuk mengambil nama.
   ```

6.
```
SELECT p.nama, SUM(o.qty * pr.harga) AS belanja
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr ON o.produk_id = pr.id
GROUP BY p.nama
ORDER BY belanja DESC;
```

Ini query 'pelanggan paling berharga' yang sering diminta tim sales. Hasil: Rina 9.450.000, Sari 7.600.000, Tono 4.300.000, Budi 750.000.

7.
```
SELECT p.nama
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id
WHERE o.id IS NULL;
```

Pola anti-join. Hasilnya Lia — target follow-up untuk tim sales.

8.
```
SELECT nama, harga
FROM produk
WHERE harga > (SELECT AVG(harga) FROM produk);
```

Subquery dihitung dulu (rata-rata = 2.362.500), lalu dipakai sebagai pembanding. Hasil: hanya Laptop.

9.
```
SELECT nama
FROM pelanggan
WHERE id IN (
  SELECT o.pelanggan_id
  FROM pesanan o
  JOIN produk pr ON o.produk_id = pr.id
  WHERE pr.kategori = 'Komputer'
);
```

IN otomatis tidak menghasilkan duplikat di query luar. Kalau memakai JOIN, tambahkan DISTINCT. Hasil: Rina, Tono, Sari.

10.
```
SELECT p.kota, SUM(o.qty * pr.harga) AS pendapatan
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr ON o.produk_id = pr.id
GROUP BY p.kota
ORDER BY pendapatan DESC;
```

Jakarta 17.050.000, Bandung 4.300.000, Surabaya 750.000. Pertanyaan seperti ini (pendapatan per wilayah) sangat sering muncul di tes teknis data analyst.

11.
```
SELECT nama, harga,
  CASE WHEN harga >= 2000000 THEN 'Premium'
       ELSE 'Reguler' END AS label
FROM produk;
```

Laptop & Monitor = Premium, Mouse & Keyboard = Reguler. Label seperti ini memudahkan filter di dashboard.

12.
```
SELECT p.nama, COUNT(o.id) AS jumlah_pesanan
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id
GROUP BY p.nama;
```

COUNT(*) akan memberi Lia nilai 1 (baris hasil LEFT JOIN-nya ikut terhitung). COUNT(o.id) tidak menghitung NULL, jadi Lia = 0.

13.
```
SELECT p.nama, COALESCE(SUM(o.qty), 0) AS total_qty
FROM pelanggan p
LEFT JOIN pesanan o ON o.pelanggan_id = p.id
GROUP BY p.nama;
```

SUM dari nilai NULL semua hasilnya NULL. COALESCE mengubahnya menjadi 0. Hasil: Rina 5, Tono 3, Sari 3, Budi 5, Lia 0.

14.
```
WITH belanja AS (
  SELECT p.nama, SUM(o.qty * pr.harga) AS total
  FROM pesanan o
  JOIN pelanggan p ON o.pelanggan_id = p.id
  JOIN produk pr  ON o.produk_id = pr.id
  GROUP BY p.nama
)
SELECT nama, total
FROM belanja
WHERE total > (SELECT AVG(total) FROM belanja);
```

Rata-rata = 5.525.000. Di atasnya: Rina (9.450.000) dan Sari (7.600.000).

15.
```
SELECT p.nama, SUM(o.qty * pr.harga) AS belanja,
  RANK() OVER (ORDER BY SUM(o.qty * pr.harga) DESC) AS
peringkat
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr  ON o.produk_id = pr.id
GROUP BY p.nama
ORDER BY peringkat;
```

Rina 1, Sari 2, Tono 3, Budi 4. Window function dihitung setelah GROUP BY, sehingga bisa memeringkat hasil agregat.

16.
```
SELECT o.tanggal, o.qty * pr.harga AS nilai,
  SUM(o.qty * pr.harga) OVER (ORDER BY o.tanggal) AS kumulatif
FROM pesanan o
JOIN produk pr ON o.produk_id = pr.id
ORDER BY o.tanggal;
```

Kumulatif naik dari 7.000.000 sampai 22.100.000 di tanggal terakhir. Ini data untuk grafik 'pencapaian vs target' bulanan.

17.
```
WITH r AS (
  SELECT kategori, nama, harga,
    ROW_NUMBER() OVER (PARTITION BY kategori ORDER BY harga
DESC) AS rn
  FROM produk
)
SELECT kategori, nama, harga
FROM r
WHERE rn = 1;
```

Komputer → Laptop, Aksesoris → Keyboard. Pola top-N per kelompok ini favorit pewawancara.

18.
```
SELECT p.nama,
  SUM(CASE WHEN pr.kategori = 'Komputer'  THEN o.qty ELSE 0
END) AS qty_komputer,
  SUM(CASE WHEN pr.kategori = 'Aksesoris' THEN o.qty ELSE 0
END) AS qty_aksesoris
FROM pesanan o
JOIN pelanggan p ON o.pelanggan_id = p.id
JOIN produk pr  ON o.produk_id = pr.id
GROUP BY p.nama;
```

Rina 2 & 3, Tono 2 & 1, Sari 1 & 2, Budi 0 & 5. Inilah pivot table yang ditulis dengan SQL.